CHAPTER 10

Minimization or Maximization of Functions

10.0 Introduction

In a nutshell: You are given a single function f that depends on one or more independent variables. You want to find the value of those variables where f takes on a maximum or a minimum value. You can then calculate what value of f is achieved at the maximum or minimum. The tasks of maximization and minimization are trivially related to each other, since one person's function f could just as well be another's $-f$. The computational desiderata are the usual ones: Do it quickly, cheaply, and in small memory. Often the computational effort is dominated by the cost of evaluating f (and also perhaps its partial derivatives with respect to all variables, if the chosen algorithm requires them). In such cases the desiderata are sometimes replaced by the simple surrogate: Evaluate f as few times as possible.

An extremum (maximum or minimum point) can be either *global* (truly the highest or lowest function value) or *local* (the highest or lowest in a finite neighborhood and not on the boundary of that neighborhood). (See Figure 10.0.1.) Finding a global extremum is, in general, a very difficult problem. Two standard heuristics are widely used: (i) Find local extrema starting from widely varying starting values of the independent variables (perhaps chosen quasi-randomly, as in §7.8), and then pick the most extreme of these (if they are not all the same); or (ii) perturb a local extremum by taking a finite amplitude step away from it, and then see if your routine returns you to a better point, or "always" to the same one. More recently, so-called *simulated annealing methods* (§10.12) have demonstrated important successes on a variety of global extremization problems.

Our chapter title could just as well be *optimization*, which is the usual name for this very large field of numerical research. The importance ascribed to the various tasks in this field depends strongly on the particular interests of whom you talk to. Economists, and some engineers, are particularly concerned with *constrained optimization*, where there are a priori limitations on the allowed values of independent variables. For example, the production of wheat in the United States must be a non-

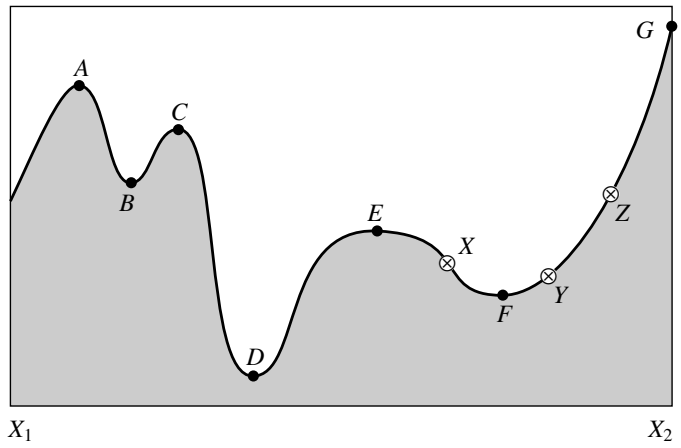


Figure 10.0.1. Extrema of a function in an interval. Points A, C, and E are local, but not global, maxima. Points B and F are local, but not global, minima. The global maximum occurs at G, which is on the boundary of the interval so that the derivative of the function need not vanish there. The global minimum is at D. At point E, derivatives higher than the first vanish, a situation that can cause difficulty for some algorithms. The points X, Y, and Z are said to *bracket* the minimum F, since Y is less than both X and Z.

negative number. One particularly well-developed area of constrained optimization is *linear programming*, where both the function to be optimized and the constraints happen to be linear functions of the independent variables. Sections 10.10 and 10.11, which are otherwise somewhat disconnected from the rest of the material that we have chosen to include in this chapter, discuss the two major approaches to such problems, the so-called *simplex algorithm* and *interior-point methods*.

Two other sections, §10.12 and §10.13, also lie outside of our main thrust, but for a different reasons. As mentioned, §10.12 discusses so-called *annealing methods*. These are stochastic, rather than deterministic, algorithms. Annealing methods have solved some problems previously thought to be practically insoluble: They address directly the problem of finding global extrema in the presence of large numbers of undesired local extrema. Section 10.13 discusses a different kind of minimization, namely that of path length along a directed graph by the technique known as *dynamic programming*. This will prove important later, in Chapter 16.

The other sections in this chapter constitute a selection of established algorithms for unconstrained minimization. (For definiteness, we will henceforth regard the optimization problem as that of minimization.) These sections are connected, with later ones depending on earlier ones. If you are just looking for the one "perfect" algorithm to solve your particular application, you may feel that we are telling you more than you want to know. Unfortunately, there is *no* perfect optimization algorithm. This is a case where we strongly urge you to try more than one method in comparative fashion. However, here are some guidelines:

For one-dimensional minimization (minimize a function of one variable), you must choose between methods that need only evaluations of the function, and methods that also require evalutations of the function's derivative. The latter are typically more powerful, but not always enough so as to compensate for the additional calculations of derivatives. We can easily construct examples favoring one approach or

the other.

- For one-dimensional minimization *without* calculation of the derivative, first bracket the minimum as described in §10.2, and then use *Brent's method* as described in §10.3. If your function has a discontinuous second (or lower) derivative, then the parabolic interpolations of Brent's method are of no advantage, and you might wish to use the simplest form of *golden section search*, as described in §10.2.
- For one-dimensional minimization *with* calculation of the derivative, §10.4 supplies a variant of Brent's method that makes limited use of the first derivative information. We shy away from the alternative of using derivative information to construct high-order interpolating polynomials. In our experience, the improvement in convergence very near a smooth, analytic minimum does not make up for the tendency of polynomials sometimes to give wildly wrong interpolations at early stages, especially for functions that may have sharp, "exponential" features.

For the multidimensional case, where you want to minimize a function of two or more variables, the analog of the derivative is the gradient, a vector quantity. You now have three options: compute the gradients using your function's known analytic form, compute the gradients by taking finite differences of computed function values, or don't compute the gradients at all. You also get to choose between methods that require storage of order N^2 and those that require only of order N, where N is the number of dimensions. For moderate values of N this is not a serious constraint; but if N is itself the number of points in a two- or three-dimensional grid, then N^2 storage may be prohibitive.

- For minimization without gradients, the *downhill simplex method* due to Nelder and Mead, discussed in §10.5, is slow, but sure. (This use of the word "simplex" is not to be confused with the simplex method of linear programming.) The method just crawls downhill in a straightforward fashion that makes almost no special assumptions about your function. While this can be extremely slow, it can also be extremely robust. Not to be overlooked is the fact that the code is concise and completely self-contained: a general N-dimensional minimization program in under 100 program lines! The storage requirement is of order N^2, and derivative calculations are not required.
- When your function has some smoothness to it, but you still don't want to compute gradients, turn to *direction set methods*, of which *Powell's method* is the prototype (§10.7). Powell's method requires a one-dimensional minimization subalgorithm such as Brent's method (see above). Storage is of order N^2. Direction set methods are much faster than the downhill simplex method. But keep reading for, possibly, an even better alternative.

Now the case where you are willing to calculate gradients from your function's known analytic form:

- *Conjugate gradient methods*, as typified by the *Fletcher-Reeves algorithm* and the closely related and probably superior *Polak-Ribiere algorithm* are widely used. Conjugate gradient methods require only of order a few times N storage, derivative calculations, and one-dimensional subminimization. Turn to §10.8 for detailed discussion and implementation.
- *Quasi-Newton* or *variable metric* methods are typified by the *Davidon-Fletcher-*

Powell (DFP) algorithm (sometimes referred to just as the *Fletcher-Powell* method) or the closely related *Broyden-Fletcher-Goldfarb-Shanno (BFGS)* algorithm. These methods require of order N^2 storage, derivative calculations, and one-dimensional subminimization. Details are in §10.9. Our personal experience is that the quasi-Newton methods dominate the conjugate gradient methods (if you can afford the storage), but there are probably applications where the reverse is true.

Finally, the case where the method uses gradients, but you are willing to let them be calculated by extra function evaluations (and finite differences):

- In our experience, the quasi-Newton (variable metric) methods work very well in this case, so much so that they can be significantly more efficient than Powell's method on suitable problems. In §10.9 we give an implementation that (almost) hides the gradient calculation completely. Thus quasi-Newton is effectively our first choice of method both when you are willing to calculate gradients, and when you are not willing!

You can now proceed to scale the peaks (and/or plumb the depths) of practical optimization.

CITED REFERENCES AND FURTHER READING:

Dennis, J.E., and Schnabel, R.B. 1983, *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*; reprinted 1996 (Philadelphia: S.I.A.M.).

Polak, E. 1971, *Computational Methods in Optimization* (New York: Academic Press).

Gill, P.E., Murray, W., and Wright, M.H. 1981, *Practical Optimization* (New York: Academic Press).

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 17.

Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), Chapter III.1.

Brent, R.P. 1973, *Algorithms for Minimization without Derivatives* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2002 (New York: Dover).

Dahlquist, G., and Bjorck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), Chapter 10.

10.1 Initially Bracketing a Minimum

What does it mean to *bracket* a minimum? A root of a function is known to be bracketed by a pair of points, a and b, when the function has opposite sign at those two points. A minimum, by contrast, is known to be bracketed only when there is a *triplet* of points, $a < b < c$ (or $c < b < a$), such that $f(b)$ is less than both $f(a)$ and $f(c)$. In this case we know that the function (if it is smooth) has a minimum in the interval (a, c).

We consider the initial bracketing of a minimum to be an essential part of any one-dimensional minimization. There are some one-dimensional algorithms that do not require a rigorous initial bracketing. However, we would *never* trade the secure feeling of *knowing* that a minimum is "in there somewhere" for the dubious reduction of function evaluations that these nonbracketing routines may promise. Please bracket your minima (or, for that matter, your zeros) before isolating them!

There is not much theory as to how to do this bracketing. Obviously you want to step downhill. But how far? We like to take larger and larger steps, starting with some (wild?) initial guess and then increasing the stepsize at each step either by a constant factor, or else by the result of a parabolic extrapolation of the preceding points that is designed to take us to the extrapolated turning point. It doesn't much matter if the steps get big. After all, we are stepping downhill, so we already have the left and middle points of the bracketing triplet. We just need to take a big enough step to stop the downhill trend and get a high third point.

Here is our standard routine, the function `bracket`. It appears in the structure `Bracketmethod` that serves as the base class for all the one-dimensional minimization methods we give in this chapter.

```
struct Bracketmethod {                                                    mins.h
```
Base class for one-dimensional minimization routines. Provides a routine to bracket a minimum and several utility functions.

```
    Doub ax,bx,cx,fa,fb,fc;
    template <class T>
    void bracket(const Doub a, const Doub b, T &func)
```
Given a function or functor `func`, and given distinct initial points `ax` and `bx`, this routine searches in the downhill direction (defined by the function as evaluated at the initial points) and returns new points `ax`, `bx`, `cx` that bracket a minimum of the function. Also returned are the function values at the three points, `fa`, `fb`, and `fc`.

```
    {
        const Doub GOLD=1.618034,GLIMIT=100.0,TINY=1.0e-20;
```
Here `GOLD` is the default ratio by which successive intervals are magnified and `GLIMIT` is the maximum magnification allowed for a parabolic-fit step.

```
        ax=a; bx=b;
        Doub fu;
        fa=func(ax);
        fb=func(bx);
        if (fb > fa) {                          Switch roles of a and b so that we can go
            SWAP(ax,bx);                            downhill in the direction from a to b.
            SWAP(fb,fa);
        }
        cx=bx+GOLD*(bx-ax);                     First guess for c.
        fc=func(cx);
        while (fb > fc) {                       Keep returning here until we bracket.
            Doub r=(bx-ax)*(fb-fc);             Compute u by parabolic extrapolation from
            Doub q=(bx-cx)*(fb-fa);                 a, b, c. TINY is used to prevent any pos-
            Doub u=bx-((bx-cx)*q-(bx-ax)*r)/        sible division by zero.
                (2.0*SIGN(MAX(abs(q-r),TINY),q-r));
            Doub ulim=bx+GLIMIT*(cx-bx);
            We won't go farther than this. Test various possibilities:
            if ((bx-u)*(u-cx) > 0.0) {          Parabolic u is between b and c: try it.
                fu=func(u);
                if (fu < fc) {                  Got a minimum between b and c.
                    ax=bx;
                    bx=u;
                    fa=fb;
                    fb=fu;
                    return;
                } else if (fu > fb) {           Got a minimum between between a and u.
                    cx=u;
                    fc=fu;
                    return;
                }
                u=cx+GOLD*(cx-bx);              Parabolic fit was no use. Use default mag-
                fu=func(u);                         nification.
            } else if ((cx-u)*(u-ulim) > 0.0) {     Parabolic fit is between c and
                fu=func(u);                             its allowed limit.
```

```
                if (fu < fc) {
                    shft3(bx,cx,u,u+GOLD*(u-cx));
                    shft3(fb,fc,fu,func(u));
                }
            } else if ((u-ulim)*(ulim-cx) >= 0.0) {
                u=ulim;
                fu=func(u);
            } else {
                u=cx+GOLD*(cx-bx);
                fu=func(u);
            }
            shft3(ax,bx,cx,u);
            shft3(fa,fb,fc,fu);
        }
    }
    inline void shft2(Doub &a, Doub &b, const Doub c)
    {
        a=b;
        b=c;
    }
    inline void shft3(Doub &a, Doub &b, Doub &c, const Doub d)
    {
        a=b;
        b=c;
        c=d;
    }
    inline void mov3(Doub &a, Doub &b, Doub &c, const Doub d, const Doub e,
        const Doub f)
    {
        a=d; b=e; c=f;
    }
};
```

Limit parabolic u to maximum allowed value.

Reject parabolic u, use default magnification.

Eliminate oldest point and continue.

Utility function used in this structure or others derived from it.

(Because of the housekeeping involved in moving around three or four points and their function values, the above program ends up looking deceptively formidable. That is true of several other programs in this chapter as well. The underlying ideas, however, are quite simple.)

10.2 Golden Section Search in One Dimension

Recall how the bisection method finds roots of functions in one dimension (§9.1): The root is supposed to have been bracketed in an interval (a, b). One then evaluates the function at an intermediate point x and obtains a new, smaller bracketing interval, either (a, x) or (x, b). The process continues until the bracketing interval is acceptably small. It is optimal to choose x to be the midpoint of (a, b) so that the decrease in the interval length is maximized when the function is as uncooperative as it can be, i.e., when the luck of the draw forces you to take the bigger bisected segment.

There is a precise, though slightly subtle, translation of these considerations to the minimization problem. The analog of bisection is to choose a new point x, either between a and b or between b and c. Suppose, to be specific, that we make the latter choice. Then we evaluate $f(x)$. If $f(b) < f(x)$, then the new bracketing triplet of points is (a, b, x); contrariwise, if $f(b) > f(x)$, then the new bracketing triplet

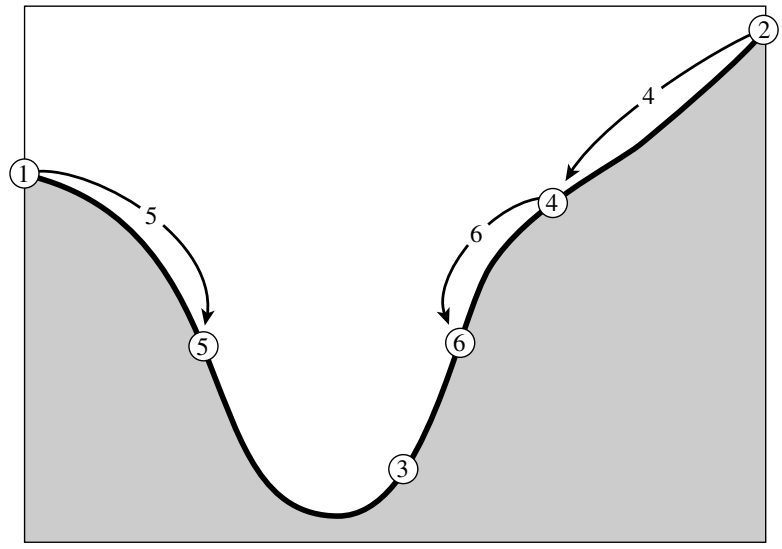


Figure 10.2.1. Successive bracketing of a minimum. The minimum is originally bracketed by points 1,3,2. The function is evaluated at 4, which replaces 2; then at 5, which replaces 1; then at 6, which replaces 4. The rule at each stage is to keep a center point that is lower than the two outside points. After the steps shown, the minimum is bracketed by points 5,3,6.

is (b, x, c). In all cases, the middle point of the new triplet is the abscissa whose ordinate is the best minimum achieved so far; see Figure 10.2.1. We continue the process of bracketing until the distance between the two outer points of the triplet is tolerably small.

How small is "tolerably" small? For a minimum located at a value b, you might naively think that you will be able to bracket it in as small a range as $(1 - \epsilon)b < b < (1 + \epsilon)b$, where ϵ is your computer's floating-point precision, a number like 10^{-7} (for `float`) or 2×10^{-16} (for `double`). Not so! In general, the shape of your function $f(x)$ near b will be given by Taylor's theorem,

$$f(x) \approx f(b) + \tfrac{1}{2} f''(b)(x - b)^2 \tag{10.2.1}$$

The second term will be negligible compared to the first (that is, will be a factor ϵ smaller and will act just like zero when added to it) whenever

$$|x - b| < \sqrt{\epsilon}|b| \sqrt{\frac{2\,|f(b)|}{b^2 f''(b)}} \tag{10.2.2}$$

The reason for writing the right-hand side in this way is that, for most functions, the final square root is a number of order unity. Therefore, as a rule of thumb, it is hopeless to ask for a bracketing interval of width less than $\sqrt{\epsilon}$ times its central value, a fractional width of only about 10^{-4} (single precision) or 10^{-8} (double precision). Knowing this inescapable fact will save you a lot of useless bisections!

The minimum-finding routines of this chapter will often call for a user-supplied argument `tol`, and return with an abscissa whose fractional precision is about $\pm$`tol` (bracketing interval of fractional size about 2×`tol`). Unless you have a better estimate for the right-hand side of equation (10.2.2), you should set `tol` equal to (or

not much less than) the square root of your machine's floating-point precision, since smaller values will gain you nothing.

It remains to decide on a strategy for choosing the new point x, given (a, b, c). Suppose that b is a fraction w of the way between a and c, i.e.,

$$\frac{b-a}{c-a} = w \qquad \frac{c-b}{c-a} = 1-w \tag{10.2.3}$$

Also suppose that our next trial point x is an additional fraction z beyond b,

$$\frac{x-b}{c-a} = z \tag{10.2.4}$$

Then the next bracketing segment will either be of length $w+z$ relative to the current one, or else of length $1-w$. If we want to minimize the worst case possibility, then we will choose z to make these equal, namely

$$z = 1-2w \tag{10.2.5}$$

We see at once that the new point is the symmetric point to b in the original interval, namely with $|b-a|$ equal to $|x-c|$. This implies that the point x lies in the larger of the two segments (z is positive only if $w < 1/2$).

But where in the larger segment? Where did the value of w itself come from? Presumably from the previous stage of applying our same strategy. Therefore, if z is chosen to be optimal, then so was w before it. This *scale similarity* implies that x should be the same fraction of the way from b to c (if that is the bigger segment) as was b from a to c, in other words,

$$\frac{z}{1-w} = w \tag{10.2.6}$$

Equations (10.2.5) and (10.2.6) give the quadratic equation

$$w^2 - 3w + 1 = 0 \qquad \text{yielding} \qquad w = \frac{3-\sqrt{5}}{2} \approx 0.38197 \tag{10.2.7}$$

In other words, the optimal bracketing interval (a, b, c) has its middle point b a fractional distance 0.38197 from one end (say a), and 0.61803 from the other end (say b). These fractions are those of the so-called *golden mean* or *golden section*, whose supposedly aesthetic properties hark back to the ancient Pythagoreans. This optimal method of function minimization, the analog of the bisection method for finding zeros, is thus called the *golden section search*, which can be summarized as follows:

Given, at each stage, a bracketing triplet of points, the next point to be tried is that which is a fraction 0.38197 into the larger of the two intervals (measuring from the central point of the triplet). If you start out with a bracketing triplet whose segments are not in the golden ratios, the procedure of choosing successive points at the golden mean point of the larger segment will quickly converge you to the proper self-replicating ratios.

The golden section search guarantees that each new function evaluation will (after self-replicating ratios have been achieved) bracket the minimum to an interval just 0.61803 times the size of the preceding interval. This is comparable to, but not

quite as good as, the 0.50000 that holds when finding roots by bisection. Note that the convergence is *linear* (in the language of Chapter 9), meaning that successive significant figures are won linearly with additional function evaluations. In the next section we will give a superlinear method, in which the rate at which successive significant figures are liberated increases with each successive function evaluation.

To use the golden section search, you need statements like the following:

```
Golden golden;
golden.bracket(a,b,func);
xmin=golden.minimize(func);
```

The value of the function at the minimum is available in `golden.fmin`. If you want to specify a function tolerance different from the default value of `3.0e-8`, simply override the default value in the constructor:

```
tol = ...
Golden golden(tol);
```

If you want to use a specified bracket as the initial condition, omit the call to `bracket` and set the bracket explicitly:

```
golden.ax = ...; golden.bx = ...; golden.cx = ...;
```

Here is the routine:

mins.h

```
struct Golden : Bracketmethod {
```

Golden section search for minimum.

```
    Doub xmin,fmin;
    const Doub tol;
    Golden(const Doub toll=3.0e-8) : tol(toll) {}
    template <class T>
    Doub minimize(T &func)
```

Given a function or functor `f`, and given a bracketing triplet of abscissas `ax`, `bx`, `cx` (such that `bx` is between `ax` and `cx`, and `f(bx)` is less than both `f(ax)` and `f(cx)`), this routine performs a golden section search for the minimum, isolating it to a fractional precision of about `tol`. The abscissa of the minimum is returned as `xmin`, and the function value at the minimum is returned as `min`, the returned function value.

```
    {
        const Doub R=0.61803399,C=1.0-R;    The golden ratios.
        Doub x1,x2;
        Doub x0=ax;                         At any given time we will keep track of four
        Doub x3=cx;                             points, x0,x1,x2,x3.
        if (abs(cx-bx) > abs(bx-ax)) {      Make x0 to x1 the smaller segment,
            x1=bx;
            x2=bx+C*(cx-bx);                and fill in the new point to be tried.
        } else {
            x2=bx;
            x1=bx-C*(bx-ax);
        }
        Doub f1=func(x1);                   The initial function evaluations. Note that
        Doub f2=func(x2);                       we never need to evaluate the function
        while (abs(x3-x0) > tol*(abs(x1)+abs(x2))) {    at the original endpoints.
            if (f2 < f1) {                  One possible outcome,
                shft3(x0,x1,x2,R*x2+C*x3);  its housekeeping,
                shft2(f1,f2,func(x2));      and a new function evaluation.
            } else {                        The other outcome,
                shft3(x3,x2,x1,R*x1+C*x0);
                shft2(f2,f1,func(x1));      and its new function evaluation.
            }
        }                                   Back to see if we are done.
        if (f1 < f2) {                      We are done. Output the best of the two
            xmin=x1;                            current values.
```

```
            fmin=f1;
        } else {
            xmin=x2;
            fmin=f2;
        }
        return xmin;
    }
};
```

10.3 Parabolic Interpolation and Brent's Method in One Dimension

We already tipped our hand about the desirability of parabolic interpolation in the `bracket` routine of §10.1, but it is now time to be more explicit. A golden section search is designed to handle, in effect, the worst possible case of function minimization, with the uncooperative minimum hunted down and cornered like a scared rabbit. But why assume the worst? If the function is nicely parabolic near to the minimum — surely the generic case for sufficiently smooth functions — then the parabola fitted through any three points ought to take us in a single leap to the minimum, or at least very near to it (see Figure 10.3.1). Since we want to find an abscissa rather than an ordinate, the procedure is technically called *inverse parabolic interpolation*.

The formula for the abscissa x that is the minimum of a parabola through three points $f(a)$, $f(b)$, and $f(c)$ is

$$x = b - \frac{1}{2}\frac{(b-a)^2[f(b)-f(c)] - (b-c)^2[f(b)-f(a)]}{(b-a)[f(b)-f(c)] - (b-c)[f(b)-f(a)]} \tag{10.3.1}$$

as you can easily derive. This formula fails only if the three points are collinear, in which case the denominator is zero (the minimum of the parabola is infinitely far away). Note, however, that (10.3.1) is as happy jumping to a parabolic maximum as to a minimum. No minimization scheme that depends solely on (10.3.1) is likely to succeed in practice.

The exacting task is to invent a scheme that relies on a sure-but-slow technique, like golden section search, when the function is not cooperative, but that switches over to (10.3.1) when the function allows. The task is nontrivial for several reasons, including these: (i) The housekeeping needed to avoid unnecessary function evaluations in switching between the two methods can be complicated. (ii) Careful attention must be given to the "endgame," where the function is being evaluated very near to the roundoff limit of equation (10.2.2). (iii) The scheme for detecting a cooperative versus noncooperative function must be very robust.

Brent's method [1] is up to the task in all particulars. At any particular stage, it is keeping track of six function points (not necessarily all distinct), a, b, u, v, w and x, defined as follows: The minimum is bracketed between a and b; x is the point with the very least function value found so far (or the most recent one in case of a tie); w is the point with the second least function value; v is the previous value of w; and u is the point at which the function was evaluated most recently. Also appearing in the algorithm is the point x_m, the midpoint between a and b; however, the function is not evaluated there.

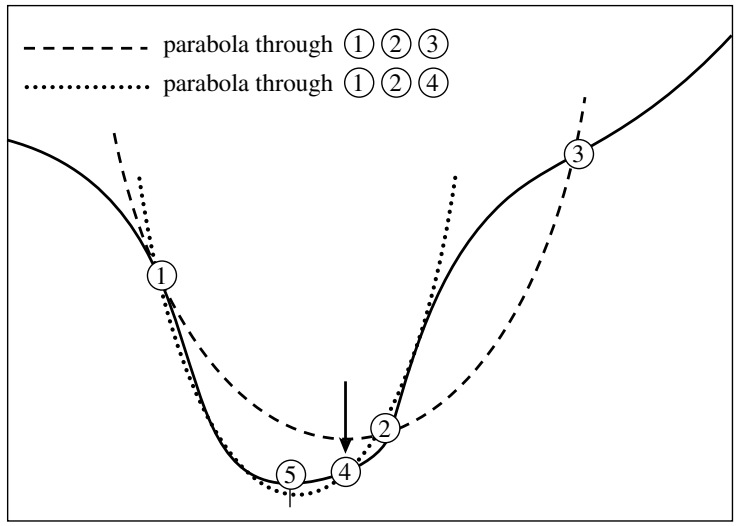


Figure 10.3.1. Convergence to a minimum by inverse parabolic interpolation. A parabola (dashed line) is drawn through the three original points 1,2,3 on the given function (solid line). The function is evaluated at the parabola's minimum, 4, which replaces point 3. A new parabola (dotted line) is drawn through points 1,4,2. The minimum of this parabola is at 5, which is close to the minimum of the function.

You can read the code below to understand the method's logical organization. Mention of a few general principles here may, however, be helpful: Parabolic interpolation is attempted, fitting through the points x, v, and w. To be acceptable, the parabolic step must (i) fall within the bounding interval (a, b), and (ii) imply a movement from the best current value x that is *less* than half the movement of the *step before last*. This second criterion ensures that the parabolic steps are actually converging to something, rather than, say, bouncing around in some nonconvergent limit cycle. In the worst possible case, where the parabolic steps are acceptable but useless, the method will approximately alternate between parabolic steps and golden sections, converging in due course by virtue of the latter. The reason for comparing to the step *before* last seems essentially heuristic: Experience shows that it is better not to "punish" the algorithm for a single bad step if it can make it up on the next one.

Another principle exemplified in the code is never to evaluate the function less than a distance `tol` from a point already evaluated (or from a known bracketing point). The reason is that, as we saw in equation (10.2.2), there is simply no information content in doing so: The function will differ from the value already evaluated only by an amount of order the roundoff error. Therefore, in the code below you will find several tests and modifications of a potential new point, imposing this restriction. This restriction also interacts subtly with the test for "doneness," which the method takes into account.

A typical ending configuration for Brent's method is that a and b are $2 \times x \times$ `tol` apart, with x (the best abscissa) at the midpoint of a and b, and therefore fractionally accurate to $\pm$`tol`.

The calling sequence for `Brent` is exactly analogous to that of `Golden` in the previous section. Indulge us a final reminder that `tol` should generally be no smaller than the square root of your machine's floating-point precision.

mins.h

```
struct Brent : Bracketmethod {
```

Brent's method to find a minimum.

```
    Doub xmin,fmin;
    const Doub tol;
    Brent(const Doub toll=3.0e-8) : tol(toll) {}
    template <class T>
    Doub minimize(T &func)
```

Given a function or functor `f`, and given a bracketing triplet of abscissas `ax`, `bx`, `cx` (such that `bx` is between `ax` and `cx`, and `f(bx)` is less than both `f(ax)` and `f(cx)`), this routine isolates the minimum to a fractional precision of about `tol` using Brent's method. The abscissa of the minimum is returned as `xmin`, and the function value at the minimum is returned as `min`, the returned function value.

```
    {
        const Int ITMAX=100;
        const Doub CGOLD=0.3819660;
        const Doub ZEPS=numeric_limits<Doub>::epsilon()*1.0e-3;
```

Here `ITMAX` is the maximum allowed number of iterations; `CGOLD` is the golden ratio; and `ZEPS` is a small number that protects against trying to achieve fractional accuracy for a minimum that happens to be exactly zero.

```
        Doub a,b,d=0.0,etemp,fu,fv,fw,fx;
        Doub p,q,r,tol1,tol2,u,v,w,x,xm;
        Doub e=0.0;                                  This will be the distance moved on
                                                         the step before last.
        a=(ax < cx ? ax : cx);                       a and b must be in ascending order,
        b=(ax > cx ? ax : cx);                           but input abscissas need not be.
        x=w=v=bx;                                    Initializations...
        fw=fv=fx=func(x);
        for (Int iter=0;iter<ITMAX;iter++) {         Main program loop.
            xm=0.5*(a+b);
            tol2=2.0*(tol1=tol*abs(x)+ZEPS);
            if (abs(x-xm) <= (tol2-0.5*(b-a))) {     Test for done here.
                fmin=fx;
                return xmin=x;
            }
            if (abs(e) > tol1) {                     Construct a trial parabolic fit.
                r=(x-w)*(fx-fv);
                q=(x-v)*(fx-fw);
                p=(x-v)*q-(x-w)*r;
                q=2.0*(q-r);
                if (q > 0.0) p = -p;
                q=abs(q);
                etemp=e;
                e=d;
                if (abs(p) >= abs(0.5*q*etemp) || p <= q*(a-x)
                        || p >= q*(b-x))
                    d=CGOLD*(e=(x >= xm ? a-x : b-x));
```

The above conditions determine the acceptability of the parabolic fit. Here we take the golden section step into the larger of the two segments.

```
                else {
                    d=p/q;                           Take the parabolic step.
                    u=x+d;
                    if (u-a < tol2 || b-u < tol2)
                        d=SIGN(tol1,xm-x);
                }
            } else {
                d=CGOLD*(e=(x >= xm ? a-x : b-x));
            }
            u=(abs(d) >= tol1 ? x+d : x+SIGN(tol1,d));
            fu=func(u);
```

This is the one function evaluation per iteration.

```
            if (fu <= fx) {                          Now decide what to do with our func-
                if (u >= x) a=x; else b=x;               tion evaluation.
                shft3(v,w,x,u);                      Housekeeping follows:
                shft3(fv,fw,fx,fu);
```

```
            } else {
                if (u < x) a=u; else b=u;
                if (fu <= fw || w == x) {
                    v=w;
                    w=u;
                    fv=fw;
                    fw=fu;
                } else if (fu <= fv || v == x || v == w) {
                    v=u;
                    fv=fu;
                }
            }
        }
        throw("Too many iterations in brent");
    }
};
```

Done with housekeeping. Back for another iteration.

CITED REFERENCES AND FURTHER READING:

Brent, R.P. 1973, *Algorithms for Minimization without Derivatives* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2002 (New York: Dover), Chapter 5.[1]

Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), §8.2.

10.4 One-Dimensional Search with First Derivatives

Here we want to accomplish precisely the same goal as in the previous section, namely to isolate a functional minimum that is bracketed by the triplet of abscissas (a, b, c), but utilizing an additional capability to compute the function's first derivative as well as its value.

In principle, we might simply search for a zero of the derivative, ignoring the function value information, using a root finder like `rtflsp` or `zbrent` (§9.2 – §9.3). It doesn't take long to reject *that* idea: How do we distinguish maxima from minima? Where do we go from initial conditions where the derivatives on one or both of the outer bracketing points indicate that "downhill" is in the direction *out* of the bracketed interval?

We don't want to give up our strategy of maintaining a rigorous bracket on the minimum at all times. The only way to keep such a bracket is to update it using function (not derivative) information, with the central point in the bracketing triplet always that with the lowest function value. Therefore, the role of the derivatives can only be to help us choose new trial points within the bracket.

One school of thought is to "use everything you've got": Compute a polynomial of relatively high order (cubic or above) that agrees with some number of previous function and derivative evaluations. For example, there is a unique cubic that agrees with function and derivative at two points, and one can jump to the interpolated minimum of that cubic (if there is a minimum within the bracket). Suggested by Davidon and others, formulas for this tactic are given in [1].

We like to be more conservative than this. Once superlinear convergence sets in, it hardly matters whether its order is moderately lower or higher. In practical

problems that we have met, most function evaluations are spent in getting globally close enough to the minimum for superlinear convergence to commence. So we are more worried about all the funny "stiff" things that high-order polynomials can do (cf. Figure 3.0.1(b)), and about their sensitivities to roundoff error.

This leads us to use derivative information only as follows: The sign of the derivative at the central point of the bracketing triplet (a, b, c) indicates uniquely whether the next test point should be taken in the interval (a, b) or in the interval (b, c). The value of this derivative and of the derivative at the second-best-so-far point are extrapolated to zero by the secant method (inverse linear interpolation), which by itself is superlinear of order 1.618. (The golden mean again: See [1], p. 57.) We impose the same sort of restrictions on this new trial point as in Brent's method. If the trial point must be rejected, we *bisect* the interval under scrutiny.

Yes, we are fuddy-duddies when it comes to making flamboyant use of derivative information in one-dimensional minimization. But we have met too many functions whose computed "derivatives" *don't* integrate up to the function value and *don't* accurately point the way to the minimum, usually because of roundoff errors, sometimes because of truncation error in the method of derivative evaluation.

You will see that the following routine is closely modeled on `Brent` in the previous section. One difference is in the input to the routine. Whereas `Brent` takes either a function or functor argument, `Dbrent` takes only a functor. The functor returns not only the function value, by overloading `operator()`, but also the derivative as the member function `df`. For example, here's how you would code the function x^2:

```
struct Funcd {                          Name Funcd is arbitrary.
    Doub operator() (const Doub x) {
        return x*x;
    }
    Doub df(const Doub x) {
        return 2.0*x;
    }
};
```

To invoke `Dbrent`, you need statements like the following:

```
Dbrent dbrent;
Funcd f;
dbrent.bracket(a,b,f);
xmin=dbrent.minimize(f);
```

The value of the function at the minimum is available in `dbrent.fmin` as usual. Here is the routine:

mins.h

```
struct Dbrent : Bracketmethod {
```

Brent's method to find a minimum, modified to use derivatives.

```
    Doub xmin,fmin;
    const Doub tol;
    Dbrent(const Doub toll=3.0e-8) : tol(toll) {}
    template <class T>
    Doub minimize(T &funcd)
```

Given a functor `funcd` that computes a function and also its derivative function `df`, and given a bracketing triplet of abscissas `ax`, `bx`, `cx` [such that `bx` is between `ax` and `cx`, and `f(bx)` is less than both `f(ax)` and `f(cx)`], this routine isolates the minimum to a fractional precision of about `tol` using a modification of Brent's method that uses derivatives. The abscissa of the minimum is returned as `xmin`, and the minimum function value is returned as `min`, the returned function value.

```
    {
        const Int ITMAX=100;
```

```
const Doub ZEPS=numeric_limits<Doub>::epsilon()*1.0e-3;
Bool ok1,ok2;                                   Will be used as flags for whether pro-
Doub a,b,d=0.0,d1,d2,du,dv,dw,dx,e=0.0;         posed steps are acceptable or not.
Doub fu,fv,fw,fx,olde,tol1,tol2,u,u1,u2,v,w,x,xm;
```

Comments following will point out only differences from the routine in `Brent`. Read that routine first.

```
a=(ax < cx ? ax : cx);
b=(ax > cx ? ax : cx);
x=w=v=bx;
fw=fv=fx=funcd(x);
dw=dv=dx=funcd.df(x);                           All our housekeeping chores are dou-
for (Int iter=0;iter<ITMAX;iter++) {                bled by the necessity of moving
    xm=0.5*(a+b);                                   aorund derivative values as well
    tol1=tol*abs(x)+ZEPS;                           as function values.
    tol2=2.0*tol1;
    if (abs(x-xm) <= (tol2-0.5*(b-a))) {
        fmin=fx;
        return xmin=x;
    }
    if (abs(e) > tol1) {
        d1=2.0*(b-a);                           Initialize these d's to an out-of-bracket
        d2=d1;                                      value.
        if (dw != dx) d1=(w-x)*dx/(dx-dw);      Secant method with one point.
        if (dv != dx) d2=(v-x)*dx/(dx-dv);      And the other.
        Which of these two estimates of d shall we take? We will insist that they be
        within the bracket, and on the side pointed to by the derivative at x:
        u1=x+d1;
        u2=x+d2;
        ok1 = (a-u1)*(u1-b) > 0.0 && dx*d1 <= 0.0;
        ok2 = (a-u2)*(u2-b) > 0.0 && dx*d2 <= 0.0;
        olde=e;                                 Movement on the step before last.
        e=d;
        if (ok1 || ok2) {                       Take only an acceptable d, and if
            if (ok1 && ok2)                         both are acceptable, then take
                d=(abs(d1) < abs(d2) ? d1 : d2); the smallest one.
            else if (ok1)
                d=d1;
            else
                d=d2;
            if (abs(d) <= abs(0.5*olde)) {
                u=x+d;
                if (u-a < tol2 || b-u < tol2)
                    d=SIGN(tol1,xm-x);
            } else {                            Bisect, not golden section.
                d=0.5*(e=(dx >= 0.0 ? a-x : b-x));
                Decide which segment by the sign of the derivative.
            }
        } else {
            d=0.5*(e=(dx >= 0.0 ? a-x : b-x));
        }
    } else {
        d=0.5*(e=(dx >= 0.0 ? a-x : b-x));
    }
    if (abs(d) >= tol1) {
        u=x+d;
        fu=funcd(u);
    } else {
        u=x+SIGN(tol1,d);
        fu=funcd(u);
        if (fu > fx) {                          If the minimum step in the downhill
            fmin=fx;                                direction takes us uphill, then
            return xmin=x;                          we are done.
        }
```

```
            }
            du=funcd.df(u);                          Now all the housekeeping, sigh.
            if (fu <= fx) {
                if (u >= x) a=x; else b=x;
                mov3(v,fv,dv,w,fw,dw);
                mov3(w,fw,dw,x,fx,dx);
                mov3(x,fx,dx,u,fu,du);
            } else {
                if (u < x) a=u; else b=u;
                if (fu <= fw || w == x) {
                    mov3(v,fv,dv,w,fw,dw);
                    mov3(w,fw,dw,u,fu,du);
                } else if (fu < fv || v == x || v == w) {
                    mov3(v,fv,dv,u,fu,du);
                }
            }
        }
        throw("Too many iterations in routine dbrent");
    }
};
```

CITED REFERENCES AND FURTHER READING:

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 55; 454–458.[1]

Brent, R.P. 1973, *Algorithms for Minimization without Derivatives* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2002 (New York: Dover), p. 78.

10.5 Downhill Simplex Method in Multidimensions

With this section we begin consideration of multidimensional minimization, that is, finding the minimum of a function of more than one independent variable. This section stands apart from those that follow, however: All of the algorithms after this section will make explicit use of a one-dimensional minimization algorithm as a part of their computational strategy. This section implements an entirely self-contained strategy, in which one-dimensional minimization does not figure.

The *downhill simplex method* is due to Nelder and Mead [1]. The method requires only function evaluations, not derivatives. It is not very efficient in terms of the number of function evaluations that it requires. Powell's method (§10.7) or the DFP method with finite differences (§10.9) is almost surely faster in all likely applications. However, the downhill simplex method may frequently be the *best* method to use if the figure of merit is "get something working quickly" for a problem whose computational burden is small.

The method has a geometrical naturalness about it that makes it delightful to describe or work through:

A *simplex* is the geometrical figure consisting, in N dimensions, of $N+1$ points (or vertices) and all their interconnecting line segments, polygonal faces, etc. In two dimensions, a simplex is a triangle. In three dimensions, it is a tetrahedron, not necessarily the regular tetrahedron. (The *simplex method* of linear programming, described in §10.10, also makes use of the geometrical concept of a simplex. Otherwise

it is completely unrelated to the algorithm that we are describing in this section.) In general we are only interested in simplexes that are nondegenerate, i.e., that enclose a finite inner N-dimensional volume. If any point of a nondegenerate simplex is taken as the origin, then the N other points define vector directions that span the N-dimensional vector space.

In one-dimensional minimization, it was possible to bracket a minimum, so that the success of a subsequent isolation was guaranteed. Alas! There is no analogous procedure in multidimensional space. For multidimensional minimization, the best we can do is give our algorithm a starting guess, that is, an N-vector of independent variables as the first point to try. The algorithm is then supposed to make its own way downhill through the unimaginable complexity of an N-dimensional topography, until it encounters a (local, at least) minimum.

The downhill simplex method must be started not just with a single point, but with $N + 1$ points, defining an initial simplex. If you think of one of these points (it matters not which) as being your initial starting point $\mathbf{P}_0$, then you can take the other N points to be

$$\mathbf{P}_i = \mathbf{P}_0 + \Delta \mathbf{e}_i \tag{10.5.1}$$

where the $\mathbf{e}_i$'s are N unit vectors, and where Δ is a constant that is your guess of the problem's characteristic length scale. (Or, you could have different Δ_i's for each vector direction.)

The downhill simplex method now takes a series of steps, most steps just moving the point of the simplex where the function is largest ("highest point") through the opposite face of the simplex to a lower point. These steps are called reflections, and they are constructed to conserve the volume of the simplex (and hence maintain its nondegeneracy). When it can do so, the method expands the simplex in one or another direction to take larger steps. When it reaches a "valley floor," the method contracts itself in the transverse direction and tries to ooze down the valley. If there is a situation where the simplex is trying to "pass through the eye of a needle," it contracts itself in all directions, pulling itself in around its lowest (best) point. The routine name `amoeba` is intended to be descriptive of this kind of behavior; the basic moves are summarized in Figure 10.5.1.

Termination criteria can be delicate in any multidimensional minimization routine. Without bracketing, and with more than one independent variable, we no longer have the option of requiring a certain tolerance for a single independent variable. We typically can identify one "cycle" or "step" of our multidimensional algorithm. It is then possible to terminate when the vector distance moved in that step is fractionally smaller in magnitude than some tolerance `tol`. Alternatively, we could require that the decrease in the function value in the terminating step be fractionally smaller than some tolerance `ftol`. Note that while `tol` should not usually be smaller than the square root of the machine precision, it is perfectly appropriate to let `ftol` be of order the machine precision (or perhaps slightly larger so as not to be confused by roundoff).

Note well that either of the above criteria might be fooled by a single anomalous step that, for one reason or another, failed to get anywhere. Therefore, it is frequently a good idea to *restart* a multidimensional minimization routine at a point where it claims to have found a minimum. For this restart, you should reinitialize any ancillary input quantities. In the downhill simplex method, for example, you should reinitialize N of the $N + 1$ vertices of the simplex again by equation (10.5.1), with

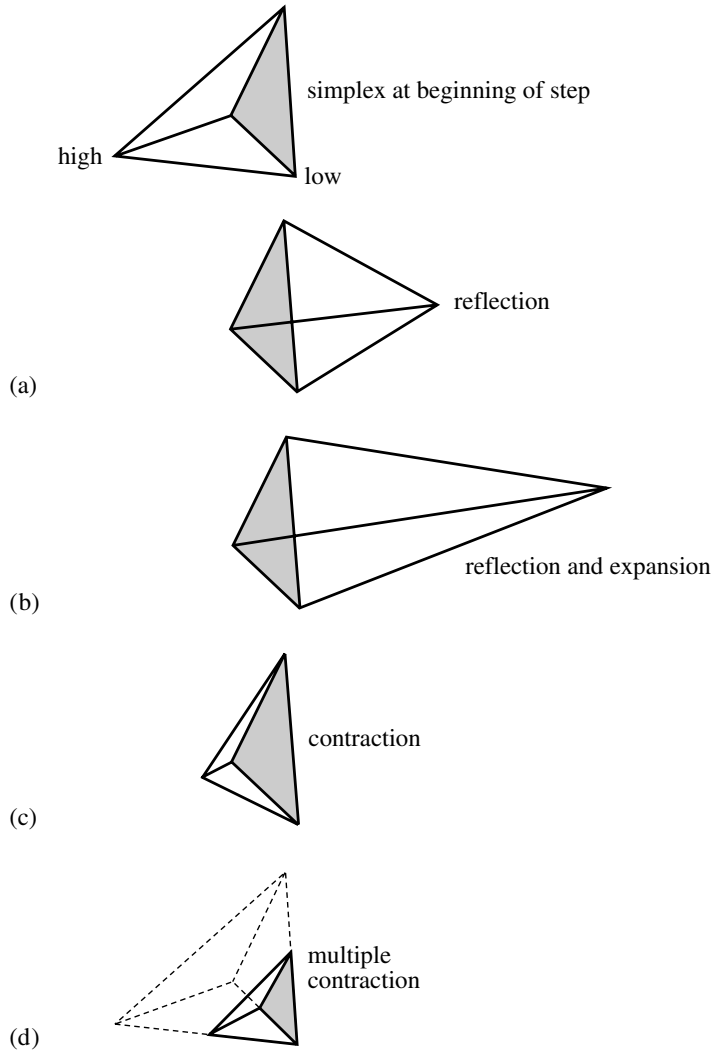


Figure 10.5.1. Possible outcomes for a step in the downhill simplex method. The simplex at the beginning of the step, here a tetrahedron, is shown, top. The simplex at the end of the step can be any one of (a) a reflection away from the high point, (b) a reflection and expansion away from the high point, (c) a contraction along one dimension from the high point, or (d) a contraction along all dimensions toward the low point. An appropriate sequence of such steps will always converge to a minimum of the function.

$\mathbf{P}_0$ being one of the vertices of the claimed minimum.

Restarts should never be very expensive; your algorithm did, after all, converge to the restart point once, and now you are starting the algorithm already there.

The routine below has three different user interfaces. The simplest requires you to supply the initial simplex as in equation (10.5.1):

```
Amoeba am(ftol);
VecDoub point = ...; Doub del = ...;
pmin=am.minimize(point,del,func);
```

The value of the function at the minimum is available in `am.fmin`.

Second, you can use equation (10.5.1) with a vector of increments Δ_i:

```
VecDoub dels = ...;
pmin=am.minimize(point,dels,func);
```

Lastly, you can provide the simplex as an $(N+1) \times N$ matrix whose rows are the coordinates of each vertex:

```
MatDoub p = ...;
pmin=am.minimize(p,func);
```

Consider, then, our N-dimensional amoeba:

amoeba.h

```
struct Amoeba {
```

Multidimensional minimization by the downhill simplex method of Nelder and Mead.

```
    const Doub ftol;
    Int nfunc;                          The number of function evaluations.
    Int mpts;
    Int ndim;
    Doub fmin;                          Function value at the minimum.
    VecDoub y;                          Function values at the vertices of the simplex.
    MatDoub p;                          Current simplex.
    Amoeba(const Doub ftoll) : ftol(ftoll) {}
```

The constructor argument `ftoll` is the fractional convergence tolerance to be achieved in the function value (n.b.!).

```
    template <class T>
    VecDoub minimize(VecDoub_I &point, const Doub del, T &func)
```

Multidimensional minimization of the function or functor `func(x)`, where `x[0..ndim-1]` is a vector in `ndim` dimensions, by the downhill simplex method of Nelder and Mead. The initial simplex is specified as in equation (10.5.1) by a `point[0..ndim-1]` and a constant displacement `del` along each coordinate direction. Returned is the location of the minimum.

```
    {
        VecDoub dels(point.size(),del);
        return minimize(point,dels,func);
    }
    template <class T>
    VecDoub minimize(VecDoub_I &point, VecDoub_I &dels, T &func)
```

Alternative interface that takes different displacements `dels[0..ndim-1]` in different directions for the initial simplex.

```
    {
        Int ndim=point.size();
        MatDoub pp(ndim+1,ndim);
        for (Int i=0;i<ndim+1;i++) {
            for (Int j=0;j<ndim;j++)
                pp[i][j]=point[j];
            if (i !=0 ) pp[i][i-1] += dels[i-1];
        }
        return minimize(pp,func);
    }
    template <class T>
    VecDoub minimize(MatDoub_I &pp, T &func)
```

Most general interface: initial simplex specified by the matrix `pp[0..ndim][0..ndim-1]`. Its `ndim+1` rows are `ndim`-dimensional vectors that are the vertices of the starting simplex.

```
    {
        const Int NMAX=5000;            Maximum allowed number of function evaluations.
        const Doub TINY=1.0e-10;
        Int ihi,ilo,inhi;
        mpts=pp.nrows();
        ndim=pp.ncols();
        VecDoub psum(ndim),pmin(ndim),x(ndim);
        p=pp;
        y.resize(mpts);
        for (Int i=0;i<mpts;i++) {
            for (Int j=0;j<ndim;j++)
                x[j]=p[i][j];
            y[i]=func(x);
```

```
        }
        nfunc=0;
        get_psum(p,psum);
        for (;;) {
            ilo=0;
```

First we must determine which point is the highest (worst), next-highest, and lowest (best), by looping over the points in the simplex.

```
            ihi = y[0]>y[1] ? (inhi=1,0) : (inhi=0,1);
            for (Int i=0;i<mpts;i++) {
                if (y[i] <= y[ilo]) ilo=i;
                if (y[i] > y[ihi]) {
                    inhi=ihi;
                    ihi=i;
                } else if (y[i] > y[inhi] && i != ihi) inhi=i;
            }
            Doub rtol=2.0*abs(y[ihi]-y[ilo])/(abs(y[ihi])+abs(y[ilo])+TINY);
```

Compute the fractional range from highest to lowest and return if satisfactory.

```
            if (rtol < ftol) {          If returning, put best point and value in slot 0.
                SWAP(y[0],y[ilo]);
                for (Int i=0;i<ndim;i++) {
                    SWAP(p[0][i],p[ilo][i]);
                    pmin[i]=p[0][i];
                }
                fmin=y[0];
                return pmin;
            }
            if (nfunc >= NMAX) throw("NMAX exceeded");
            nfunc += 2;
```

Begin a new iteration. First extrapolate by a factor -1 through the face of the simplex across from the high point, i.e., reflect the simplex from the high point.

```
            Doub ytry=amotry(p,y,psum,ihi,-1.0,func);
            if (ytry <= y[ilo])
```

Gives a result better than the best point, so try an additional extrapolation by a factor 2.

```
                ytry=amotry(p,y,psum,ihi,2.0,func);
            else if (ytry >= y[inhi]) {
```

The reflected point is worse than the second-highest, so look for an intermediate lower point, i.e., do a one-dimensional contraction.

```
                Doub ysave=y[ihi];
                ytry=amotry(p,y,psum,ihi,0.5,func);
                if (ytry >= ysave) {        Can't seem to get rid of that high point.
                    for (Int i=0;i<mpts;i++) {     Better contract around the lowest
                        if (i != ilo) {            (best) point.
                            for (Int j=0;j<ndim;j++)
                                p[i][j]=psum[j]=0.5*(p[i][j]+p[ilo][j]);
                            y[i]=func(psum);
                        }
                    }
                    nfunc += ndim;          Keep track of function evaluations.
                    get_psum(p,psum);       Recompute psum.
                }
            } else --nfunc;                 Correct the evaluation count.
        }                                   Go back for the test of doneness and the next
    }                                           iteration.
}
inline void get_psum(MatDoub_I &p, VecDoub_O &psum)
```

Utility function.

```
{
    for (Int j=0;j<ndim;j++) {
        Doub sum=0.0;
        for (Int i=0;i<mpts;i++)
            sum += p[i][j];
        psum[j]=sum;
    }
}
```

```
    template <class T>
    Doub amotry(MatDoub_IO &p, VecDoub_O &y, VecDoub_IO &psum,
        const Int ihi, const Doub fac, T &func)
    Helper function: Extrapolates by a factor fac through the face of the simplex across from
    the high point, tries it, and replaces the high point if the new point is better.
    {
        VecDoub ptry(ndim);
        Doub fac1=(1.0-fac)/ndim;
        Doub fac2=fac1-fac;
        for (Int j=0;j<ndim;j++)
            ptry[j]=psum[j]*fac1-p[ihi][j]*fac2;
        Doub ytry=func(ptry);           Evaluate the function at the trial point.
        if (ytry < y[ihi]) {            If it's better than the highest, then replace the
            y[ihi]=ytry;                    highest.
            for (Int j=0;j<ndim;j++) {
                psum[j] += ptry[j]-p[ihi][j];
                p[ihi][j]=ptry[j];
            }
        }
        return ytry;
    }
};
```


CITED REFERENCES AND FURTHER READING:

Nelder, J.A., and Mead, R. 1965, "A Simplex Method for Function Minimization," *Computer Journal*, vol. 7, pp. 308–313.[1]

Yarbro, L.A., and Deming, S.N. 1974, "Selection and Preprocessing of Factors for Simplex Optimization," *Analytica Chimica Acta*, vol. 73, pp. 391–398.

Jacoby, S.L.S, Kowalik, J.S., and Pizzo, J.T. 1972, *Iterative Methods for Nonlinear Optimization Problems* (Englewood Cliffs, NJ: Prentice-Hall).

10.6 Line Methods in Multidimensions

We know (§10.2 – §10.4) how to minimize a function of one variable. If we start at a point $\mathbf{P}$ in N-dimensional space, and proceed from there in some vector direction $\mathbf{n}$, then any function of N variables $f(\mathbf{P})$ can be minimized along the line $\mathbf{n}$ by our one-dimensional methods. One can dream up various multidimensional minimization methods that consist of sequences of such line minimizations. Different methods will differ only by how, at each stage, they choose the next direction $\mathbf{n}$ to try. The minimization methods in the next two sections fall under this general schema of successive line minimizations. (The quasi-Newton algorithm in §10.9 does not need very accurate line minimizations. Accordingly, it uses the approximate line minimization routine, `lnsrch` from §9.7.1.)

In this section we provide the line minimization routine `linmin` as part of the base class `Linemethod` from which we will derive the minimization methods in the next two sections. These minimization routines regard `linmin` as a black-box subalgorithm, whose definition is

> `linmin`: Given as input the vectors $\mathbf{P}$ and $\mathbf{n}$, and the function f, find the scalar λ that minimizes $f(\mathbf{P} + \lambda\mathbf{n})$. Replace $\mathbf{P}$ by $\mathbf{P} + \lambda\mathbf{n}$. Replace $\mathbf{n}$ by $\lambda\mathbf{n}$. Done.

Since we will want to use `linmin` with methods whose choice of successive directions does not involve explicit computation of the function's gradient, `linmin` itself cannot use gradient information. Later, in §10.8, we will consider a method that does use gradient information. Accordingly, there we provide a routine `dlinmin` that makes use of this information to reduce the total computational burden.

The obvious way to implement `linmin` is to use the methods of one-dimensional minimization described in §10.2 – §10.4, but to rewrite the programs of those sections so that their bookkeeping is done on vector-valued points **P** (all lying along a given direction **n**) rather than scalar-valued abscissas x. That straightforward task produces long routines densely populated with "`for(k=0;k<n;k++)`" loops.

As an alternative, we can simply reuse the one-dimensional minimization routines by constructing a functor `F1dim`, which gives the value of your function, say `func`, along the line going through the point `p` in the direction `xi`. The function `linmin` calls our familiar one-dimensional routine `Brent` (§10.4) and instructs it to minimize `F1dim`. The routine `linmin` communicates with `F1dim` "over the head" of `Brent` through the constructor, our usual C++ idiom.

mins_ndim.h

```
template <class T>
struct Linemethod {
```

Base class for line-minimization algorithms. Provides the line-minimization routine `linmin`.

```
    VecDoub p;
    VecDoub xi;
    T &func;
    Int n;
    Linemethod(T &funcc) : func(funcc) {}
```

Constructor argument is the user-supplied function or functor to be minimized.

```
    Doub linmin()
```

Line-minimization routine. Given an `n`-dimensional point `p[0..n-1]` and an `n`-dimensional direction `xi[0..n-1]`, moves and resets `p` to where the function or functor `func(p)` takes on a minimum along the direction `xi` from `p`, and replaces `xi` by the actual vector displacement that `p` was moved. Also returns the value of `func` at the returned location `p`. This is actually all accomplished by calling the routines `bracket` and `minimize` of `Brent`.

```
    {
        Doub ax,xx,xmin;
        n=p.size();
        F1dim<T> f1dim(p,xi,func);
        ax=0.0;                                  Initial guess for brackets.
        xx=1.0;
        Brent brent;
        brent.bracket(ax,xx,f1dim);
        xmin=brent.minimize(f1dim);
        for (Int j=0;j<n;j++) {                  Construct the vector results to return.
            xi[j] *= xmin;
            p[j] += xi[j];
        }
        return brent.fmin;
    }
};
```

mins_ndim.h

```
template <class T>
struct F1dim {
```

Must accompany `linmin` in `Linemethod`.

```
    const VecDoub &p;
    const VecDoub &xi;
    Int n;
    T &func;
    VecDoub xt;
```

```
    F1dim(VecDoub_I &pp, VecDoub_I &xii, T &funcc) : p(pp),
        xi(xii), n(pp.size()), func(funcc), xt(n) {}
    Constructor takes as inputs an n-dimensional point p[0..n-1] and an n-dimensional di-
    rection xi[0..n-1] from linmin, as well as the function or functor that takes a vector
    argument.
    Doub operator() (const Doub x)
    Functor returning value of the given function along a one-dimensional line.
    {
        for (Int j=0;j<n;j++)
            xt[j]=p[j]+x*xi[j];
        return func(xt);
    }
};
```

10.7 Direction Set (Powell's) Methods in Multidimensions

With a routine for line minimization in hand, you might think of this simple method for general multidimensional minimization: Take the unit vectors $\mathbf{e}_0, \mathbf{e}_1, \ldots \mathbf{e}_{N-1}$ as a *set of directions*. Using `linmin`, move along the first direction to its minimum, then *from there* along the second direction to *its* minimum, and so on, cycling through the whole set of directions as many times as necessary, until the function stops decreasing.

This simple method is actually not too bad for many functions. Even more interesting is why it *is* bad, i.e., very inefficient, for some other functions. Consider a function of two dimensions whose contour map (level lines) happens to define a long, narrow valley at some angle to the coordinate basis vectors (see Figure 10.7.1). Then the only way "down the length of the valley" going along the basis vectors at each stage is by a series of many tiny steps. More generally, in N dimensions, if the function's second derivatives are much larger in magnitude in some directions than in others, then many cycles through all N basis vectors will be required in order to get anywhere. This condition is not all that unusual; according to Murphy's Law, you should count on it.

Obviously what we need is a better set of directions than the $\mathbf{e}_i$'s. All *direction set methods* consist of prescriptions for updating the set of directions as the method proceeds, attempting to come up with a set that either (i) includes some very good directions that will take us far along narrow valleys, or else (more subtly) (ii) includes some number of "noninterfering" directions with the special property that minimization along one is not "spoiled" by subsequent minimization along another, so that interminable cycling through the set of directions can be avoided.

10.7.1 Conjugate Directions

This concept of "noninterfering" directions, more conventionally called *conjugate directions*, is worth making mathematically explicit.

First, note that if we minimize a function along some direction $\mathbf{u}$, then the gradient of the function must be perpendicular to $\mathbf{u}$ at the line minimum; if not, then there would still be a nonzero directional derivative along $\mathbf{u}$.

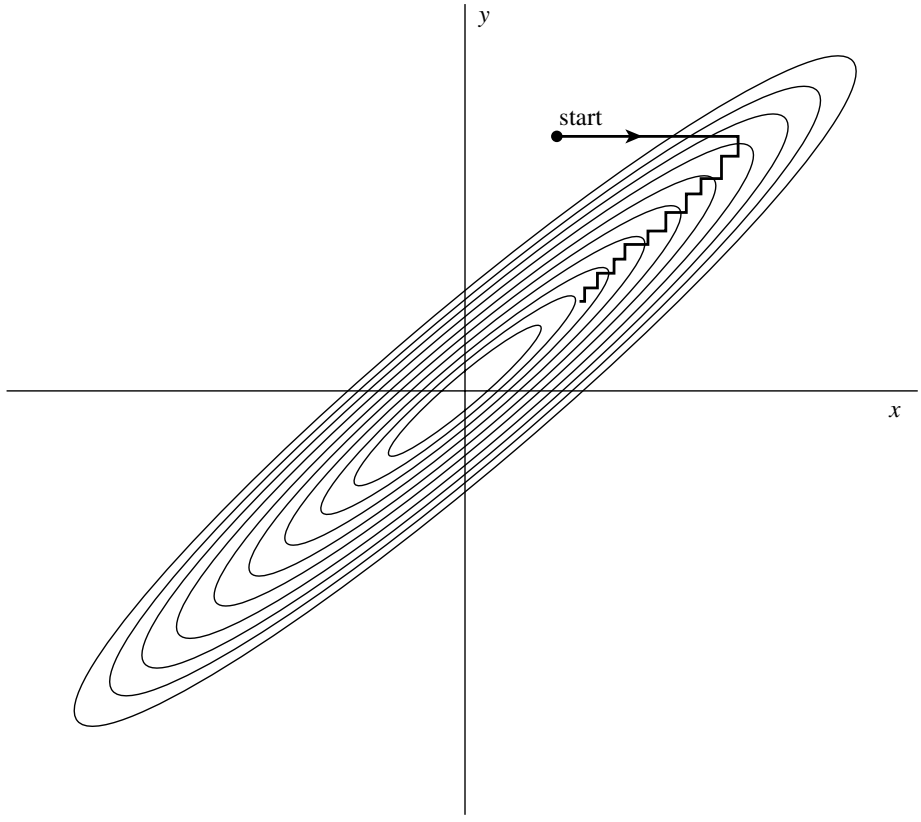


Figure 10.7.1. Successive minimizations along coordinate directions in a long, narrow "valley" (shown as contour lines). Unless the valley is optimally oriented, this method is extremely inefficient, taking many tiny steps to get to the minimum, crossing and re-crossing the principal axis.

Next take some particular point $\mathbf{P}$ as the origin of the coordinate system with coordinates $\mathbf{x}$. Then any function f can be approximated by its Taylor series

$$\begin{aligned} f(\mathbf{x}) &= f(\mathbf{P}) + \sum_i \frac{\partial f}{\partial x_i} x_i + \frac{1}{2} \sum_{i,j} \frac{\partial^2 f}{\partial x_i \partial x_j} x_i x_j + \cdots \\ &\approx c - \mathbf{b} \cdot \mathbf{x} + \frac{1}{2} \mathbf{x} \cdot \mathbf{A} \cdot \mathbf{x} \end{aligned} \tag{10.7.1}$$

where

$$c \equiv f(\mathbf{P}) \qquad \mathbf{b} \equiv -\nabla f|_{\mathbf{P}} \qquad [\mathbf{A}]_{ij} \equiv \left. \frac{\partial^2 f}{\partial x_i \partial x_j} \right|_{\mathbf{P}} \tag{10.7.2}$$

The matrix $\mathbf{A}$ whose components are the second partial derivative matrix of the function is called the *Hessian matrix* of the function at $\mathbf{P}$.

In the approximation of (10.7.1), the gradient of f is easily calculated as

$$\nabla f = \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \tag{10.7.3}$$

(This implies that the gradient will vanish — the function will be at an extremum — at a value of $\mathbf{x}$ obtained by solving $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. We will return to this idea in §10.9!)

How does the gradient ∇f *change* as we move along some direction? Evidently

$$\delta(\nabla f) = \mathbf{A} \cdot (\delta \mathbf{x}) \tag{10.7.4}$$

Suppose that we have moved along some direction $\mathbf{u}$ to a minimum and now propose to move along some new direction $\mathbf{v}$. The condition that motion along $\mathbf{v}$ not *spoil* our minimization along $\mathbf{u}$ is just that the gradient stay perpendicular to $\mathbf{u}$, i.e., that the change in the gradient be perpendicular to $\mathbf{u}$. By equation (10.7.4) this is just

$$0 = \mathbf{u} \cdot \delta(\nabla f) = \mathbf{u} \cdot \mathbf{A} \cdot \mathbf{v} \tag{10.7.5}$$

When (10.7.5) holds for two vectors $\mathbf{u}$ and $\mathbf{v}$, they are said to be *conjugate*. When the relation holds pairwise for all members of a set of vectors, they are said to be a conjugate set. If you do successive line minimizations of a function along a conjugate set of directions, then you don't need to redo any of those directions (unless, of course, you spoil things by minimizing along a direction that they are *not* conjugate to).

A triumph for a direction set method is to come up with a set of N linearly independent, mutually conjugate directions. Then, one pass of N line minimizations will put it exactly at the minimum of a quadratic form like (10.7.1). For functions f that are not exactly quadratic forms, it won't be exactly at the minimum, but repeated cycles of N line minimizations will in due course converge *quadratically* to the minimum.

10.7.2 Powell's Quadratically Convergent Method

Powell first discovered a direction set method that does produce N mutually conjugate directions. Here is how it goes: Initialize the set of directions $\mathbf{u}_i$ to the basis vectors,

$$\mathbf{u}_i = \mathbf{e}_i \qquad i = 0, \ldots, N-1 \tag{10.7.6}$$

Now repeat the following sequence of steps ("basic procedure") until your function stops decreasing:

- Save your starting position as $\mathbf{P}_0$.
- For $i = 0, \ldots, N-1$, move $\mathbf{P}_i$ to the minimum along direction $\mathbf{u}_i$ and call this point $\mathbf{P}_{i+1}$.
- For $i = 0, \ldots, N-2$, set $\mathbf{u}_i \leftarrow \mathbf{u}_{i+1}$.
- Set $\mathbf{u}_{N-1} \leftarrow \mathbf{P}_N - \mathbf{P}_0$.
- Move $\mathbf{P}_N$ to the minimum along direction $\mathbf{u}_{N-1}$ and call this point $\mathbf{P}_0$.

Powell, in 1964, showed that, for a quadratic form like (10.7.1), k iterations of the above basic procedure produce a set of directions $\mathbf{u}_i$ whose last k members are mutually conjugate. Therefore, N iterations of the basic procedure, amounting to $N(N+1)$ line minimizations in all, will exactly minimize a quadratic form. Brent [1] gives proofs of these statements in accessible form.

Unfortunately, there is a problem with Powell's quadratically convergent algorithm. The procedure of throwing away, at each stage, $\mathbf{u}_0$ in favor of $\mathbf{P}_N - \mathbf{P}_0$ tends to produce sets of directions that "fold up on each other" and become linearly dependent. Once this happens, the procedure finds the minimum of the function f only over a subspace of the full N-dimensional case; in other words, it gives the wrong answer. Therefore, the algorithm must not be used in the form given above.

There are a number of ways to fix up the problem of linear dependence in Powell's algorithm, among them:

1. You can reinitialize the set of directions $\mathbf{u}_i$ to the basis vectors $\mathbf{e}_i$ after every N or $N+1$ iterations of the basic procedure. This produces a serviceable method, which we commend to you if quadratic convergence is important for your application (i.e., if your functions are close to quadratic forms and if you desire high accuracy).

2. Brent points out that the set of directions can equally well be reset to the columns of any orthogonal matrix. Rather than throw away the information on conjugate directions already built up, he resets the direction set to calculated principal directions of the matrix $\mathbf{A}$ (which he gives a procedure for determining). The calculation is essentially a singular value decomposition algorithm (see §2.6). Brent has a number of other cute tricks up his sleeve, and his modification of Powell's method is probably the best presently known. Consult [1] for a detailed description and listing of the program. Unfortunately it is rather too elaborate for us to include here.

3. You can give up the property of quadratic convergence in favor of a more heuristic scheme (due to Powell) that tries to find a few good directions along narrow valleys instead of N necessarily conjugate directions. This is the method that we now implement. (It is also the version of Powell's method given in Acton [2], from which parts of the following discussion are drawn.)

10.7.3 Discarding the Direction of Largest Decrease

The fox and the grapes: Now that we are going to give up the property of quadratic convergence, was it so important after all? That depends on the function that you are minimizing. Some applications produce functions with long, twisty valleys. Quadratic convergence is of no particular advantage to a program that must slalom down the length of a valley floor that twists one way and another (and another, and another, ... — there are N dimensions!). Along the long direction, a quadratically convergent method is trying to extrapolate to the minimum of a parabola that just isn't (yet) there while the conjugacy of the $N-1$ transverse directions keeps getting spoiled by the twists.

Sooner or later, however, we do arrive at an approximately ellipsoidal minimum (cf. equation 10.7.1 when $\mathbf{b}$, the gradient, is zero). Then, depending on how much accuracy we require, a method with quadratic convergence can save us several times N^2 extra line minimizations, since quadratic convergence *doubles* the number of significant figures at each iteration.

The basic idea of our now-modified Powell's method is still to take $\mathbf{P}_N - \mathbf{P}_0$ as a new direction; it is, after all, the average direction moved in after trying all N possible directions. For a valley whose long direction is twisting slowly, this direction is likely to give us a good run along the new long direction. The change is to discard the old direction along which the function f made its *largest decrease*. This seems paradoxical, since that direction was the *best* of the previous iteration. However, it is also likely to be a major component of the new direction that we are adding, so dropping it gives us the best chance of avoiding a buildup of linear dependence.

There are a couple of exceptions to this basic idea. Sometimes it is better *not* to add a new direction at all. Define

$$f_0 \equiv f(\mathbf{P}_0) \qquad f_N \equiv f(\mathbf{P}_N) \qquad f_E \equiv f(2\mathbf{P}_N - \mathbf{P}_0) \tag{10.7.7}$$

Here f_E is the function value at an "extrapolated" point somewhat further along the proposed new direction. Also define Δf to be the magnitude of the largest decrease along one particular direction of the present basic procedure iteration. (Δf is a positive number.) Then:

1. If $f_E \geq f_0$, then keep the old set of directions for the next basic procedure, because the average direction $\mathbf{P}_N - \mathbf{P}_0$ is all played out.

2. If $2(f_0 - 2f_N + f_E)[(f_0 - f_N) - \Delta f]^2 \geq (f_0 - f_E)^2 \Delta f$, then keep the old set of directions for the next basic procedure, because either (i) the decrease along the average direction was not primarily due to any single direction's decrease, or (ii) there is a substantial second derivative along the average direction and we seem to be near to the bottom of its minimum.

The following routine implements Powell's method in the version just described. In the routine, `xi` is the matrix whose columns are the set of directions $\mathbf{n}_i$; otherwise the correspondence of notation should be self-evident. If the function to be minimized is provided as a functor `Func`

```
struct Func {
    Doub operator()(VecDoub_I &x);
};
```

then the normal calling sequence for `Powell` looks something like this:

```
VecDoub p = ...;
Func func;
Powell<Func> powell(func);
p=powell.minimize(p);          OK to overwrite initial guess.
```

The function value at the minimum is available as `powell.fret`.

If, on the other hand, the function to be minimized is provided as a normal C++ function,

```
Doub func(VecDoub_I &x);
```

then the constructor call looks like this instead:

```
Powell<Doub (VecDoub_I &)> powell(func);
```

Note that the constructor takes an optional argument that specifies the function tolerance for the minimization.

mins_ndim.h

```
template <class T>
struct Powell : Linemethod<T> {
```

Multidimensional minimization by Powell's method.

```
    Int iter;
    Doub fret;                      Value of the function at the minimum.
    using Linemethod<T>::func;      Variables from a templated base class are not auto-
    using Linemethod<T>::linmin;        matically inherited.
    using Linemethod<T>::p;
    using Linemethod<T>::xi;
    const Doub ftol;
    Powell(T &func, const Doub ftoll=3.0e-8) : Linemethod<T>(func),
        ftol(ftoll) {}
```

Constructor arguments are `func`, the function or functor to be minimized, and an optional argument `ftoll`, the fractional tolerance in the function value such that failure to decrease by more than this amount on one iteration signals doneness.

```
    VecDoub minimize(VecDoub_I &pp)
```

Minimization of a function or functor `n` variables. Input consists of an initial starting point `pp[0..n-1]`. The initial matrix `ximat[0..n-1][0..n-1]`, whose columns contain the initial set of directions, is set to the `n` unit vectors. Returned is the best point found, at which point `fret` is the minimum function value and `iter` is the number of iterations taken.

```
    {
        Int n=pp.size();
        MatDoub ximat(n,n,0.0);
        for (Int i=0;i<n;i++) ximat[i][i]=1.0;
        return minimize(pp,ximat);
    }
    VecDoub minimize(VecDoub_I &pp, MatDoub_IO &ximat)
    Alternative interface: Input consists of the initial starting point pp[0..n-1] and an initial
    matrix ximat[0..n-1][0..n-1], whose columns contain the initial set of directions. On
    output ximat is the then-current direction set.
    {
        const Int ITMAX=200;          Maximum allowed iterations.
        const Doub TINY=1.0e-25;      A small number.
        Doub fptt;
        Int n=pp.size();
        p=pp;
        VecDoub pt(n),ptt(n);
        xi.resize(n);
        fret=func(p);
        for (Int j=0;j<n;j++) pt[j]=p[j];  Save the initial point.
        for (iter=0;;++iter) {
            Doub fp=fret;
            Int ibig=0;
            Doub del=0.0;             Will be the biggest function decrease.
            for (Int i=0;i<n;i++) {   In each iteration, loop over all directions in the set.
                for (Int j=0;j<n;j++) xi[j]=ximat[j][i];     Copy the direction,
                fptt=fret;
                fret=linmin();                  minimize along it,
                if (fptt-fret > del) {          and record it if it is the largest decrease
                    del=fptt-fret;                  so far.
                    ibig=i+1;
                }
            }                                   Here comes the termination criterion:
            if (2.0*(fp-fret) <= ftol*(abs(fp)+abs(fret))+TINY) {
                return p;
            }
            if (iter == ITMAX) throw("powell exceeding maximum iterations.");
            for (Int j=0;j<n;j++) {             Construct the extrapolated point and the
                ptt[j]=2.0*p[j]-pt[j];              average direction moved. Save the
                xi[j]=p[j]-pt[j];                   old starting point.
                pt[j]=p[j];
            }
            fptt=func(ptt);                     Function value at extrapolated point.
            if (fptt < fp) {
                Doub t=2.0*(fp-2.0*fret+fptt)*SQR(fp-fret-del)-del*SQR(fp-fptt);
                if (t < 0.0) {
                    fret=linmin();              Move to the minimum of the new direc-
                    for (Int j=0;j<n;j++) {         tion, and save the new direction.
                        ximat[j][ibig-1]=ximat[j][n-1];
                        ximat[j][n-1]=xi[j];
                    }
                }
            }
        }
    }
};
```

CITED REFERENCES AND FURTHER READING:

Brent, R.P. 1973, *Algorithms for Minimization without Derivatives* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2002 (New York: Dover), Chapter 7.[1]

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 464–467.[2]

Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), pp. 259–262.

10.8 Conjugate Gradient Methods in Multidimensions

Consider now the case where you are able to calculate, at a given N-dimensional point $\mathbf{P}$, not just the value of a function $f(\mathbf{P})$ but also the gradient (vector of first partial derivatives) $\nabla f(\mathbf{P})$.

A rough counting argument will show how advantageous it is to use the gradient information: Suppose that the function f is roughly approximated as a quadratic form, as above in equation (10.7.1),

$$f(\mathbf{x}) \approx c - \mathbf{b} \cdot \mathbf{x} + \frac{1}{2}\mathbf{x} \cdot \mathbf{A} \cdot \mathbf{x} \tag{10.8.1}$$

Then the number of unknown parameters in f is equal to the number of free parameters in $\mathbf{A}$ and $\mathbf{b}$, which is $\frac{1}{2}N(N+1)$, which we see to be of order N^2. Changing any one of these parameters can move the location of the minimum. Therefore, we should not expect to be able to *find* the minimum until we have collected an equivalent information content, of order N^2 numbers.

In the direction set methods of §10.7, we collected the necessary information by making on the order of N^2 separate line minimizations, each requiring "a few" (but sometimes a *big* few!) function evaluations. Now, each evaluation of the gradient will bring us N new components of information. If we use them wisely, we should need to make only of order N separate line minimizations. That is in fact the case for the algorithms in this section and the next.

A factor of N improvement in computational speed is not necessarily implied. As a rough estimate, we might imagine that the calculation of *each component* of the gradient takes about as long as evaluating the function itself. In that case there will be of order N^2 equivalent function evaluations both with and without gradient information. Even if the advantage is not of order N, however, it is nevertheless quite substantial: (i) Each calculated component of the gradient will typically save not just one function evaluation, but a number of them, equivalent to, say, a whole line minimization. (ii) There is often a high degree of redundancy in the formulas for the various components of a function's gradient. When this is so, especially when there is also redundancy with the calculation of the function, the calculation of the gradient may cost significantly less than N function evaluations.

A common beginner's error is to assume that any reasonable way of incorporating gradient information should be about as good as any other. This line of thought leads to the following *not-very-good* algorithm, the *steepest descent method*:

> Steepest Descent: Start at a point $\mathbf{P}_0$. As many times as needed, move from point $\mathbf{P}_i$ to the point $\mathbf{P}_{i+1}$ by minimizing along the line from $\mathbf{P}_i$ in the direction of the local downhill gradient $-\nabla f(\mathbf{P}_i)$.

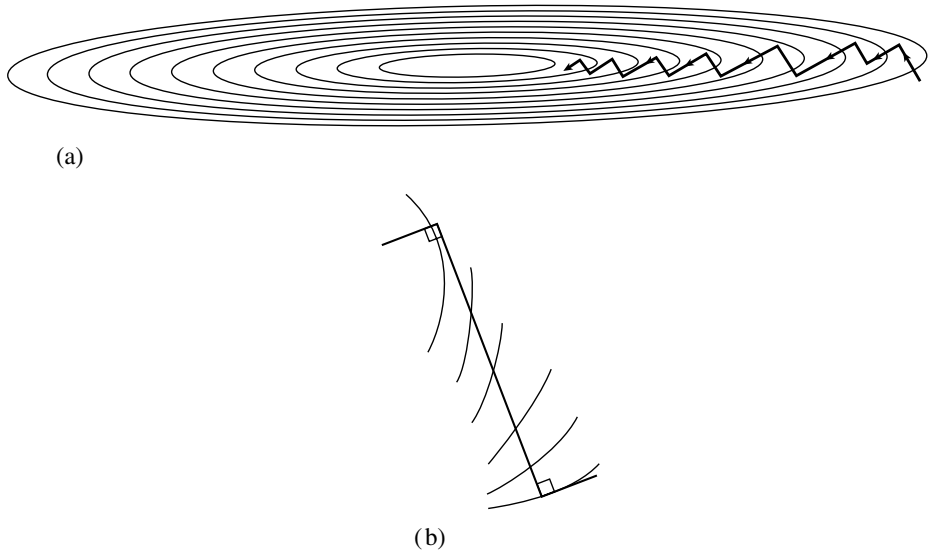


Figure 10.8.1. (a) Steepest descent method in a long, narrow "valley." While more efficient than the strategy of Figure 10.7.1, steepest descent is nonetheless an inefficient strategy, taking many steps to reach the valley floor. (b) Magnified view of one step: A step starts off in the local gradient direction, perpendicular to the contour lines, and traverses a straight line until a local minimum is reached, where the traverse is parallel to the local contour lines.

The problem with the steepest descent method (which, incidentally, goes back to Cauchy), is similar to the problem that was shown in Figure 10.7.1. The method will perform many small steps in going down a long, narrow valley, even if the valley is a perfect quadratic form. You might have hoped that, say in two dimensions, your first step would take you to the valley floor, the second step directly down the long axis; but remember that the new gradient at the minimum point of any line minimization is perpendicular to the direction just traversed. Therefore, with the steepest descent method, you *must* make a right angle turn, which does *not*, in general, take you to the minimum. (See Figure 10.8.1.)

Just as in the discussion that led up to equation (10.7.5), we really want a way of proceeding not down the new gradient, but rather in a direction that is somehow constructed to be *conjugate* to the old gradient, and, insofar as possible, to all previous directions traversed. Methods that accomplish this construction are called *conjugate gradient* methods.

In §2.7 we discussed the conjugate gradient method as a technique for solving linear algebraic equations by minimizing a quadratic form. That formalism can also be applied to the problem of minimizing a function *approximated* by the quadratic form (10.8.1). Recall that, starting with an arbitrary initial vector $\mathbf{g}_0$ and letting $\mathbf{h}_0 = \mathbf{g}_0$, the conjugate gradient method constructs two sequences of vectors from the recurrence

$$\mathbf{g}_{i+1} = \mathbf{g}_i - \lambda_i \mathbf{A} \cdot \mathbf{h}_i \qquad \mathbf{h}_{i+1} = \mathbf{g}_{i+1} + \gamma_i \mathbf{h}_i \qquad i = 0, 1, 2, \ldots \tag{10.8.2}$$

The vectors satisfy the orthogonality and conjugacy conditions

$$\mathbf{g}_i \cdot \mathbf{g}_j = 0 \qquad \mathbf{h}_i \cdot \mathbf{A} \cdot \mathbf{h}_j = 0 \qquad \mathbf{g}_i \cdot \mathbf{h}_j = 0 \qquad j < i \tag{10.8.3}$$

The scalars λ_i and γ_i are given by

$$\lambda_i = \frac{\mathbf{g}_i \cdot \mathbf{g}_i}{\mathbf{h}_i \cdot \mathbf{A} \cdot \mathbf{h}_i} = \frac{\mathbf{g}_i \cdot \mathbf{h}_i}{\mathbf{h}_i \cdot \mathbf{A} \cdot \mathbf{h}_i} \tag{10.8.4}$$

$$\gamma_i = \frac{\mathbf{g}_{i+1} \cdot \mathbf{g}_{i+1}}{\mathbf{g}_i \cdot \mathbf{g}_i} \tag{10.8.5}$$

Equations (10.8.2) – (10.8.5) are simply equations (2.7.32) – (2.7.35) for a symmetric $\mathbf{A}$ in a new notation. (A self-contained derivation of these results in the context of function minimization is given by Polak [1].)

Now suppose that we knew the Hessian matrix $\mathbf{A}$ in equation (10.8.1). Then we could use the construction (10.8.2) to find successively conjugate directions $\mathbf{h}_i$ along which to line-minimize. After N such, we would efficiently have arrived at the minimum of the quadratic form. But we don't know $\mathbf{A}$.

Here is a remarkable theorem to save the day: Suppose we happen to have $\mathbf{g}_i = -\nabla f(\mathbf{P}_i)$, for some point $\mathbf{P}_i$, where f is of the form (10.8.1). Suppose also that we proceed from $\mathbf{P}_i$ along the direction $\mathbf{h}_i$ to the local minimum of f located at some point $\mathbf{P}_{i+1}$ and then set $\mathbf{g}_{i+1} = -\nabla f(\mathbf{P}_{i+1})$. Then, this $\mathbf{g}_{i+1}$ is the same vector as would have been constructed by equation (10.8.2). (And we have constructed it without knowledge of $\mathbf{A}$!)

Proof: By equation (10.7.3), $\mathbf{g}_i = -\mathbf{A} \cdot \mathbf{P}_i + \mathbf{b}$ and

$$\mathbf{g}_{i+1} = -\mathbf{A} \cdot (\mathbf{P}_i + \lambda \mathbf{h}_i) + \mathbf{b} = \mathbf{g}_i - \lambda \mathbf{A} \cdot \mathbf{h}_i \tag{10.8.6}$$

with λ chosen to take us to the line minimum. But at the line minimum $\mathbf{h}_i \cdot \nabla f = -\mathbf{h}_i \cdot \mathbf{g}_{i+1} = 0$. This latter condition is easily combined with (10.8.6) to solve for λ. The result is exactly the expression (10.8.4). But with this value of λ, (10.8.6) is the same as (10.8.2), q.e.d.

We have, then, the basis of an algorithm that requires neither knowledge of the Hessian matrix $\mathbf{A}$ nor even the storage necessary to store such a matrix. A sequence of directions $\mathbf{h}_i$ is constructed, using only line minimizations, evaluations of the gradient vector, and an auxiliary vector to store the latest in the sequence of $\mathbf{g}$'s.

The algorithm described so far is the original Fletcher-Reeves version of the conjugate gradient algorithm. Later, Polak and Ribiere introduced one tiny, but sometimes significant, change. They proposed using the form

$$\gamma_i = \frac{(\mathbf{g}_{i+1} - \mathbf{g}_i) \cdot \mathbf{g}_{i+1}}{\mathbf{g}_i \cdot \mathbf{g}_i} \tag{10.8.7}$$

instead of equation (10.8.5). "Wait," you say, "aren't they equal by the orthogonality conditions (10.8.3)?" They are equal for exact quadratic forms. In the real world, however, your function is not exactly a quadratic form. Arriving at the supposed minimum of the quadratic form, you may still need to proceed for another set of iterations. There is some evidence [2] that the Polak-Ribiere formula accomplishes the transition to further iterations more gracefully: When it runs out of steam, it tends to reset $\mathbf{h}$ to be down the local gradient, which is equivalent to beginning the conjugate gradient procedure anew.

The following routine implements the Polak-Ribiere variant, which we recommend; but changing one program line, as shown, will give you Fletcher-Reeves. The

routine presumes the existence of a functor (not a function) that returns the function value by overloading `operator()` and also provides a function to set the vector gradient `df[0..n-1]` evaluated at the input point `p`. Here's an example for the function $x_0^2 + x_1^2$:

```
struct Funcd {                                          Name Funcd is arbitrary.
    Doub operator() (VecDoub_I &x)
    {
        return x[0]*x[0]+x[1]*x[1];
    }
    void df(VecDoub_I &x, VecDoub_O &deriv)             Name df is fixed.
    {
        deriv[0]=2.0*x[0];
        deriv[1]=2.0*x[1];
    }
};
```

To use `frprmn`, you need statements like the following:

```
Funcd funcd;
Frprmn<Funcd> frprmn(funcd);
VecDoub p = ...;
p=frprmn.minimize(p);          OK to overwrite initial guess.
```

The function value at the minimum is available as `frprmn.fret`. Note that the constructor takes an optional argument that specifies the function tolerance for the minimization.

The routine calls `linmin` to do the line minimizations. As already discussed, you may wish to use a modified version of `linmin` that uses `Dbrent` instead of `Brent`, i.e., that uses the gradient in doing the line minimizations. See note below (§10.8.1).

mins_ndim.h

```
template <class T>
struct Frprmn : Linemethod<T> {
```

Multidimensional minimization by the Fletcher-Reeves-Polak-Ribiere method.

```
    Int iter;
    Doub fret;                                  Value of the function at the minimum.
    using Linemethod<T>::func;                  Variables from a templated base class
    using Linemethod<T>::linmin;                    are not automatically inherited.
    using Linemethod<T>::p;
    using Linemethod<T>::xi;
    const Doub ftol;
    Frprmn(T &funcd, const Doub ftoll=3.0e-8) : Linemethod<T>(funcd),
        ftol(ftoll) {}
```

Constructor arguments are `funcd`, the function or functor to be minimized, and an optional argument `ftoll`, the fractional tolerance in the function value such that failure to decrease by more than this amount on one iteration signals doneness.

```
    VecDoub minimize(VecDoub_I &pp)
```

Given a starting point `pp[0..n-1]`, performs the minimization on a function whose value and gradient are provided by a functor `funcd` (see text).

```
    {
        const Int ITMAX=200;
        const Doub EPS=1.0e-18;
        const Doub GTOL=1.0e-8;
```

Here `ITMAX` is the maximum allowed number of iterations; `EPS` is a small number to rectify the special case of converging to exactly zero function value; and `GTOL` is the convergence criterion for the zero gradient test.

```
        Doub gg,dgg;
        Int n=pp.size();                        Initializations.
        p=pp;
        VecDoub g(n),h(n);
```

```
        xi.resize(n);
        Doub fp=func(p);
        func.df(p,xi);
        for (Int j=0;j<n;j++) {
            g[j] = -xi[j];
            xi[j]=h[j]=g[j];
        }
        for (Int its=0;its<ITMAX;its++) {          Loop over iterations.
            iter=its;
            fret=linmin();                         Next statement is one possible return:
            if (2.0*abs(fret-fp) <= ftol*(abs(fret)+abs(fp)+EPS))
                return p;
            fp=fret;
            func.df(p,xi);
            Doub test=0.0;                         Test for convergence on zero gradient.
            Doub den=MAX(fp,1.0);
            for (Int j=0;j<n;j++) {
                Doub temp=abs(xi[j])*MAX(abs(p[j]),1.0)/den;
                if (temp > test) test=temp;
            }
            if (test < GTOL) return p;             The other possible return.
            dgg=gg=0.0;
            for (Int j=0;j<n;j++) {
                gg += g[j]*g[j];
//              dgg += xi[j]*xi[j];                This statement for Fletcher-Reeves.
                dgg += (xi[j]+g[j])*xi[j];         This statement for Polak-Ribiere.
            }
            if (gg == 0.0)                         Unlikely. If gradient is exactly zero, then
                return p;                              we are already done.
            Doub gam=dgg/gg;
            for (Int j=0;j<n;j++) {
                g[j] = -xi[j];
                xi[j]=h[j]=g[j]+gam*h[j];
            }
        }
        throw("Too many iterations in frprmn");
    }
};
```

10.8.1 Note on Line Minimization Using Derivatives

Kindly reread §10.6. We here want to do the same thing, but using derivative information in performing the line minimization. Simply replace all occurrences of `Linemethod` in `Frprmn` with `Dlinemethod`. The routine `Dlinemethod` is exactly the same as `Linemethod` except that `Brent` is replaced by `Dbrent` and `F1dim` by `Df1dim`:

mins_ndim.h

```
template <class T>
struct Dlinemethod {
```

Base class for line-minimization algorithms using derivative information. Provides the line-minimization routine `linmin`.

```
    VecDoub p;
    VecDoub xi;
    T &func;
    Int n;
    Dlinemethod(T &funcc) : func(funcc) {}
```

Constructor argument is the user-supplied function or functor to be minimized.

```
    Doub linmin()
```

Line-minimization routine. Given an `n`-dimensional point `p[0..n-1]` and an `n`-dimensional direction `xi[0..n-1]`, moves and resets `p` to where the function or functor `func(p)` takes on

a minimum along the direction xi from p, and replaces xi by the actual vector displacement that p was moved. Also returns the value of func at the returned location p. All of this is actually accomplished by calling the routines bracket and minimize of Dbrent.

```
    {
        Doub ax,xx,xmin;
        n=p.size();
        Df1dim<T> df1dim(p,xi,func);
        ax=0.0;                                  Initial guess for brackets.
        xx=1.0;
        Dbrent dbrent;
        dbrent.bracket(ax,xx,df1dim);
        xmin=dbrent.minimize(df1dim);
        for (Int j=0;j<n;j++) {                  Construct the vector results to return.
            xi[j] *= xmin;
            p[j] += xi[j];
        }
        return dbrent.fmin;
    }
};
```

mins_ndim.h

```
template <class T>
struct Df1dim {
```

Must accompany linmin in Dlinemethod.

```
    const VecDoub &p;
    const VecDoub &xi;
    Int n;
    T &funcd;
    VecDoub xt;
    VecDoub dft;
    Df1dim(VecDoub_I &pp, VecDoub_I &xii, T &funcdd) : p(pp),
        xi(xii), n(pp.size()), funcd(funcdd), xt(n), dft(n) {}
```

Constructor takes as inputs an n-dimensional point p[0..n-1] and an n-dimensional direction xi[0..n-1] from linmin, as well as the functor funcd.

```
    Doub operator()(const Doub x)
```

Functor returning value of the given function along a one-dimensional line.

```
    {
        for (Int j=0;j<n;j++)
            xt[j]=p[j]+x*xi[j];
        return funcd(xt);
    }
    Doub df(const Doub x)
```

Returns the derivative along the line.

```
    {
        Doub df1=0.0;
        funcd.df(xt,dft);                        Dbrent always evaluates the derivative at the
        for (Int j=0;j<n;j++)                        same value as the function, so xt is un-
            df1 += dft[j]*xi[j];                     changed.
        return df1;
    }
};
```

CITED REFERENCES AND FURTHER READING:

Polak, E. 1971, *Computational Methods in Optimization* (New York: Academic Press), §2.3.[1]

Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), Chapter III.1.7 (by K.W. Brodlie).[2]

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §8.7.

10.9 Quasi-Newton or Variable Metric Methods in Multidimensions

The goal of *quasi-Newton* methods, which are also called *variable metric* methods, is not different from the goal of conjugate gradient methods: to accumulate information from successive line minimizations so that N such line minimizations lead to the exact minimum of a quadratic form in N dimensions. In that case, the method will also be quadratically convergent for more general smooth functions.

Both quasi-Newton and conjugate gradient methods require that you are able to compute your function's gradient, or first partial derivatives, at arbitrary points. The quasi-Newton approach differs from the conjugate gradient in the way that it stores and updates the information that is accumulated. Instead of requiring intermediate storage on the order of N, the number of dimensions, it requires a matrix of size $N \times N$. Generally, for any moderate N, this hardly matters.

On the other hand, there is not, as far as we know, any overwhelming advantage that the quasi-Newton methods hold over the conjugate gradient techniques, except perhaps a historical one. Developed somewhat earlier, and more widely propagated, the quasi-Newton methods have by now developed a wider constituency of satisfied users. Likewise, some fancier implementations of quasi-Newton methods (going beyond the scope of this book; see below) have been developed to a greater level of sophistication on issues like the minimization of roundoff error, handling of special conditions, and so on.

Quasi-Newton methods come in two main flavors. One is the *Davidon-Fletcher-Powell (DFP)* algorithm (sometimes referred to as simply *Fletcher-Powell*). The other goes by the name *Broyden-Fletcher-Goldfarb-Shanno (BFGS)*. The BFGS and DFP schemes differ only in details of their roundoff error, convergence tolerances, and similar "dirty" issues that are outside of our scope [1,2]. However, it has become generally recognized that, empirically, the BFGS scheme is superior in these details. We will implement BFGS in this section.

As before, we imagine that our arbitrary function $f(\mathbf{x})$ can be locally approximated by the quadratic form of equation (10.8.1). We don't, however, have any information about the values of the quadratic form's parameters $\mathbf{A}$ and $\mathbf{b}$, except insofar as we can glean such information from our function evaluations and line minimizations.

The basic idea of the quasi-Newton method is to build up, iteratively, a good approximation to the inverse Hessian matrix $\mathbf{A}^{-1}$, that is, to construct a sequence of matrices $\mathbf{H}_i$ with the property

$$\lim_{i \to \infty} \mathbf{H}_i = \mathbf{A}^{-1} \tag{10.9.1}$$

Even better if the limit is achieved after N iterations instead of ∞.

The reason that these methods are called *quasi-Newton* can now be explained. Consider finding a minimum by using Newton's method to search for a zero of the gradient of the function. Near the current point $\mathbf{x}_i$, we have to second order

$$f(\mathbf{x}) = f(\mathbf{x}_i) + (\mathbf{x} - \mathbf{x}_i) \cdot \nabla f(\mathbf{x}_i) + \tfrac{1}{2}(\mathbf{x} - \mathbf{x}_i) \cdot \mathbf{A} \cdot (\mathbf{x} - \mathbf{x}_i) \tag{10.9.2}$$

so

$$\nabla f(\mathbf{x}) = \nabla f(\mathbf{x}_i) + \mathbf{A} \cdot (\mathbf{x} - \mathbf{x}_i) \tag{10.9.3}$$

In Newton's method we set $\nabla f(\mathbf{x}) = 0$ to determine the next iteration point:

$$\mathbf{x} - \mathbf{x}_i = -\mathbf{A}^{-1} \cdot \nabla f(\mathbf{x}_i) \tag{10.9.4}$$

The left-hand side is the finite step we need to take to get to the exact minimum; the right-hand side is known once we have accumulated an accurate $\mathbf{H} \approx \mathbf{A}^{-1}$.

The "quasi" in quasi-Newton is because we don't use the actual Hessian matrix of f, but instead use our current approximation of it. This is often *better* than using the true Hessian. We can understand this paradoxical result by considering the *descent directions* of f at $\mathbf{x}_i$. These are the directions $\mathbf{p}$ along which f decreases: $\nabla f \cdot \mathbf{p} < 0$. For the Newton direction (10.9.4) to be a descent direction, we must have

$$\nabla f(\mathbf{x}_i) \cdot (\mathbf{x} - \mathbf{x}_i) = -(\mathbf{x} - \mathbf{x}_i) \cdot \mathbf{A} \cdot (\mathbf{x} - \mathbf{x}_i) < 0 \tag{10.9.5}$$

which is true if $\mathbf{A}$ is positive-definite. In general, far from a minimum, we have no guarantee that the Hessian is positive-definite. Taking the actual Newton step with the real Hessian can move us to points where the function is *increasing* in value. The idea behind quasi-Newton methods is to start with a positive-definite, symmetric approximation to $\mathbf{A}$ (usually the unit matrix) and build up the approximating $\mathbf{H}_i$'s in such a way that the matrix $\mathbf{H}_i$ remains positive-definite and symmetric. Far from the minimum, this guarantees that we always move in a downhill direction. Close to the minimum, the updating formula approaches the true Hessian and we enjoy the quadratic convergence of Newton's method.

When we are not close enough to the minimum, taking the full Newton step $\mathbf{p}$ even with a positive-definite $\mathbf{A}$ need not decrease the function; we may move too far for the quadratic approximation to be valid. All we are guaranteed is that *initially* f decreases as we move in the Newton direction. Once again we can use the backtracking strategy described in §9.7 to choose a step along the *direction* of the Newton step $\mathbf{p}$, but not necessarily all the way.

We won't rigorously derive the DFP algorithm for taking $\mathbf{H}_i$ into $\mathbf{H}_{i+1}$; you can consult [3] for clear derivations. Following Brodlie (in [2]), we will give the following heuristic motivation of the procedure.

Subtracting equation (10.9.4) at $\mathbf{x}_{i+1}$ from that same equation at $\mathbf{x}_i$ gives

$$\mathbf{x}_{i+1} - \mathbf{x}_i = \mathbf{A}^{-1} \cdot (\nabla f_{i+1} - \nabla f_i) \tag{10.9.6}$$

where $\nabla f_j \equiv \nabla f(\mathbf{x}_j)$. Having made the step from $\mathbf{x}_i$ to $\mathbf{x}_{i+1}$, we might reasonably want to require that the new approximation $\mathbf{H}_{i+1}$ satisfy (10.9.6) as if it were actually $\mathbf{A}^{-1}$, that is,

$$\mathbf{x}_{i+1} - \mathbf{x}_i = \mathbf{H}_{i+1} \cdot (\nabla f_{i+1} - \nabla f_i) \tag{10.9.7}$$

We might also imagine that the updating formula should be of the form $\mathbf{H}_{i+1} = \mathbf{H}_i + \text{correction}$.

What "objects" are around out of which to construct a correction term? Most notable are the two vectors $\mathbf{x}_{i+1} - \mathbf{x}_i$ and $\nabla f_{i+1} - \nabla f_i$, and there is also $\mathbf{H}_i$. There are not infinitely many natural ways of making a matrix out of these objects, especially if (10.9.7) must hold! One such way, the *DFP updating formula*, is

$$\begin{aligned} \mathbf{H}_{i+1} = \mathbf{H}_i &+ \frac{(\mathbf{x}_{i+1} - \mathbf{x}_i) \otimes (\mathbf{x}_{i+1} - \mathbf{x}_i)}{(\mathbf{x}_{i+1} - \mathbf{x}_i) \cdot (\nabla f_{i+1} - \nabla f_i)} \\ &- \frac{[\mathbf{H}_i \cdot (\nabla f_{i+1} - \nabla f_i)] \otimes [\mathbf{H}_i \cdot (\nabla f_{i+1} - \nabla f_i)]}{(\nabla f_{i+1} - \nabla f_i) \cdot \mathbf{H}_i \cdot (\nabla f_{i+1} - \nabla f_i)} \end{aligned} \tag{10.9.8}$$

where $\otimes$ denotes the "outer" or "direct" product of two vectors, a matrix: The ij component of $\mathbf{u} \otimes \mathbf{v}$ is $u_i v_j$. (You might want to verify that 10.9.8 does indeed satisfy 10.9.7.)

The *BFGS updating formula* is exactly the same, but with one additional term,

$$\cdots + [(\nabla f_{i+1} - \nabla f_i) \cdot \mathbf{H}_i \cdot (\nabla f_{i+1} - \nabla f_i)]\, \mathbf{u} \otimes \mathbf{u} \tag{10.9.9}$$

where $\mathbf{u}$ is defined as the vector

$$\mathbf{u} \equiv \frac{(\mathbf{x}_{i+1} - \mathbf{x}_i)}{(\mathbf{x}_{i+1} - \mathbf{x}_i) \cdot (\nabla f_{i+1} - \nabla f_i)} - \frac{\mathbf{H}_i \cdot (\nabla f_{i+1} - \nabla f_i)}{(\nabla f_{i+1} - \nabla f_i) \cdot \mathbf{H}_i \cdot (\nabla f_{i+1} - \nabla f_i)} \tag{10.9.10}$$

(You might also verify that this satisfies 10.9.7.)

You will have to take on faith — or else consult [3] for details of — the "deep" result that equation (10.9.8), with or without (10.9.9), does in fact converge to $\mathbf{A}^{-1}$ in N steps, if f is a quadratic form.

Here now is the routine `dfpmin` that implements the quasi-Newton method and uses `lnsrch` from §9.7. As mentioned at the end of `newt` in §9.7, this algorithm can fail if your variables are badly scaled. You must provide a functor with the same format as the one for `frprmn` in §10.8 to calculate the function and its gradient.

quasinewton.h

```
template <class T>
void dfpmin(VecDoub_IO &p, const Doub gtol, Int &iter, Doub &fret, T &funcd)
```

Given a starting point `p[0..n-1]`, the Broyden-Fletcher-Goldfarb-Shanno variant of Davidon-Fletcher-Powell minimization is performed on a function whose value and gradient are provided by a functor `funcd` (see text in §10.8). The convergence requirement on zeroing the gradient is input as `gtol`. Returned quantities are `p[0..n-1]` (the location of the minimum), `iter` (the number of iterations that were performed), and `fret` (the minimum value of the function). The routine `lnsrch` is called to perform approximate line minimizations.

```
{
    const Int ITMAX=200;
    const Doub EPS=numeric_limits<Doub>::epsilon();
    const Doub TOLX=4*EPS,STPMX=100.0;
```

Here `ITMAX` is the maximum allowed number of iterations; `EPS` is the machine precision; `TOLX` is the convergence criterion on x values; and `STPMX` is the scaled maximum step length allowed in line searches.

```
    Bool check;
    Doub den,fac,fad,fae,fp,stpmax,sum=0.0,sumdg,sumxi,temp,test;
    Int n=p.size();
    VecDoub dg(n),g(n),hdg(n),pnew(n),xi(n);
    MatDoub hessin(n,n);
    fp=funcd(p);                                  Calculate starting function value and gra-
    funcd.df(p,g);                                    dient,
    for (Int i=0;i<n;i++) {                       and initialize the inverse Hessian to the
        for (Int j=0;j<n;j++) hessin[i][j]=0.0;       unit matrix.
        hessin[i][i]=1.0;
        xi[i] = -g[i];                            Initial line direction.
        sum += p[i]*p[i];
    }
    stpmax=STPMX*MAX(sqrt(sum),Doub(n));
    for (Int its=0;its<ITMAX;its++) {             Main loop over the iterations.
        iter=its;
        lnsrch(p,fp,g,xi,pnew,fret,stpmax,check,funcd);
```

The new function evaluation occurs in `lnsrch`; save the function value in `fp` for the next line search. It is usually safe to ignore the value of `check`.

```
        fp=fret;
        for (Int i=0;i<n;i++) {
            xi[i]=pnew[i]-p[i];                   Update the line direction,
```

```
            p[i]=pnew[i];                          and the current point.
        }
        test=0.0;                                  Test for convergence on Δx.
        for (Int i=0;i<n;i++) {
            temp=abs(xi[i])/MAX(abs(p[i]),1.0);
            if (temp > test) test=temp;
        }
        if (test < TOLX)
            return;
        for (Int i=0;i<n;i++) dg[i]=g[i];          Save the old gradient,
        funcd.df(p,g);                             and get the new gradient.
        test=0.0;                                  Test for convergence on zero gradient.
        den=MAX(fret,1.0);
        for (Int i=0;i<n;i++) {
            temp=abs(g[i])*MAX(abs(p[i]),1.0)/den;
            if (temp > test) test=temp;
        }
        if (test < gtol)
            return;
        for (Int i=0;i<n;i++)                      Compute difference of gradients,
            dg[i]=g[i]-dg[i];
        for (Int i=0;i<n;i++) {                    and difference times current matrix.
            hdg[i]=0.0;
            for (Int j=0;j<n;j++) hdg[i] += hessin[i][j]*dg[j];
        }
        fac=fae=sumdg=sumxi=0.0;                   Calculate dot products for the denomi-
        for (Int i=0;i<n;i++) {                        nators.
            fac += dg[i]*xi[i];
            fae += dg[i]*hdg[i];
            sumdg += SQR(dg[i]);
            sumxi += SQR(xi[i]);
        }
        if (fac > sqrt(EPS*sumdg*sumxi)) {         Skip update if fac not sufficiently posi-
            fac=1.0/fac;                               tive.
            fad=1.0/fae;
            The vector that makes BFGS different from DFP:
            for (Int i=0;i<n;i++) dg[i]=fac*xi[i]-fad*hdg[i];
            for (Int i=0;i<n;i++) {                The BFGS updating formula:
                for (Int j=i;j<n;j++) {
                    hessin[i][j] += fac*xi[i]*xi[j]
                        -fad*hdg[i]*hdg[j]+fae*dg[i]*dg[j];
                    hessin[j][i]=hessin[i][j];
                }
            }
        }
        for (Int i=0;i<n;i++) {                    Now calculate the next direction to go,
            xi[i]=0.0;
            for (Int j=0;j<n;j++) xi[i] -= hessin[i][j]*g[j];
        }
    }                                              and go back for another iteration.
    throw("too many iterations in dfpmin");
}
```

Quasi-Newton methods like `dfpmin` work well with the approximate line minimization done by `lnsrch`. The routines `Powell` (§10.7) and `Frprmn` (§10.8), however, need more accurate line minimization, which is carried out by the routine `linmin` in `Linemethod` or `Dlinemethod`.

10.9.1 Quasi-Newton Methods Without Derivatives

In using Newton's method to find a zero of a function in multidimensions, we saw in §9.7 that one can use finite differences to calculate the partial derivatives

instead of providing them analytically. Similarly, dfpmin very often succeeds when the gradient is calculated with finite differences. In our experience, this method often involves less total computation than one of the other methods that avoids analytic derivatives, such as Powell.

To use this idea, all you need to do is supply a suitable functor to dfpmin, which remains unchanged. Here is the code, which is very similar to that of Fdjac in §9.7:

quasinewton.h

```
template <class T>
struct Funcd {
    Doub EPS;
    T &func;
    Doub f;
    Funcd(T &funcc) : EPS(1.0e-8), func(funcc) {}
    Doub operator() (VecDoub_I &x)
    {
        return f=func(x);
    }

    void df(VecDoub_I &x, VecDoub_O &df)
    {
        Int n=x.size();
        VecDoub xh=x;
        Doub fold=f;
        for (Int j=0;j<n;j++) {
            Doub temp=x[j];
            Doub h=EPS*abs(temp);
            if (h == 0.0) h=EPS;
            xh[j]=temp+h;
            h=xh[j]-temp;
            Doub fh=operator()(xh);
            xh[j]=temp;
            df[j]=(fh-fold)/h;
        }
    }
};
```

Set to approximate square root of the machine precision.

Trick to reduce finite-precision error.

10.9.2 Advanced Implementations of Variable Metric Methods

Although rare, it can conceivably happen that roundoff errors cause the matrix $\mathbf{H}_i$ to become nearly singular or non-positive-definite. This can be serious, because the supposed search directions might then not lead downhill, and because nearly singular $\mathbf{H}_i$'s tend to give subsequent $\mathbf{H}_i$'s that are also nearly singular.

There is a simple fix for this rare problem, the same as was mentioned in §10.5: In case of any doubt, you should *restart* the algorithm at the claimed minimum point and see if it goes anywhere. Simple, but not very elegant. Modern implementations of quasi-Newton methods deal with the problem in a more sophisticated way.

Instead of building up an approximation to $\mathbf{A}^{-1}$, it is possible to build up an approximation of $\mathbf{A}$ itself. Then, instead of calculating the left-hand side of (10.9.4) directly, one solves the set of linear equations

$$\mathbf{A} \cdot (\mathbf{x} - \mathbf{x}_i) = -\nabla f(\mathbf{x}_i) \tag{10.9.11}$$

At first glance this seems like a bad idea, since solving (10.9.11) is a process of order N^3 — and anyway, how does this help the roundoff problem? The trick is not to store $\mathbf{A}$ but rather a triangular decomposition of $\mathbf{A}$, its *Cholesky decomposition* (cf. §2.9). The updating formula used for the Cholesky decomposition of $\mathbf{A}$ is of order N^2 and can be arranged to guarantee that the matrix remains positive-definite and nonsingular, even in the presence of finite roundoff. This method is due to Gill and Murray [1,2].

CITED REFERENCES AND FURTHER READING:

Dennis, J.E., and Schnabel, R.B. 1983, *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*; reprinted 1996 (Philadelphia: S.I.A.M.).[1]

Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), Chapter III.1, §3 – §6 (by K. W. Brodlie).[2]

Polak, E. 1971, *Computational Methods in Optimization* (New York: Academic Press), pp. 56ff.[3]

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 467–468.

10.10 Linear Programming: The Simplex Method

The subject of *linear programming*, sometimes called *linear optimization*, concerns itself with the following problem: For n independent variables $x_1, \ldots, x_n$, *minimize* the function

$$\zeta = c_1 x_1 + c_2 x_2 + \cdots + c_n x_n \tag{10.10.1}$$

subject to the nonnegativity conditions

$$x_1 \geq 0, \quad x_2 \geq 0, \quad \ldots \quad x_n \geq 0 \tag{10.10.2}$$

and simultaneously subject to m additional constraints of the form

$$a_{i1} x_1 + a_{i2} x_2 + \cdots + a_{in} x_n \leq b_i \tag{10.10.3}$$

or

$$a_{i1} x_1 + a_{i2} x_2 + \cdots + a_{in} x_n = b_i \tag{10.10.4}$$

Here $i = 1, \ldots, m$. Note that an inequality with a $\geq$ can be converted to a $\leq$ by multiplying by -1. Some formulations of linear programming require you to write all the constraints with the b's nonnegative and separately treat the $\geq$ and $\leq$ constraints. We will use the above formulation, with either sign of b_i, instead. However, it is still useful to refer to the inequalities with $b_i \leq 0$ as "$\geq$ inequalities" (which they would be with $b_i \geq 0$), since, as we shall see, they enter the problem in a different way from the $\leq$ inequalities.

There is no particular significance in the number of constraints m being less than, equal to, or greater than the number of unknowns n. Also, note that there is no special significance to minimizing ζ in equation (10.10.1): We can convert a maximizing problem to a minimizing problem by changing the signs of all the c's. The solution $x_1, \ldots, x_n$ is the same, and the required maximum is the negative of the minimum ζ found.

A set of values $x_1, \ldots, x_n$ that satisfies the constraints (10.10.2) – (10.10.4) is called a *feasible vector*. The function that we are trying to minimize is called the *objective function*. The feasible vector that minimizes the objective function is called the *optimal feasible vector*. An optimal feasible vector can fail to exist for two distinct reasons: (i) There are *no* feasible vectors, i.e., the given constraints

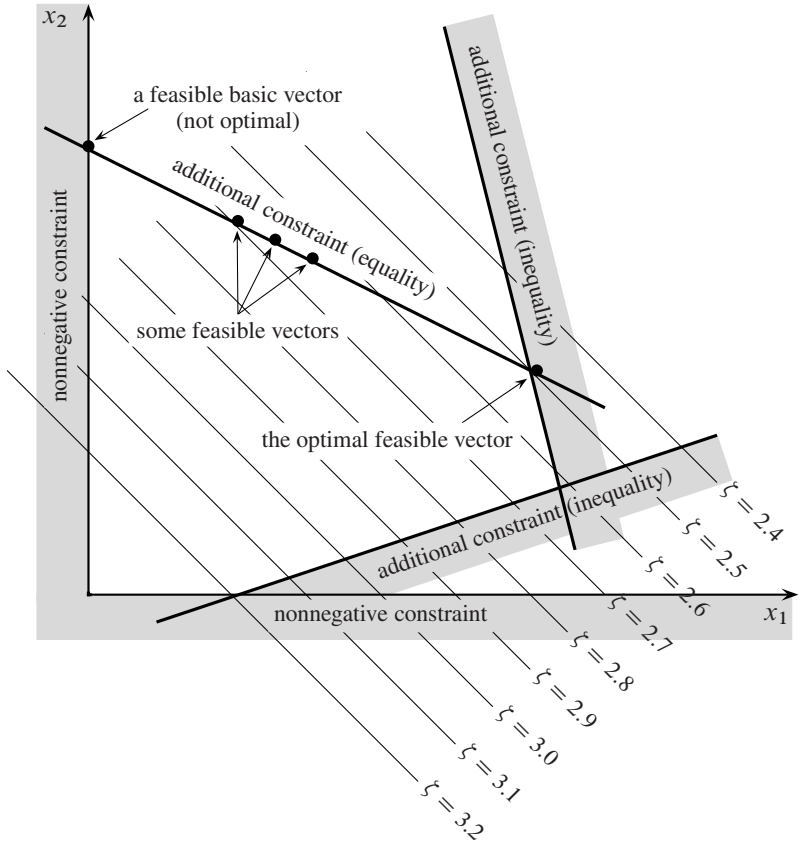


Figure 10.10.1. Basic concepts of linear programming. The case of only two independent variables, x_1, x_2, is shown. The linear function ζ, to be minimized, is represented by its contour lines. Nonnegativity constraints require x_1 and x_2 to be positive. Additional constraints may restrict the solution to regions (inequality constraints) or to surfaces of lower dimensionality (equality constraints). Feasible vectors satisfy all constraints. Feasible basic vectors also lie on the boundary of the allowed region. The simplex method steps among feasible basic vectors until the optimal feasible vector is found.

are incompatible, or (ii) there is no minimum, i.e., there is a direction in n-space where one or more of the variables can be taken to infinity while still satisfying the constraints, giving an unbounded value for the objective function. Figure 10.10.1 summarizes some of the terminology thus far.

As you see, the subject of linear programming is surrounded by notational and terminological thickets. Both of these thorny defenses are lovingly cultivated by a coterie of stern acolytes who have devoted themselves to the field. Actually, the basic ideas of linear programming are quite simple. Avoiding the shrubbery, let's elucidate the basics by means of a couple of specific examples; it should then be quite obvious how to generalize.

Why is linear programming so important? (i) Because "nonnegativity" is the usual constraint on any variable x_i that represents the tangible amount of some physical commodity, like guns, butter, dollars, units of vitamin E, food calories, kilowatt hours, mass, etc. Hence equation (10.10.2). (ii) Because one is often interested in additive (linear) limitations or bounds imposed by man or nature: minimum nutritional

requirement, maximum affordable cost, maximum on available labor or capital, minimum tolerable level of voter approval, etc. Hence equations (10.10.3) – (10.10.4). (iii) Because the function that one wants to optimize may be linear, or else may at least be approximated by a linear function — since that is the problem that linear programming *can* solve. Hence equation (10.10.1). For a short, semipopular survey of linear programming applications, see Bland [1].

10.10.1 Fundamental Theorem of Linear Optimization

Imagine that we start with a full n-dimensional space of candidate vectors. Then (in our mind's eye, at least) we carve away the regions that are eliminated in turn by each imposed constraint. Since the constraints are linear, every boundary introduced by this process is a plane, or rather a hyperplane. Equality constraints of the form (10.10.4) force the feasible region onto hyperplanes of smaller dimension, while inequalities simply divide the then-feasible region into allowed and nonallowed pieces.

When all the constraints are imposed, either we are left with some feasible region or else there are no feasible vectors. Since the feasible region is bounded by hyperplanes, it is geometrically a kind of convex polyhedron or simplex (cf. §10.5). If there is a feasible region, can the optimal feasible vector be somewhere in its interior, away from the boundaries? No, because the objective function is linear. This means that it always has a nonzero vector gradient. This, in turn, means that we could always decrease the objective function by running down the gradient until we hit a boundary wall.

The boundary of any geometrical region has one less dimension than its interior. Therefore, we can now run down the gradient projected into the boundary wall until we reach an edge of that wall. We can then run down that edge, and so on, down through whatever number of dimensions, until we finally arrive at a point, a *vertex* of the original simplex. Since this point has all n of its coordinates defined, it must be the solution of n simultaneous *equalities* drawn from the original set of equalities and inequalities (10.10.2) – (10.10.4).

Points that are feasible vectors and that satisfy n of the original constraints as equalities, including the nonnegativity constraints, are termed *feasible basic vectors*. If $n > m$, then a feasible basic vector has *at least* $n - m$ of its components equal to zero, since at least that many of the constraints (10.10.2) will be needed to make up the total of n. Put the other way, *at most* m components of a feasible basic vector are nonzero.

Put together the two preceding paragraphs and you have the *Fundamental Theorem of Linear Optimization*: If an optimal feasible vector exists, then there is a feasible basic vector that is optimal. (Didn't we warn you about the terminological thicket?)

The importance of the fundamental theorem is that it reduces the optimization problem to a "combinatorial" problem, that of determining which n constraints (out of the $m + n$ constraints in 10.10.2 – 10.10.4) should be satisfied by the optimal feasible vector. We have only to keep trying different combinations, and computing the objective function for each trial, until we find the best.

Doing this blindly would take halfway to forever. The *simplex method*, first published by Dantzig in 1948 (see [2]), is a way of organizing the procedure so that (i) a series of combinations is tried for which the objective function decreases at each step, and (ii) the optimal feasible vector is reached after a number of iterations that

is almost always no larger than of order m or n, whichever is larger. An interesting mathematical sidelight is that this second property, although known empirically ever since the simplex method was devised, was not proved to be true until the 1982 work of Stephen Smale. (For a contemporary account, see [3].)

10.10.2 Writing the General Problem in Standard Form

There is a standard form for linear programming problems, and we have to learn how to write a general problem like (10.10.1) – (10.10.4) in this standard form. For definiteness, consider the problem

$$\text{Minimize} \quad \zeta = -40x_1 - 60x_2 \tag{10.10.5}$$

with the x's nonnegative and also with

$$2x_1 + x_2 \le 70 \tag{10.10.6}$$
$$x_1 + x_2 \ge 40 \tag{10.10.7}$$
$$x_1 + 3x_2 = 90 \tag{10.10.8}$$

First, we rewrite the inequalities as equalities. We do this by adding to the problem so-called *slack variables* $x_{n+1}, x_{n+2}, \ldots$ In our example, equations (10.10.6) and (10.10.7) become

$$2x_1 + x_2 + x_3 = 70 \tag{10.10.9}$$
$$-x_1 - x_2 + x_4 = -40 \tag{10.10.10}$$

(A slack variable like x_4 for a $\ge$ inequality is sometimes called a *surplus* variable.) Requiring the slack variables to be nonnegative makes these equalities equivalent to the original inequalities. Once they are introduced, you treat slack variables on an equal footing with the original variables x_i; then, at the very end, you just ignore them. The simplex solution for each slack variable is simply the amount by which the original inequality is satisfied.

The key idea in the simplex method is to start with a feasible basic vector and make a sequence of exchanges between basic and nonbasic variables. At each step the vector stays feasible (satisfies the constraints), and the objective function decreases (or at least does not increase).

How do we find an initial feasible basic vector to start the procedure? Suppose that our example were changed so that equations (10.10.7) and (10.10.8) were both $\le$ inequalities, like (10.10.6). Then, after introducing slack variables, we would have

$$2x_1 + x_2 + x_3 = 70 \tag{10.10.11}$$
$$x_1 + x_2 + x_4 = 40 \tag{10.10.12}$$
$$x_1 + 3x_2 + x_5 = 90 \tag{10.10.13}$$

In this case it is easy to write down a feasible basic vector: Set the original variables x_1 and x_2 to zero and take $(x_3, x_4, x_5) = (70, 40, 90)$. Here $n = 2$ of the constraints, namely $x_1 \ge 0$, $x_2 \ge 0$, are satisfied as equalities, while $m = 3$ components of the feasible basic vector are nonzero. The variables (x_3, x_4, x_5) are called *basic variables*, while the variables that are zero, (x_1, x_2), are called *nonbasic variables*. Note

that if we write equations (10.10.11) – (10.10.13) as a 3×5 matrix equation, then the last three columns of the matrix, corresponding to the slack variables (x_3, x_4, x_5), form a 3×3 unit matrix.

So $\leq$ constraints are easy. But how do we handle constraints like equations (10.10.7) and (10.10.8)? The trick is again to invent new variables called *artificial variables*. We rewrite equation (10.10.8) as

$$x_1 + 3x_2 + x_5 = 90 \tag{10.10.14}$$

Now equations (10.10.9), (10.10.10), and (10.10.14) are almost in the form to give us an easy initial feasible basic vector by setting $x_1 = x_2 = 0$. The obstacle is equation (10.10.10), which would give a negative value for x_4. We have to precede the actual simplex procedure by a preliminary procedure, called *phase one* of the simplex method, to find an initial feasible vector. (The actual optimization is called *phase two*.)

In phase one, we replace our objective function (10.10.5) by a so-called *auxiliary objective function*,

$$\zeta' \equiv -x_4 \tag{10.10.15}$$

We now perform the simplex method on the auxiliary objective function (10.10.15) with the constraints (10.10.9), (10.10.10), and (10.10.14), starting with the basis given by $x_1 = x_2 = 0$. The variable x_4 starts off negative (at -40). Minimizing the function (10.10.15) drives x_4 toward satisfying $x_4 \geq 0$, the condition for feasibility. In fact, we don't even have to solve phase one all the way to the exact minimum. As we do the exchanges between variables during this phase, we continually redefine the auxiliary objective function at each iteration to be minus the sum of all negative basic variables. As soon as all the basic variables are nonnegative, we are done with phase one.

And what if the first phase *doesn't* drive the auxiliary objective function to a negative value (i.e., all basic variables nonnegative)? That signals that there is *no* initial feasible basic vector, i.e., that the constraints given to us are inconsistent among themselves. Report that fact, and you are done.

An artificial variable in an equality constraint is an example of a *zero variable*, a variable that must vanish in the optimal solution. Typically the way a zero variable gets to be zero is by being nonbasic in the optimal solution. So we can precede phase one with a "phase zero" in which we exchange each zero variable out of the basis.

One last piece of jargon: Slack and artificial variables are often called *logical* variables, to distinguish them from the original independent variables, which are sometimes called *structural* variables.

10.10.3 The Simplex Method: A Worked Example

The easiest way to describe the actual simplex procedure is with a worked example. We write the general linear programming problem in the following form: Minimize the objective function

$$\zeta = \mathbf{c} \cdot \mathbf{x} = c_1 x_1 + c_2 x_2 + \cdots + c_n x_n \tag{10.10.16}$$

subject to the constraints

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \tag{10.10.17}$$

and

$$x_i \geq 0, \qquad i = 1, \ldots, n+m \tag{10.10.18}$$

Here we assume that we started with an $m \times n$ matrix of constraint coefficients given by equations like (10.10.3) and (10.10.4). We then added slack variables to the inequality constraints and artificial variables to the equality constraints so that the constraint matrix is now the $m \times (n+m)$ matrix $\mathbf{A}$. The last m columns form an $m \times m$ identity matrix. Note that the coefficients of the slack variables are taken to be $+1$, so that an original $\geq$ inequality will have a negative right-hand side. For our example given in equations (10.10.5) – (10.10.8), transformed as in equations (10.10.9), (10.10.10), and (10.10.14), the matrix $\mathbf{A}$ has five columns:

$$\mathbf{a}_1 = \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix} \quad \mathbf{a}_2 = \begin{pmatrix} 1 \\ -1 \\ 3 \end{pmatrix} \quad \mathbf{a}_3 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad \mathbf{a}_4 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad \mathbf{a}_5 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \tag{10.10.19}$$

The right-hand side and the objective function coefficients are

$$\mathbf{b} = \begin{pmatrix} 70 \\ -40 \\ 90 \end{pmatrix} \qquad \mathbf{c} = (-40, -60, 0, 0, 0)^T \tag{10.10.20}$$

We partition the matrix $\mathbf{A}$ into two submatrices,

$$\mathbf{A} = \left[\mathbf{A}_B \;\middle|\; \mathbf{A}_N \right] \tag{10.10.21}$$

where we have permuted the columns corresponding to the basic variables to be in $\mathbf{A}_B$, while the nonbasic columns are in $\mathbf{A}_N$. In our example, the initial basic variables are (x_3, x_4, x_5) and the initial basis $\mathbf{A}_B$ is the unit matrix composed of the last three columns of $\mathbf{A}$, $\mathbf{a}_3$, $\mathbf{a}_4$ and $\mathbf{a}_5$. A basic solution of $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ consists of a set of basic and nonbasic variables $[\mathbf{x}_B \mid \mathbf{x}_N]$ with $\mathbf{x}_N = 0$. In our example, initially $\mathbf{x}_N = (x_1, x_2) = 0$. The basic solution satisfies $\mathbf{A}_B \cdot \mathbf{x}_B = \mathbf{b}$, or $\mathbf{x}_B = \mathbf{A}_B^{-1} \cdot \mathbf{b}$.

To derive the simplex method, we need one simple equation: how a basic vector changes as a nonbasic variable (i.e., one that is zero) becomes nonzero. This corresponds to starting at a vertex of the simplex and sliding along an edge toward another vertex. Suppose the variable x_k is the one increasing from zero. The constraint equation $\mathbf{A} \cdot \mathbf{x} = \mathbf{A}_B \cdot \mathbf{x}_B = \mathbf{b}$ becomes

$$\mathbf{A}_B \mathbf{x}'_B + \mathbf{a}_k x_k = \mathbf{b} \tag{10.10.22}$$

since only x_k is nonzero among the nonbasic variables. Multiplying this equation by $\mathbf{A}_B^{-1}$ gives

$$\mathbf{x}'_B = \mathbf{A}_B^{-1} \cdot \mathbf{b} - x_k \mathbf{A}_B^{-1} \cdot \mathbf{a}_k = \mathbf{x}_B - x_k \mathbf{A}_B^{-1} \cdot \mathbf{a}_k \tag{10.10.23}$$

The first application of equation (10.10.23) is to the idea of a *reduced cost*. The coefficient c_i in the objective function (10.10.16) is sometimes called the cost of variable x_i, because it represents the cost of having x_i amount of quantity i in the objective function. The simplex method requires instead the cost of *changing* a variable that is zero (not in the basis) to a nonzero value. If the initial value of the objective function is $\mathbf{c} \cdot \mathbf{x}_B = \mathbf{c}_B \cdot \mathbf{x}_B$ and the final value is $\mathbf{c} \cdot (\mathbf{x}'_B + x_k \mathbf{e}_k)$, where $\mathbf{e}_k$

is a unit vector, then using equation (10.10.23) you find that the difference is $x_k u_k$, where the reduced cost of x_k is given by

$$u_k = c_k - \mathbf{a}_k \cdot \mathbf{y}, \qquad \mathbf{y} \equiv (\mathbf{A}_B^{-1})^T \cdot \mathbf{c}_B \tag{10.10.24}$$

Note that if $u_k < 0$, you can make the value of the objective function smaller by bringing x_k into the basis (making it nonzero).

The simplex procedure consists of the following steps:

1. Find a feasible basis (phase 1)
2. Compute the reduced costs (10.10.24) for all x_k not in the basis.
3. If $u_k \geq 0$ for all k, the solution is optimal: No exchange will improve things. Otherwise, choose k corresponding to the most negative u_k as the entering column.
4. Choose the leaving column i from the *minimum ratio test* (motivated below): Compute
$$\mathbf{x}_B = \mathbf{A}_B^{-1} \cdot \mathbf{b}, \qquad \mathbf{w} = \mathbf{A}_B^{-1} \cdot \mathbf{a}_k \tag{10.10.25}$$
For each component $w_i > 0$, compute the ratio x_B^i / w_i. Choose i that corresponds to the smallest such ratio. (If no $w_i > 0$, the objective function is unbounded. Exit and report this.)
5. Exchange columns i and k and go back to step 2.

The minimum ratio test is the second application of equation (10.10.23), which can be written as

$$(x_B^i)' = x_B^i - x_k w_i \tag{10.10.26}$$

where w_i is defined in equation (10.10.25). For each $w_i > 0$, x_B^i decreases as x_k increases from zero. The minimum ratio test selects the i corresponding to the first x_B^i to hit zero, while the other basis variables are still positive. The idea is to allow x_k to be as big as possible so that the objective function is reduced as much as possible by bringing it into the basis.

Let's see how this applies to our example. We start with phase zero, where we remove the zero variable x_5 from the basis. Suppose we choose x_2 to be the incoming variable (x_1 would work fine, too). Using x_2, we find for the new basis and its inverse

$$\mathbf{A}_B = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & -1 \\ 0 & 0 & 3 \end{pmatrix} \qquad \mathbf{A}_B^{-1} = \begin{pmatrix} 1 & 0 & -\frac{1}{3} \\ 0 & 1 & \frac{1}{3} \\ 0 & 0 & \frac{1}{3} \end{pmatrix} \tag{10.10.27}$$

The new basic solution is

$$\mathbf{x}_B = \begin{pmatrix} x_3 \\ x_4 \\ x_2 \end{pmatrix} = \mathbf{A}_B^{-1} \cdot \mathbf{b} = \begin{pmatrix} 40 \\ -10 \\ 30 \end{pmatrix} \tag{10.10.28}$$

The solution (10.10.28) is not feasible because x_4 is negative. We enter phase one, with $\zeta' = \overline{\mathbf{c}} \cdot \mathbf{x} = -x_4$, i.e.,

$$\overline{\mathbf{c}}_B = \begin{pmatrix} 0 \\ -1 \\ 0 \end{pmatrix} \tag{10.10.29}$$

Here the order of elements corresponds to the order (x_3, x_4, x_2) for the basic variables. We compute the reduced costs from equation (10.10.24). Only $k = 1$ is relevant, since x_5 is never allowed to re-enter the basis (zero variable). We find

$$u_1 = -\mathbf{a}_1 \cdot (\mathbf{A}_B^{-1})^T \cdot \bar{\mathbf{c}}_B = -\tfrac{2}{3} \tag{10.10.30}$$

which is negative, confirming that x_1 should enter. For the minimum ratio test to determine which variable leaves, we need the quantity

$$\mathbf{A}_B^{-1} \cdot \mathbf{a}_1 = \begin{pmatrix} \frac{5}{3} \\ -\frac{2}{3} \\ \frac{1}{3} \end{pmatrix} \tag{10.10.31}$$

So the ratios of the elements in equation (10.10.28) to those in equation (10.10.31) are

$$\frac{40}{5/3} = 24, \qquad \frac{-10}{-2/3} = 15, \qquad \frac{30}{1/3} = 90 \tag{10.10.32}$$

The middle ratio is the minimum, so x_4 goes out. (Note that in phase one we relax the requirement that $w_i > 0$, since we haven't yet made all the variables nonnegative.)

Now the basic variables are (x_3, x_1, x_2). Proceeding as before, we find

$$\mathbf{A}_B = \begin{pmatrix} 1 & 2 & 1 \\ 0 & -1 & -1 \\ 0 & 1 & 3 \end{pmatrix} \qquad \mathbf{A}_B^{-1} = \begin{pmatrix} 1 & \frac{5}{2} & \frac{1}{2} \\ 0 & -\frac{3}{2} & -\frac{1}{2} \\ 0 & \frac{1}{2} & \frac{1}{2} \end{pmatrix} \tag{10.10.33}$$

and

$$\mathbf{x}_B = \begin{pmatrix} x_3 \\ x_1 \\ x_2 \end{pmatrix} = \mathbf{A}_B^{-1} \cdot \mathbf{b} = \begin{pmatrix} 15 \\ 15 \\ 25 \end{pmatrix} \tag{10.10.34}$$

All the variables are positive, so the basis is feasible and we enter phase two, with $\mathbf{c}_B = (0, -40, -60)^T$. We find the reduced cost $u_4 = -30$, so x_4 re-enters the basis. The minimum ratio test (10.10.25) gives a minimum for the term involving x_3, so the next basis is (x_4, x_1, x_2). The basic solution turns out to be $(6, 24, 22)$. When we compute the reduced cost u_3 for this basis, it is positive, so we are done. The minimum occurs at $x_1 = 24$, $x_2 = 22$, and the minimum value, obtained by substitution in the objective function, is -2280. The meaning of $x_4 = 6$ is that the inequality (10.10.7) is satisfied by 6. The other two constraints are satisfied as equalities.

The graphical interpretation of the solution procedure is shown in Figure 10.10.2. The initial basic vector is at the origin. We first proceed to the vertex A, which puts us on the line where the equality (10.10.8) is satisfied. This is not a feasible point, since we are on the wrong side of the line Y. So we move along the line X to the vertex B, which is now feasible. Finally we move to vertex C, which is the minimum value of the objective function.

10.10.4 Degeneracy

Nonbasic variables in a basic feasible solution are all zero. If any *basic* variable is zero, we say the basis is *degenerate*. Geometrically, this situation corresponds in n dimensions to having more than n hyperplanes intersect at a vertex. Degeneracy can cause problems in the simplex method. Consider the simple case when three

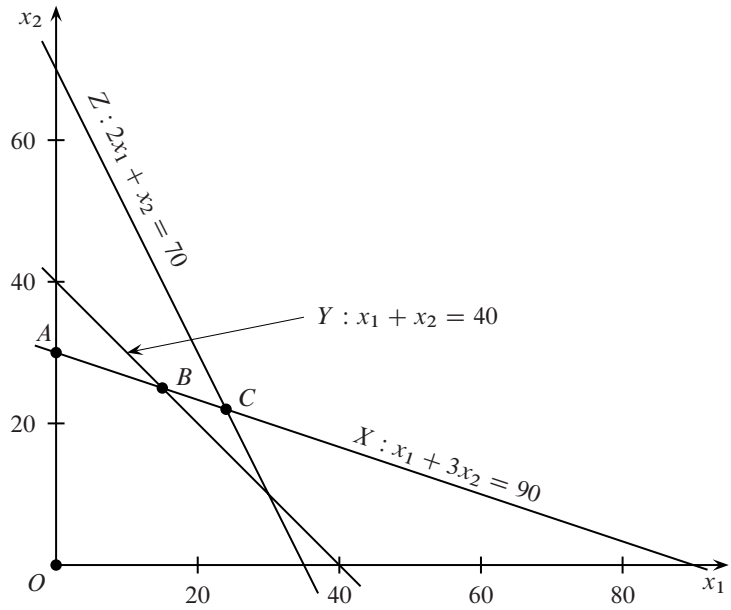


Figure 10.10.2. Graphical interpretation of the simplex solution of the problem (10.10.5) – (10.10.8). The initial basic vector is at the origin O. To satisfy the equality (10.10.8), the first step moves to A, on the line X. This is not yet a feasible point, since it is on the wrong side of the line Y. The next move is to B, which is feasible. We enter phase two, and find that we can reduce the objective function by moving to C. No further moves are possible, so we are done. Note that the figure is really the projection of a five-dimensional simplex onto the x_1-x_2 plane.

lines intersect at a point in two dimensions. Only two of the lines are necessary to define the vertex. When the leaving variable is chosen, it can correspond to the third direction at the vertex. On making this change in the basis, the objective function doesn't improve. You can see this algebraically from equation (10.10.26): If $x_{\boldsymbol{B}}^i = 0$ and $w^i > 0$, then a step size of zero is required for the new variable x_k. Clearly special measures need to be taken.

Degeneracy allows the possibility of *cycling*, where you keep exchanging the same set of vectors in and out of the basis without making any progress. In practice, however, cycling is almost never a problem. More common in very degenerate problems is *stalling*, where you spend a long time making exchanges before finally leaving the vertex.

10.10.5 Sparseness and Stability

If you examine the operations carried out during each simplex step, you see that a key ingredient is to solve equations of the form $\mathbf{A}_{\boldsymbol{B}} \cdot \mathbf{x} = \mathbf{b}$ and similar equations with $\mathbf{A}_{\boldsymbol{B}}^T$. We know that a good method for doing this, absent other considerations, is to use the LU decomposition of $\mathbf{A}_{\boldsymbol{B}}$ (cf. §2.3), since we can use partial pivoting to maintain stability. Decomposing $\mathbf{A}_{\boldsymbol{B}}$ afresh each step is expensive, but since successive bases differ only by the replacement of a single column, one can use techniques analogous to the Sherman-Morrison formula (§2.7) to update $\mathbf{L}$ and $\mathbf{U}$.

However, most linear programming problems that occur in practice have a constraint matrix $\mathbf{A}$ that is very sparse. It is crucial to take advantage of this sparseness in the linear algebra procedures that make up each simplex step, since real-life prob-

lems can involve many thousands or even millions of constraints and variables. Standard LU decomposition with partial pivoting to maintain stability is not desirable, because it leads to excessive *fill-in*, that is, the generation of nonzero matrix elements where there were zeros before. Instead, one chooses the pivot element by a trade-off between stability and sparsity. A popular strategy is based on the *Markowitz criterion* [4]. Here the pivot is a nonzero with the smallest product of the number of other nonzeros in its row and its column. Empirically, the Markowitz criterion works about as well as any other general strategy for minimizing fill-in. In linear programming applications, one also generally needs to impose some kind of stability criterion. In *threshold partial pivoting*, no pivot is less than α times the largest element in its row, where α is a parameter between zero and one. A typical value is $\alpha \sim 0.1$; $\alpha = 1$ gives straight partial pivoting.

The first stable updating procedure for sparse LU decomposition was given by Bartels and Golub [5,6]. This procedure updates $\mathbf{L}$ and $\mathbf{U}$ when a column is replaced in $\mathbf{A}_B$. A good sparse LU algorithm that includes the Bartels-Golub update is that of [7]. It is freely available in the software package LUSOL as part of [8], and we use it in our pedagogical implementation described below.

There are newer methods for the LU decomposition and the update procedure. According to [9], the best of these is probably that of [10].

10.10.6 State-of-the-Art Simplex Codes

A high-quality implementation of the simplex method will have a number of features that we have not discussed so far.

- It will implement more than one variant of the simplex method. In addition to the algorithm we have described, the *primal algorithm*, it will typically also implement the so-called *dual algorithm* (duality is discussed in §10.11). The number of iterations can be significantly reduced by the appropriate use of more than one algorithm.
- It will accept multiple input formats for the specification of the problem, with suitable error checking.
- It will preprocess the problem, with the aim of reducing its size and improving its numerical properties. Many complex problems are inadvertently specified in reducible form.
- It will have multiple options for scaling the problem. As with solving linear equations, no universal scaling algorithm is known for linear programming.
- It will have multiple options for starting the iteration, including several procedures for phase 1.
- It will have several pricing strategies (procedures for selecting the incoming variable). These go by names like multiple pricing, Devex, and steepest edge.
- It will have multiple pivot methods (variants of the ratio test) for the outgoing variable.
- It will handle bounded variables, that is, variables that satisfy a requirement $l_i \le x_i \le u_i$ instead of $x_i \ge 0$. It is possible to handle such bounds using slack variables, but that increases the size of the matrix $\mathbf{A}$. A slight generalization of the algorithm we have described allows you to handle bounded variables directly.
- It will have efficient sparse matrix algorithms.

All these issues are thoroughly discussed in [9].

There are several excellent public domain simplex implementations that incorporate most of the above items. These include CLP [11], GLPK [12] and lp_solve [8]. If you are planning to do any serious LP solving, you should definitely explore these options. It may even be worthwhile to invest in a commercial LP solver. For more information on all these options, see [13]. But first look at the next section, where *interior-point methods* are discussed. It appears now that for many very big problems (but not all), interior-point methods can out-perform simplex methods [14-16]. Even for moderate-sized problems, an interior-point method could be your best choice.

The simplex codes mentioned above are large software efforts, with many thousands of lines of code. They are quite formidable if you want to study how the simplex algorithm works and play around with various options. Accordingly, in a Webnote [17], we give a pedagogical implementation of the simplex method. Even though it uses reasonably good sparse matrix algebra, it is slower than the public domain implementations by one to two orders of magnitude on problems with $\sim 10^4$ variables. If you don't care about the simplex algorithm and you just want a simple method to get your problem solved quickly without getting a public domain code up and running, take a look at our pedagogical interior-point code in the next section.

10.10.7 Other Topics Briefly Mentioned

Problems where the objective function and/or one or more of the constraints are replaced by expressions nonlinear in the variables are called *nonlinear programming problems*. The literature on such problems is vast, but outside our scope. The special case of quadratic expressions is called *quadratic programming*. Optimization problems where the variables take on only integer values are called *integer programming* problems, a special case of *discrete optimization* generally. Section 10.12 looks at a particular kind of discrete optimization problem.

CITED REFERENCES AND FURTHER READING:

Bland, R.G. 1981, "The Allocation of Resources by Linear Programming," *Scientific American*, vol. 244 (June), pp. 126–144.[1]

Dantzig, G.B. 1963, *Linear Programming and Extensions* (Princeton, NJ: Princeton University Press).[2]

Kolata, G. 1982, "Mathematician Solves Simplex Problem," *Science*, vol. 217, p. 39.[3]

Markowitz, H.M. 1957, "The Elimination Form of the Inverse and Its Application to Linear Programming," *Management Science*, vol. 3, pp. 255–269.[4]

Bartels. R.H., and Golub, G.H. 1969, "The Simplex Method of Linear Programming Using the LU Decomposition," *Communications of the ACM*, vol. 12, pp. 266–268.[5]

Bartels, R.H. 1971, "A Stabilization of the Simplex Method," *Numerische Mathematik*, vol. 16, pp. 414–434.[6]

Gill, P.E., Murray, W., Saunders, M.A. and Wright, M.H. 1987, "Maintaining LU Factors of a General Sparse Matrix," *Linear Algebra and Its Applications*, vol. 88/89, pp. 239–270.[7]

lp_solve, `http://groups.yahoo.com/group/lp_solve`.[8]

Maros, I. 2003, *Computational Techniques of the Simplex Method* (Boston: Kluwer).[9]

Suhl, U., and Suhl, L. 1990, "Computing Sparse LU Factorizations for Large-Scale Linear Programming Bases," *ORSA Journal on Computing*, vol. 2, pp. 325–335; 1993, "A Fast LU Update for Linear Programming," *Annals of Operations Research*, vol. 43, pp. 33–47.[10]

CLP, `http://www.coin-or.org/Clp`.[11]

GLPK (GNU Linear Programming Kit), `http://www.gnu.org/software/glpk`.[12]

Linear Programming Frequently Asked Questions, `http://www-unix.mcs.anl.gov/otc/Guide/faq/linear-programming-faq.html`.[13]

Lustig, I.J., Marsten, R.E., and Shanno, D.F. 1994, "Interior Point Methods: Computational State of the Art," *ORSA Journal on Computing*, vol. 6, pp. 1–14. See also Commentaries and Rejoinder, pp. 15–36.[14]

Wright, S.J. 1997, *Primal-Dual Interior-Point Methods* (Philadelphia: S.I.A.M.).[15]

Bixby, R.E. 2002, "Solving Real-World Linear Programs: A Decade and More of Progress," *Operations Research*, vol. 50, pp. 3–15.[16]

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §4.10.

Nemhauser, G.L., Rinnooy Kan, A.H.G., and Todd, M.J. (eds.) 1989, *Optimization* (Amsterdam: North-Holland).

Gill, P.E., Murray, W., and Wright, M.H. 1991, *Numerical Linear Algebra and Optimization*, vol. 1 (Redwood City, CA: Addison-Wesley), Chapters 7–8.

Ignizio, J.P., and Cavalier, T.M. 1994, *Linear Programming* (Englewood Cliffs, NJ: Prentice-Hall). [Undergraduate text]

Vanderbei, R.J. 2001, *Linear Programming: Foundations and Extensions*, 2nd ed. (Boston: Kluwer). [Undergraduate/graduate text]

Chvátal, V. 1983, *Linear Programming* (New York: Freeman). [Undergraduate text]

Gass, S.I. 1985, *Linear Programming: Methods and Applications*, 5th ed. (New York: McGraw-Hill), reprinted 2003 (New York: Dover). [Undergraduate text]

Murty, K.G. 1983, *Linear Programming* (New York: Wiley). [Undergraduate text]

Numerical Recipes Software 2007, "Routine Implementing the Simplex Method," *Numerical Recipes Webnote No. 12*, at `http://www.nr.com/webnotes?12` [17]

10.11 Linear Programming: Interior-Point Methods

As we mentioned in §10.10, the worst-case number of iterations for the simplex method is an exponential function of n. (The worst case occurs for $m = n$.) The average number of iterations, however, is a small multiple of m. For a long time it was not known if there was another algorithm for linear programming that would be bounded by, for example, some polynomial in n. In 1979, Khachian published a new algorithm [1], the *ellipsoid method*, that is in fact polynomial in n. Disappointingly, however, in practical implementations it was much slower than the simplex method.

In 1984, the field was electrified by Karmarkar's paper [2] describing an *interior-point* method. Not only was it polynomial in n, but he claimed it solved large LP problems significantly faster than the simplex method. This claim turned out to be somewhat exaggerated, but in the frenzy of activity over the next decade interior-point algorithms were developed that *do* solve many problems much faster than the simplex method, especially very large problems. Ironically, the rivalry between the two algorithms led to improvements of about a factor of 100 in the simplex method itself over the same period. We give some recommendations on which method to use at the end of this section.

Originally, interior-point methods traversed the interior of the feasible region, homing in on the optimal vertex. So-called infeasible interior-point methods follow

a path in the interior of the nonnegative region, $x_i \geq 0$, $i = 1, \ldots, n$, but possibly through the infeasible region.

To understand how interior-point methods work, we need to develop some more theory about linear programming, in particular about *duality*.

10.11.1 Dual Problem

As we saw in §10.10, any LP problem can be written in standard form:

$$\begin{array}{ll} \text{minimize} & \mathbf{c} \cdot \mathbf{x} \\ \text{subject to} & \mathbf{A} \cdot \mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq 0 \end{array} \tag{10.11.1}$$

Here slack variables have been appended to the x's to write all inequality constraints as equalities, but no other logical variables have been added. This is called the *primal* problem. Recall that if the constraints in (10.11.1) are satisfied, we say that $\mathbf{x}$ is a feasible point.

The *dual* problem corresponding to (10.11.1) interchanges the roles of variables and constraints: Corresponding to the m constraints is a set of variables $(y_1, \ldots, y_m)$ determined by

$$\begin{array}{ll} \text{maximize} & \mathbf{b} \cdot \mathbf{y} \\ \text{subject to} & \mathbf{A}^T \cdot \mathbf{y} \leq \mathbf{c} \\ & \mathbf{y} \text{ free} \end{array} \tag{10.11.2}$$

Here "free" means unconstrained. Most of the textbooks mentioned at the end of the previous section discuss exactly how to go from the primal problem to its dual. For a hint, see §16.5.2, where a different primal-dual problem is discussed. After that, a particularly clear discussion is in [3]. Note how the constraint matrix for the dual problem is the transpose of the matrix for the primal problem. Forming the dual of the dual simply takes you back to the primal problem.

The dual problem (10.11.2) can be rewritten by adding slack variables $(z_1, \ldots, z_n)$:

$$\begin{array}{ll} \text{maximize} & \mathbf{b} \cdot \mathbf{y} \\ \text{subject to} & \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} = \mathbf{c} \\ & \mathbf{z} \geq 0, \quad \mathbf{y} \text{ free} \end{array} \tag{10.11.3}$$

If $(\mathbf{y}, \mathbf{z})$ satisfy the constraints in (10.11.3), we say that they are *dual feasible*. For a good introduction to the meaning of the relation between primal and dual problems, see [4].

The *weak duality theorem* asserts that the value of the dual objective function provides a lower bound to the value of the objective function if they are each evaluated at feasible points. Proof: $\mathbf{b} \cdot \mathbf{y} = \mathbf{y} \cdot \mathbf{A} \cdot \mathbf{x} = \mathbf{x} \cdot \mathbf{A}^T \cdot \mathbf{y} \leq \mathbf{x} \cdot \mathbf{c}$. The difference

$$\mathbf{c} \cdot \mathbf{x} - \mathbf{b} \cdot \mathbf{y} \tag{10.11.4}$$

is called the *duality gap*. If the primal is unbounded (the objective can be made arbitrarily negative), then the dual must be infeasible, and vice versa. Moreover, we

have the *strong duality theorem*: If either the primal or the dual has a finite optimal solution, so does the other, and $\mathbf{c} \cdot \mathbf{x} = \mathbf{b} \cdot \mathbf{y}$ for the optimal solution.

There is a further important relation between the primal and dual variables at optimality. Consider a particular x_j and the corresponding $z_j = (\mathbf{c} - \mathbf{A}^T \cdot \mathbf{y})_j$. The *Karush-Kuhn-Tucker complementarity condition* says that they can't both be strictly greater than zero: At least one must be equal to zero. In other words,

$$x_j z_j = 0, \quad j = 1, \ldots, n \tag{10.11.5}$$

Adopting the convention that an uppercase letter denotes a matrix with the corresponding lowercase vector along the diagonal, we can write equation (10.11.5) alternatively as

$$\mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} = 0, \qquad \mathbf{X} = \mathrm{diag}(x_1, \ldots, x_n), \quad \mathbf{Z} = \mathrm{diag}(z_1, \ldots, z_n), \quad \mathbf{e} = (1, \ldots, 1) \tag{10.11.6}$$

Since each x_j and z_j is nonnegative, equation (10.11.5) is equivalent to $\mathbf{x} \cdot \mathbf{z} = 0$. In fact, this result holds in both directions: The *complementary slackness theorem* says that feasible solutions are optimal if and only if $\mathbf{x} \cdot \mathbf{z} = 0$. It is easy to show that for feasible solutions, $\mathbf{x} \cdot \mathbf{z}$ is simply equal to the duality gap (10.11.4).

Note that complementary slackness allows the possibility that both x_j and z_j might be zero at an optimal solution. *Strict complementarity* is the property that exactly one of these quantities is zero for all j. The *Goldman-Tucker theorem* says that if the primal and dual are feasible, there exists a strictly complementary pair of optimal solutions. As we'll see, interior-point methods find such a solution.

10.11.2 The KKT Conditions

Linear programming is a special case of general constrained optimization, where one wants to minimize some function $f(\mathbf{x})$ subject to constraints. The general optimality conditions are called the Karush-Kuhn-Tucker or KKT conditions. Specialized to the LP problem (10.11.1), the KKT conditions are

$$\begin{aligned} \mathbf{A} \cdot \mathbf{x} &= \mathbf{b} & \mathbf{x} &\geq 0 \\ \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} &= \mathbf{c} & \mathbf{z} &\geq 0 \\ \mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} &= 0 & \mathbf{y} &\text{ free} \end{aligned} \tag{10.11.7}$$

Note that these are exactly the conditions that follow from strong duality and complementarity. Later we will see how to derive these conditions directly using Lagrange multipliers to handle the constraints.

The KKT conditions (10.11.7) are necessary and sufficient for $\mathbf{x}$ to be an optimal solution of (10.11.1). Moreover, they are necessary and sufficient for $(\mathbf{x}, \mathbf{y}, \mathbf{z})$ to solve the primal and dual problems (10.11.1) and (10.11.2). In this case, we call $(\mathbf{x}, \mathbf{y}, \mathbf{z})$ a *primal-dual solution*. Primal-dual interior-point methods solve the equations (10.11.7) in such a way that the inequalities are satisfied strictly at every iteration, that is, $\mathbf{x}, \mathbf{z} > 0$.

The equations are solved using a variant of Newton's method. Recall that if we define the vector of equations in (10.11.7) as

$$\mathbf{F}(\mathbf{v}) = \begin{bmatrix} \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \\ \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} - \mathbf{c} \\ \mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} \end{bmatrix} = 0 \tag{10.11.8}$$

where $\mathbf{v}$ is shorthand for $(\mathbf{x}, \mathbf{y}, \mathbf{z})$, then Newton's method determines the update $\Delta\mathbf{v}$ to the current point by solving

$$\mathbf{J} \cdot \Delta\mathbf{v} = -\mathbf{F} \tag{10.11.9}$$

Here $\mathbf{J}$ is the Jacobian matrix of $\mathbf{F}$ (see §9.7). A full step with this value of $\Delta\mathbf{v}$ is usually not allowed because it would violate the condition $(\mathbf{x}, \mathbf{z}) \geq 0$. So the new iterate is chosen from a line search along the Newton direction:

$$\mathbf{v}_{\text{new}} = \mathbf{v}_{\text{old}} + \alpha\Delta\mathbf{v}, \qquad \alpha \in (0, 1] \tag{10.11.10}$$

You choose $\alpha = 1$ if possible; otherwise, you choose the maximum α that preserves nonnegativity.

Note the importance of keeping the nonnegative variables strictly positive at all times: The Newton equation for $x_j\, z_j = 0$ is $x_j \Delta z_j + z_j \Delta x_j = -x_j\, z_j$. Suppose z_j is zero. Then the Newton equation becomes $x_j \Delta z_j = 0$, or $\Delta z_j = 0$. So z_j remains zero if it ever becomes zero. The algorithm can never recover. Of course, one should also expect difficulties if any variable gets "too close" to zero.

This simple damped Newton's method is not a practical algorithm, because too often the allowed stepsize becomes very small ($\alpha \ll 1$). There are two important modifications that are crucial to producing a viable algorithm:

- Change the search direction so that it aims toward the "center" of the nonnegative region. The idea is to allow larger steps before one of the variables would become negative.
- Don't allow the variables to come "too close" to the boundary of the nonnegative region. As discussed above, little progress tends to be made from such points.

10.11.3 The Central Path

One way to bias the search direction away from boundary is to arrange for all the complementarity pairs $x_j z_j$ to converge to zero at the same rate, say $x_j z_j = \tau$, where $\tau \to 0$ during the iterations. In other words, modify the last equation in (10.11.8) so that the system becomes

$$\mathbf{F}(\mathbf{v}) = \begin{bmatrix} \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \\ \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} - \mathbf{c} \\ \mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \tau\mathbf{e} \end{bmatrix} \tag{10.11.11}$$

The set of solutions $\mathbf{v}(\tau)$ to equations (10.11.11) defines the *central path*. Primal-dual algorithms take steps toward points on the central path with $\tau > 0$. During the iterations, $\tau \to 0$ and the central path homes in on the optimal solution.

If you plot the central path in the hyperspace of $(\mathbf{x}, \mathbf{z})$ coordinates, it's some contorted line that doesn't look central to anything. However, if you plot it in coordinates $(x_1 z_1, x_2 z_2, \ldots)$, you see that it is equidistant from all the coordinate surfaces, with the optimal solution at the origin. When the current iterate is close to the central path, the next iteration can make a large step toward the optimal solution. When the current iterate is close to one of the boundaries, a good algorithm makes the next iteration get close to the central path again.

10.11.4 Path-Following Methods

Path-following methods don't just aim steps in the direction of the central path; they explicitly attempt to stay close to it. These methods are currently the most successful interior-point methods. In primal-dual methods, the duality gap (10.11.4), which is equal to $\mathbf{x} \cdot \mathbf{z}$ for feasible points, provides a figure-of-merit for how close one is to the optimal solution. Accordingly, we set

$$\mu = \mathbf{x} \cdot \mathbf{z}/n, \qquad \tau = \mu\delta, \quad \delta \in [0, 1] \tag{10.11.12}$$

The quantity δ is called the *centering parameter*, while μ is called the *duality measure*. If $\delta = 1$, the Newton step calculated from (10.11.11) is in a *centering direction*, toward a point at which each product $x_j\, z_j$ is equal to the average value μ defined in (10.11.12). On the other hand, the value $\delta = 0$ defines the Newton step for the original system (10.11.8). Good algorithms use intermediate values to trade off between improving centrality and reducing μ.

Methods that keep δ close to 1, so that unit steps ($\alpha = 1$) stay close to the central path, are called *short-step* methods. Methods that allow small values of δ are called *long-step* methods (less conservative choices of δ). There is an interesting gap between theory and practice between the methods. Short-step methods have been proved to converge in $O(\sqrt{n} \log \frac{1}{\epsilon})$ iterations, where ϵ is the desired tolerance. Long-step methods take $O(n \log \frac{1}{\epsilon})$ iterations, according to theory. Yet in practice short-step methods take hopelessly small steps, while long-step methods provide practical algorithms.

This is a somewhat academic discussion anyway. Real-life examples take many fewer than $O(\sqrt{n})$ iterations — a few dozen is typical for large problems.

10.11.5 Barrier Methods

Introducing a "penalty" function is a standard technique to enforce a constraint in general optimization problems. For example, to enforce the condition $\mathbf{x} \geq 0$, consider the logarithmic penalty function

$$\sum_{j=1}^{n} \log x_j \tag{10.11.13}$$

If any $x_j \to 0$, this function tends to $-\infty$. So instead of trying to minimize $\mathbf{c} \cdot \mathbf{x}$ in the standard primal problem (10.11.1), consider minimizing

$$\mathbf{c} \cdot \mathbf{x} - \tau \sum_{j=1}^{n} \log x_j \tag{10.11.14}$$

If one takes the limit $\tau \to 0$ after the minimization, we expect this to be equivalent to the original problem.

Equation (10.11.14) is called a *logarithmic barrier function*. It defines a family of nonlinear objective functions that gives the solution to the original problem as the parameter $\tau \to 0$.

The power of the barrier function idea is that it lets us handle the constraint $\mathbf{x} \geq 0$ with calculus. To minimize (10.11.14) subject to the constraint $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$, introduce a Lagrange multiplier $-\mathbf{y}$ and extremize the Lagrangian

$$L = \mathbf{c} \cdot \mathbf{x} - \tau \sum_{j=1}^{n} \log x_j - \mathbf{y} \cdot (\mathbf{A} \cdot \mathbf{x} - \mathbf{b}) \tag{10.11.15}$$

The optimality conditions $\nabla_x L = 0$ and $\nabla_y L = 0$ give

$$\begin{aligned} \mathbf{A} \cdot \mathbf{x} &= \mathbf{b} \\ \mathbf{A}^T \cdot \mathbf{y} + \tau \mathbf{X}^{-1} \cdot \mathbf{e} &= \mathbf{c} \end{aligned} \tag{10.11.16}$$

Define the vector

$$\mathbf{z} = \tau \mathbf{X}^{-1} \cdot \mathbf{e}, \quad \text{i.e.,} \quad z_j = \tau / x_j \tag{10.11.17}$$

Then equation (10.11.16) becomes

$$\begin{aligned} \mathbf{A} \cdot \mathbf{x} &= \mathbf{b} \\ \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} &= \mathbf{c} \\ \mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} &= \tau \mathbf{e} \end{aligned} \tag{10.11.18}$$

These are exactly the equations (10.11.11) defining the central path, and they reduce to the KKT conditions (10.11.7) if we set τ to zero.

Note that equation (10.11.16) can be used to define an algorithm, the *primal interior-point method*, that doesn't depend on $\mathbf{z}$. Similarly, by starting with a logarithmic barrier function for the dual objective function, one can derive a purely dual method that doesn't involve $\mathbf{x}$. In practice, These methods are not competitive with the primal-dual methods.

Originally the logarithmic barrier function idea played an important role in motivating interior-point methods. More recently, the viewpoint has shifted to emphasize the importance of τ as defining the centering property of the algorithm rather than being simply a parameter to enforce the nonnegativity constraint.

10.11.6 A Primal-Dual Infeasible Interior-Point Algorithm

Let's pull all the pieces together now to define the algorithm. Write equation (10.11.11) for the new iterate:

$$\begin{aligned} \mathbf{A} \cdot (\mathbf{x} + \Delta\mathbf{x}) - \mathbf{b} &= 0 \\ \mathbf{A}^T \cdot (\mathbf{y} + \Delta\mathbf{y}) + \mathbf{z} + \Delta\mathbf{z} - \mathbf{c} &= 0 \\ (\mathbf{X} + \Delta\mathbf{X}) \cdot (\mathbf{Z} + \Delta\mathbf{Z}) \cdot \mathbf{e} &= \tau \mathbf{e} \end{aligned} \tag{10.11.19}$$

Drop the quadratic term $\Delta\mathbf{X} \cdot \Delta\mathbf{Z} \cdot \mathbf{e}$ and get

$$\begin{bmatrix} \mathbf{A} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{A}^T & \mathbf{1} \\ \mathbf{Z} & \mathbf{0} & \mathbf{X} \end{bmatrix} \begin{bmatrix} \Delta\mathbf{x} \\ \Delta\mathbf{y} \\ \Delta\mathbf{z} \end{bmatrix} = \begin{bmatrix} -\mathbf{r}_p \\ -\mathbf{r}_d \\ \tau\mathbf{e} - \mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} \end{bmatrix} \tag{10.11.20}$$

where the primal and dual residuals are defined by

$$\begin{aligned} \mathbf{r}_p &= \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \\ \mathbf{r}_d &= \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} - \mathbf{c} \end{aligned} \tag{10.11.21}$$

Equation (10.11.20) is simply the Newton equation (10.11.9) for (10.11.11). Note that the only nonlinearity comes from the innocuous looking quadratic term for complementary slackness. Yet it's exactly what leads to all the difficulty!

Since $\mathbf{X}$ is a diagonal positive-definite matrix, we can trivially invert it and use the last equation in (10.11.20) to eliminate $\Delta\mathbf{z}$ from the second equation. Interchanging the order of the variables $\Delta\mathbf{x}$ and $\Delta\mathbf{y}$, we get

$$\begin{bmatrix} \mathbf{0} & \mathbf{A} \\ \mathbf{A}^T & -\mathbf{X}^{-1}\cdot\mathbf{Z} \end{bmatrix} \begin{bmatrix} \Delta\mathbf{y} \\ \Delta\mathbf{x} \end{bmatrix} = \begin{bmatrix} -\mathbf{r}_p \\ \mathbf{z} - \tau\mathbf{X}^{-1}\cdot\mathbf{e} - \mathbf{r}_d \end{bmatrix} \tag{10.11.22}$$

Similarly, since $-\mathbf{X}^{-1}\cdot\mathbf{Z}$ is easily invertible, we can use the second equation in (10.11.22) to eliminate $\Delta\mathbf{x}$ from the first. This gives

$$\mathbf{A}\cdot(\mathbf{X}\cdot\mathbf{Z}^{-1})\cdot\mathbf{A}^T\cdot\Delta\mathbf{y} = -\mathbf{r}_p + \mathbf{A}\cdot(\mathbf{x} - \tau\mathbf{Z}^{-1}\cdot\mathbf{e} - \mathbf{X}\cdot\mathbf{Z}^{-1}\cdot\mathbf{r}_d) \tag{10.11.23}$$

These are called the *normal equations*, by analogy with the normal equations that occur in least-squares problems (cf. 15.4.10). The predecessor equations in (10.11.22) are called the *augmented equations*. Note that the matrix on the left-hand side of the normal equations is symmetric and positive-definite, except for some delicacy as $\mathbf{x}$ and $\mathbf{z} \to 0$. This suggests solving them with some version of the Cholesky decomposition (§2.9).

Once $\Delta\mathbf{y}$ is determined from the normal equations, the second equation in (10.11.20) gives $\Delta\mathbf{z}$:

$$\Delta\mathbf{z} = -\mathbf{A}^T\cdot\Delta\mathbf{y} - \mathbf{r}_d \tag{10.11.24}$$

Finally, the third equation in (10.11.20) gives $\Delta\mathbf{x}$:

$$\Delta\mathbf{x} = -\mathbf{X}\cdot\mathbf{Z}^{-1}\cdot\Delta\mathbf{z} + \tau\mathbf{Z}^{-1}\cdot\mathbf{e} - \mathbf{x} \tag{10.11.25}$$

In a feasible interior-point method, an initial point is somehow found in the feasible region, that is, with $\mathbf{r}_p = \mathbf{r}_d = 0$ and $(\mathbf{x}, \mathbf{z}) > 0$. Then equations (10.11.23) – (10.11.25) are solved with $\mathbf{r}_p$ and $\mathbf{r}_d$ set to zero. The consensus now is that it is not necessary to do this. It is much easier to choose a point that may be infeasible initially, and allow the iterations to converge toward a feasible point. As explained above, it is still crucial to maintain nonnegativity, however.

Equation (10.11.23) contains three contributions to the step $\Delta\mathbf{y}$ and hence to $\Delta\mathbf{x}$ and $\Delta\mathbf{z}$. First there are the terms that involve $\mathbf{r}_p$ and $\mathbf{r}_d$, which drive the solution toward feasibility. Then there is the term independent of τ. It drives the solution toward optimality. In the literature, this term is called the *affine scaling* term, because there is a geometric interpretation of its effect in terms of a linear scaling of variables. Finally there is the term proportional to τ, which is the centering term.

Here is the framework for a simple primal-dual infeasible interior-point method:

1. Choose an initial nonnegative point.
2. If the infeasibilities $\mathbf{r}_p$ and $\mathbf{r}_d$ and the complementarity gap $\mathbf{x}\cdot\mathbf{z}$ are below the desired tolerance, exit. Otherwise, continue.
3. Set the value of τ from equation (10.11.12). A value of $\delta \approx 0.02$ works well.
4. Compute the direction of the step $(\Delta\mathbf{x}, \Delta\mathbf{y}, \Delta\mathbf{z})$ from equations (10.11.23) – (10.11.25). The solution of the normal equations is done in two steps: factorization of the matrix to some easily invertible form, followed by solution using this factorization.
5. Determine the maximum stepsizes that do not allow the variables to become negative. Separate stepsizes can be determined for the primal and dual variables:

$$\begin{aligned} \mathbf{x}_{\text{new}} &= \mathbf{x}_{\text{old}} + \alpha_p \Delta\mathbf{x} \\ \mathbf{y}_{\text{new}} &= \mathbf{y}_{\text{old}} + \alpha_d \Delta\mathbf{y} \\ \mathbf{z}_{\text{new}} &= \mathbf{z}_{\text{old}} + \alpha_d \Delta\mathbf{z} \end{aligned} \tag{10.11.26}$$

where α_p and α_d are initially chosen to be the largest values that keep all components of $\mathbf{x}_{\text{new}}$ and $\mathbf{z}_{\text{new}}$ nonnegative but no larger than unity. Then reduce the values of α_p and α_d by a safety factor σ. A conservative choice is $\sigma = 0.9$, but $\sigma = 0.99995$ works for many problems.

6. Go back to step 2 for the next iteration.

Since in real-life linear programming the constraint matrix $\mathbf{A}$ is sparse, the code must take advantage of this. The various matrix products such as $\mathbf{A} \cdot \mathbf{x}$, $\mathbf{A}^T \cdot \mathbf{y}$ and $\mathbf{A} \cdot (\mathbf{X} \cdot \mathbf{Z}^{-1}) \cdot \mathbf{A}^T$ should be computed efficiently. More important, the factorization and backsubstitution involved in solving the normal equations must use a suitable sparse matrix Cholesky decomposition. The factorization step in fact dominates the running time of the algorithm. Our implementation uses the relatively simple package LDL [5], combined with the package AMD [6] to compute an ordering (permutation) of the matrix that minimizes fill-in during the factorization. Both of these packages are freely available. Note that LDL has to be modified to deal with the singularities that occur as the diagonal matrix elements $x_j/z_j \to 0$. It is sufficient to modify the line of code in LDL that tests for a diagonal element equal to zero to something like

```
if (D[k] < 1.0e-40)
    D[k] = 1.0e128;
```

This has the effect of setting the corresponding variable to zero, which is the desired behavior. Here is our interface `NRldl.h` to these packages. The full implementation is given in a Webnote [13].

interior.h

```
extern "C" {
    #include "ldl.h"
    #include "amd.h"
}

struct NRldl {
```

Interface between Numerical Recipes routine `intpt` and the required external packages LDL and AMD.

```
    Doub Info [AMD_INFO];
    Int lnz,n,nz;
    VecInt PP,PPinv,PPattern,LLnz,LLp,PParent,FFlag,*LLi;
    VecDoub YY,DD,*LLx;
    Doub *Ax, *Lx, *B, *D, *X, *Y;
    Int *Ai, *Ap, *Li, *Lp, *P, *Pinv, *Flag,*Pattern, *Lnz, *Parent;
    NRldl(NRsparseMat &adat);
```

Constructor only needs `adat` to have been declared with appropriate dimensions.

```
    void order();
```

AMD ordering and LDL symbolic factorization. Only needs nonzero pattern of `adat`, not actual values.

```
    void factorize();
```

Numerical factorization of matrix.

```
    void solve(VecDoub_O &y,VecDoub &rhs);
```

Solves for `y` given `rhs`. Can be invoked multiple times after a single call to `factorize`.

```
    ~NRldl();
};
```

Here is a simple implementation of the interior-point algorithm. Although it is a pedagogical code, it is actually quite powerful — better than the pedagogical

simplex code of the previous section. Below we will explain what would be required to turn this code into a serious implementation.

```
Doub dotprod(VecDoub_I &x, VecDoub_I &y)
```
interior.h

Compute the dot product of two vectors, $\mathbf{x} \cdot \mathbf{y}$.

```
{
    Doub sum=0.0;
    for (Int i=0;i<x.size();i++)
        sum += x[i]*y[i];
    return sum;
}
```

```
Int intpt(const NRsparseMat &a, VecDoub_I &b, VecDoub_I &c, VecDoub_O &x)
```

Interior-point method for linear programming. On input a contains the coefficient matrix for the constraints in the form $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. The right-hand side of the constraints is input in `b[0..m-1]`. The coefficients of the objective function to be minimized, $\mathbf{c} \cdot \mathbf{x}$, are input in `c[0..n-1]`. Note that `c` should generally be padded with zeros corresponding to the slack variables that extend the number of columns to be `n`. The function returns 0 if an optimal solution is found; 1 if the problem is infeasible; 2 if the dual problem is infeasible, i.e., if the problem is unbounded or perhaps infeasible; and 3 if the number of iterations is exceeded. The solution is returned in `x[0..n-1]`.

```
{
    const Int MAXITS=200;                Maximum iterations.
    const Doub EPS=1.0e-6;               Tolerance for optimality and feasibility.
    const Doub SIGMA=0.9;                Stepsize reduction factor (conservative choice).
    const Doub DELTA=0.02;               Factor to set centrality parameter μ.
    const Doub BIG=numeric_limits<Doub>::max();
    Int i,j,iter,status;
    Int m=a.nrows;
    Int n=a.ncols;
    VecDoub y(m),z(n),ax(m),aty(n),rp(m),rd(n),d(n),dx(n),dy(m),dz(n),
        rhs(m),tempm(m),tempn(n);
    NRsparseMat at=a.transpose();        Compute A^T.
    ADAT adat(a,at);                     Setup for A · D · A^T, where D = X · Z^{-1}.
    NRldl solver(adat.ref());            Initialize interface to LDL package.
    solver.order();                      AMD ordering and LDL symbolic factorization.
    Doub rpfact=1.0+sqrt(dotprod(b,b));      Compute factors for convergence test.
    Doub rdfact=1.0+sqrt(dotprod(c,c));
    for (j=0;j<n;j++) {                  Initial point.
        x[j]=1000.0;
        z[j]=1000.0;
    }
    for (i=0;i<m;i++) {
        y[i]=1000.0;
    }
    Doub normrp_old=BIG;
    Doub normrd_old=BIG;
    cout << setw(4) << "iter" << setw(12) << "Primal obj." << setw(9) <<
        "||r_p||" << setw(13) << "Dual obj." << setw(11) << "||r_d||" <<
        setw(13) << "duality gap" << setw(16) << "normalized gap" << endl;
    cout << scientific << setprecision(4);
    for (iter=0;iter<MAXITS;iter++) {    Start of main loop.
        ax=a.ax(x);                      Compute normalized residuals r_p and r_d.
        for (i=0;i<m;i++)
            rp[i]=ax[i]-b[i];
        Doub normrp=sqrt(dotprod(rp,rp))/rpfact;
        aty=at.ax(y);
        for (j=0;j<n;j++)
            rd[j]=aty[j]+z[j]-c[j];
        Doub normrd=sqrt(dotprod(rd,rd))/rdfact;
        Doub gamma=dotprod(x,z);         Duality gap is x · z for feasible points.
        Doub mu=DELTA*gamma/n;           Choice of μ.
        Doub primal_obj=dotprod(c,x);    Print current iteration.
```

```
        Doub dual_obj=dotprod(b,y);
        Doub gamma_norm=gamma/(1.0+abs(primal_obj));
         cout << setw(3) << iter << setw(12) << primal_obj << setw(12) <<
            normrp << setw(12) << dual_obj << setw(12) << normrd << setw(12)
            << gamma << setw(12) << gamma_norm<<endl;
        if (normrp < EPS && normrd < EPS && gamma_norm < EPS)
            return status=0;            Optimal solution found.
        if (normrp > 1000*normrp_old && normrp > EPS)
            return status=1;            Primal infeasible.
        if (normrd > 1000*normrd_old && normrd > EPS)
            return status=2;            Dual infeasible.
        for (j=0;j<n;j++)               Compute step directions. First form matrix
            d[j]=x[j]/z[j];                 A·X·Z^{-1}·A^T.
        adat.updateD(d);
        solver.factorize();             Factorize matrix.
        for (j=0;j<n;j++)               Form right-hand side.
            tempn[j]=x[j]-mu/z[j]-d[j]*rd[j];
        tempm=a.ax(tempn);
        for (i=0;i<m;i++)
            rhs[i]=-rp[i]+tempm[i];
        solver.solve(dy,rhs);           Solve for dy.
        tempn=at.ax(dy);                Solve for dz.
        for (j=0;j<n;j++)
            dz[j]=-tempn[j]-rd[j];
        for (j=0;j<n;j++)               Solve for dx.
            dx[j]=-d[j]*dz[j]+mu/z[j]-x[j];
        Doub alpha_p=1.0;               Find step length.
        for (j=0;j<n;j++)
            if (x[j]+alpha_p*dx[j] < 0.0)
                alpha_p=-x[j]/dx[j];
        Doub alpha_d=1.0;
        for (j=0;j<n;j++)
            if (z[j]+alpha_d*dz[j] < 0.0)
                alpha_d=-z[j]/dz[j];
        alpha_p = MIN(alpha_p*SIGMA,1.0);
        alpha_d = MIN(alpha_d*SIGMA,1.0);
        for (j=0;j<n;j++) {             Step to new point.
            x[j]+=alpha_p*dx[j];
            z[j]+=alpha_d*dz[j];
        }
        for (i=0;i<m;i++)
            y[i]+=alpha_d*dy[i];
        normrp_old=normrp;              Update norms.
        normrd_old=normrd;
    }
    return status=3;                    Maximum iterations exceeded.
}
```

10.11.7 Practical Interior-Point Codes

There are a number of important features that would be needed to turn the above simple implementation into a state-of-the-art code.

- Initial point. Choosing a good starting point cuts down the number of iterations required. A good algorithm is described in [7].
- Preprocessing. As for the simplex method, preprocessing can often reduce the size of the problem.
- Scaling. A badly scaled problem can lead to numerical difficulties.
- Handling bounded variables. Suppose that instead of the requirement $\mathbf{x} \geq 0$ the variables are *bounded*:

$$\mathbf{l} \le \mathbf{x} \le \mathbf{u} \tag{10.11.27}$$

Here, for simplicity, we have written the vectors of lower and upper bounds, $\mathbf{l}$ and $\mathbf{u}$, as being of length n. In practice, only some of the variables $\mathbf{x}$ may have bounds. One way to deal with bounds is to add them to the system $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ with slack variables in the usual way. However, this increases the dimension of the matrix $\mathbf{A}$. There is a simpler way to proceed. First, lower bounds of the form $l_j \le x_j$ can be handled by a simple shift: $x'_j = x_j - l_j \ge 0$. Making this replacement everywhere allows the problem to be solved as before, and then you simply undo the shift to get the solution in terms of the original x_j. So without loss of generality we can assume all the bounds are of the form

$$0 \le \mathbf{x} \le \mathbf{u} \tag{10.11.28}$$

If we introduce slack variables $\mathbf{s}$ and dual slack variables $\mathbf{w}$, the optimality conditions are

$$\begin{aligned} \mathbf{A} \cdot \mathbf{x} &= \mathbf{b} \\ \mathbf{x} + \mathbf{s} &= \mathbf{u} \\ \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} - \mathbf{w} &= \mathbf{c} \\ \mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} &= 0 \\ \mathbf{S} \cdot \mathbf{W} \cdot \mathbf{e} &= 0 \end{aligned} \tag{10.11.29}$$

with $\mathbf{x}$, $\mathbf{s}$, $\mathbf{z}$, and $\mathbf{w}$ all nonnegative. It is simple to change the right-hand sides of the last two equations in (10.11.29) to $\tau\mathbf{e}$ and apply Newton's method as for equation (10.11.11). You find that the equations to be solved are very similar in form to equations (10.11.23) – (10.11.25).

- Predictor-corrector. Most of the time spent in an iteration goes into the factorization of the matrix in the normal equations. Given the factorization, the solve step is relatively cheap. Mehotra's predictor-corrector method [7] takes advantage of this by using an extra solve step per iteration to improve the overall efficiency of the algorithm.

  Recall that in going from equation (10.11.19) to equation (10.11.20) we dropped the term $\Delta\mathbf{X} \cdot \Delta\mathbf{Z} \cdot \mathbf{e}$. The idea in Mehotra's method is to first take a predictor step that solves equation (10.11.20), but with the τ term omitted. The values of $(\Delta\mathbf{x}, \Delta\mathbf{y}, \Delta\mathbf{z})$ obtained are used to estimate $\Delta\mathbf{X} \cdot \Delta\mathbf{Z} \cdot \mathbf{e}$. Then the corrector step solves for an additional set $(\Delta\mathbf{x}, \Delta\mathbf{y}, \Delta\mathbf{z})$ from equation (10.11.20) with the right-hand side replaced by

$$\begin{bmatrix} 0 \\ 0 \\ \tau\mathbf{e} - \Delta\mathbf{X} \cdot \Delta\mathbf{Z} \cdot \mathbf{e} \end{bmatrix} \tag{10.11.30}$$

  The value of τ in equation (10.11.30) is set differently from equation (10.11.12). First $\widehat{\mu}$ is computed using the predictor step:

$$\widehat{\mu} = (\mathbf{x} + \alpha_p \Delta\mathbf{x}) \cdot (\mathbf{z} + \alpha_d \Delta\mathbf{z})/n \tag{10.11.31}$$

  Here α_p and α_d are the largest values that maintain nonnegativity, but no larger than unity. (No safety factor is used.) Then τ is set as

$$\tau = \left(\frac{\widehat{\mu}}{\mu}\right)^2 \widehat{\mu} \tag{10.11.32}$$

where μ is computed using the starting values of $\mathbf{x}$ and $\mathbf{z}$ as in equation (10.11.12). This heuristic choice makes τ small when the predictor step gives a large decrease in complementarity, and large otherwise.

The total step is the sum of the predictor and corrector steps. The cutdown factor from α_p or α_d equal to unity is calculated from a heuristic procedure described in [7] or [8].

Gondzio [9,10] has developed an extension to the predictor-corrector algorithm that incorporates higher-order corrections when they can improve the efficiency.

- Better sparse matrix algebra. While AMD is a good general-purpose choice for an ordering algorithm, LDL is a good but basic sparse Cholesky routine, chosen mainly for its simplicity and availability. More powerful algorithms are known and are starting to become publicly available.

  One of the problems with the normal equations is that the matrix can be quite dense, even when $\mathbf{A}$ itself is rather sparse. This has motivated algorithms that solve the augmented equations (10.11.22) directly. On some problems, this leads to significant savings. A good implementation will provide both alternatives.

  Solving the equations can become numerically delicate, especially as the optimal point is approached. Good implementations will use some form of iterative refinement to preserve accuracy.
- Crossover to the simplex method. Often the convergence of the interior-point algorithm slows down near the optimal point. By switching to a simplex method with a basis that is presumably close to optimal, one can get rapid convergence to the answer. This feature has the additional benefit that the optimal point is given in terms of basis vectors, whereas the interior-point solution never actually attains zeros for any $\mathbf{x}$'s. Some kinds of post-analysis need the actual basis.

  Interestingly, using a previously-found solution as the initial point for a "nearby" problem seldom helps much for interior-point methods. The reason is that interior-point methods don't make good progress from a point near the boundary. The simplex method, by contrast, generally converges much more rapidly with a "warm start." A good strategy for solving a sequence of closely related problems is therefore interior-point with crossover to an optimal basis for the first one, then simplex with a warm start for the remainder.

There are several codes that are free for noncommercial use and that give full implementations of interior-point methods. We particularly like PCx (in C with Fortran sparse algebra routines) [11] and HOPDM (in Fortran) [10]. For a discussion of more options, including commercial codes, see [12].

So which should you use: a simplex or an interior-point code? If you have only our codes, use the interior-point one. If you have a production implementation of either algorithm, it will probably suffice. If you are solving many large problems, however, you should have both so you can use the optimum algorithm in each case. If you are solving a large problem for the first time, there is a lot to be said for using an interior-point code. There are fewer choices to make to get almost optimal performance. By contrast, finding which particular choices of components in a simplex method give optimal performance can involve a lot of experimentation. Your first try will usually not be as good as a default interior-point code. And often the

interior-point code will beat all simplex variants.

CITED REFERENCES AND FURTHER READING:

Khachian, L. 1979, *Doklady Academiia Nauk SSSR*, vol. 244, pp. 191–194. English translation: "A Polynomial Time Algorithm in Linear Programming," *Soviet Mathematics Doklady*, vol. 20, pp. 191–194.[1]

Karmarkar, N. 1984, "A New Polynomial-time Algorithm for Linear Programming," *Combinatorica* vol. 4, pp. 373–395.[2]

Maros, I. 2003, *Computational Techniques of the Simplex Method* (Boston: Kluwer).[3]

Nazareth, J.L. 2004, *An Optimization Primer* (New York: Springer).[4]

LDL, `http://www.cise.ufl.edu/research/sparse`.[5]

AMD, `http://www.cise.ufl.edu/research/sparse`. See also Amestoy, P.R., Enseeiht-Irit, Davis, T.A., and Duff, I.S. 2004, "AMD, an Approximate Minimum Degree Ordering Algorithm." *ACM Transactions on Mathematical Software* vol. 30, pp. 381–388.[6]

Mehrotra, S. 1992, "On the Implementation of a Primal-dual Interior Point Method," *SIAM Journal on Optimization* vol. 2, pp. 575–601.[7]

Wright, S.J. 1997, *Primal-Dual Interior-Point Methods* (Philadelphia: S.I.A.M.).[8]

Gondzio, J. 1996, "Multiple Centrality Corrections in a Primal-dual Method for Linear Programming," *Computational Optimization and Applications* vol. 6, pp. 137–156.[9]

HOPDM, `http://www.maths.ed.ac.uk/~gondzio/software/hopdm.html`.[10]

PCX, `http://www-fp.mcs.anl.gov/otc/Tools/PCx`.[11]

Linear Programming Frequently Asked Questions, `http://www-unix.mcs.anl.gov/otc/Guide/faq/linear-programming-faq.html`.[12]

Vanderbei, R.J. 2001, *Linear Programming: Foundations and Extensions*, 2nd ed. (Boston: Kluwer). [Undergraduate/graduate text]

Numerical Recipes Software 2007, "Interface to AMD and LDL Packages," *Numerical Recipes Webnote No. 13*, at `http://www.nr.com/webnotes?13` [13]

10.12 Simulated Annealing Methods

The *method of simulated annealing* [1,2] is a technique that has attracted significant attention as suitable for optimization problems of large scale, especially ones where a desired global extremum is hidden among many poorer, local extrema. For practical purposes, simulated annealing has effectively "solved" the famous *traveling salesman problem* of finding the shortest cyclical itinerary for a traveling salesman who must visit each of N cities in turn. (Other practical methods have also been found.) The method has also been used successfully for designing complex integrated circuits: The arrangement of several hundred thousand circuit elements on a tiny silicon substrate is optimized so as to minimize interference among their connecting wires [3,4]. Surprisingly, the implementation of the algorithm is relatively simple.

Notice that the two applications cited are both examples of *combinatorial minimization*. There is an objective function to be minimized, as usual, but the space over which that function is defined is not simply the N-dimensional space of N continuously variable parameters. Rather, it is a discrete, but very large, configuration space, like the set of possible orders of cities, or the set of possible allocations of silicon

"real estate" blocks to circuit elements. The number of elements in the configuration space is factorially large, so that they cannot be explored exhaustively. Furthermore, since the set is discrete, we are deprived of any notion of "continuing downhill in a favorable direction." The concept of "direction" may not have any meaning in the configuration space.

Below, we will also discuss how to use simulated annealing methods for spaces with continuous control parameters, like those of §10.5 – §10.9. This application is actually more complicated than the combinatorial one, since the familiar problem of "long, narrow valleys" again asserts itself. Simulated annealing, as we will see, tries "random" steps; but in a long, narrow valley, almost all random steps are uphill! Some additional finesse is therefore required.

At the heart of the method of simulated annealing is an analogy with thermodynamics, specifically with the way that liquids freeze and crystallize or metals cool and anneal. At high temperatures, the molecules of a liquid move freely with respect to one another. If the liquid is cooled slowly, thermal mobility is lost. The atoms are often able to line themselves up and form a pure crystal that is completely ordered over a distance up to billions of times the size of an individual atom in all directions. This crystal is the state of minimum energy for this system. The amazing fact is that, for slowly cooled systems, nature is able to find this minimum energy state. In fact, if a liquid metal is cooled quickly or "quenched," it does not reach this state but rather ends up in a polycrystalline or amorphous state having somewhat higher energy.

So the essence of the process is *slow* cooling, allowing ample time for the redistribution of the atoms as they lose mobility. This is the technical definition of *annealing*, and it is essential for ensuring that a low energy state will be achieved.

Although the analogy is not perfect, there is a sense in which all of the minimization algorithms thus far in this chapter correspond to rapid cooling or quenching. In all cases, we have gone greedily for the quick, nearby solution: From the starting point, go immediately downhill as far as you can go. This, as often remarked above, leads to a local, but not necessarily a global, minimum. Nature's own minimization algorithm is based on quite a different procedure. The so-called Boltzmann probability distribution,

$$\text{Prob}\,(E) \sim \exp(-E/kT) \tag{10.12.1}$$

expresses the idea that a system in thermal equilibrium at temperature T has its energy probabilistically distributed among all different energy states E. Even at low temperature, there is a chance, albeit a very small one, of a system being in a high energy state. Therefore, there is a corresponding chance for the system to get out of a local energy minimum in favor of finding a better, more global one. The quantity k (Boltzmann's constant) is a constant of nature that relates temperature to energy. In other words, the system sometimes goes *uphill* as well as downhill; but the lower the temperature, the less likely is any significant uphill excursion.

In 1953, Metropolis and coworkers [5] first incorporated these kinds of principles into numerical calculations. Offered a succession of options, a simulated thermodynamic system was assumed to change its configuration from energy E_1 to energy E_2 with probability $p = \exp[-(E_2 - E_1)/kT]$. Notice that if $E_2 < E_1$, this probability is greater than unity; in such cases the change is arbitrarily assigned a probability $p = 1$, i.e., the system *always* took such an option. This general scheme, of always taking a downhill step while *sometimes* taking an uphill step, has come to be known as the Metropolis algorithm.

To make use of the Metropolis algorithm for other than thermodynamic systems, one must provide the following elements:

1. A description of possible system configurations.
2. A generator of random changes in the configuration; these changes are the "options" presented to the system.
3. An objective function E (analog of energy) whose minimization is the goal of the procedure.
4. A control parameter T (analog of temperature) and an *annealing schedule* that tells how it is lowered from high to low values, e.g., after how many random changes in configuration is each downward step in T taken, and how large is that step. The meaning of "high" and "low" in this context, and the assignment of a schedule, may require physical insight and/or trial-and-error experiments.

We will return to these ideas in §15.8, with a more rigorous discussion of *Markov chain Monte Carlo* and the *Metropolis-Hastings algorithm*.

10.12.1 Combinatorial Minimization: The Traveling Salesman

A concrete illustration is provided by the traveling salesman problem. The proverbial seller visits N cities with given positions (x_i, y_i), returning finally to his or her city of origin. Each city is to be visited only once, and the route is to be made as short as possible. This problem belongs to a class known as *NP-complete* problems, whose computation time for an *exact* solution increases with N as exp(const. $\times N$), becoming rapidly prohibitive in cost as N increases. The traveling salesman problem also belongs to a class of minimization problems for which the objective function E has many local minima. In practical cases, it is often enough to be able to choose from these a minimum that, even if not absolute, cannot be significantly improved upon. The annealing method manages to achieve this, while limiting its calculations to scale as a small power of N.

As a problem in simulated annealing, the traveling salesman problem is handled as follows:

1. *Configuration.* The cities are numbered $i = 0 \ldots N-1$ and each has coordinates (x_i, y_i). A configuration is a permutation of the number $0 \ldots N-1$, interpreted as the order in which the cities are visited.
2. *Rearrangements.* An efficient set of moves has been suggested by Lin [6]. The moves consist of two types: (i) A section of path is removed and then replaced with the same cities running in the opposite order; or (ii) a section of path is removed and then replaced in between two cities on another, randomly chosen, part of the path.
3. *Objective function.* In the simplest form of the problem, E is taken just as the total length of the journey,

$$E = L \equiv \sum_{i=0}^{N-1} \sqrt{(x_i - x_{i+1})^2 + (y_i - y_{i+1})^2} \tag{10.12.2}$$

with the convention that point N is identified with point 0. To illustrate the flexibility of the method, however, we can add the following additional wrinkle: Suppose that the salesman has an irrational fear of flying over the Missis-

sippi River. In that case, we would assign each city a parameter μ_i, equal to $+1$ if it is east of the Mississippi and -1 if it is west, and take the objective function to be

$$E = \sum_{i=0}^{N-1} \left[\sqrt{(x_i - x_{i+1})^2 + (y_i - y_{i+1})^2} + \lambda(\mu_i - \mu_{i+1})^2 \right] \quad (10.12.3)$$

A penalty 4λ is thereby assigned to any river crossing. The algorithm now finds the shortest path that avoids crossings. The relative importance that it assigns to length of path versus river crossings is determined by our choice of λ. Figure 10.12.1 shows the results obtained. Clearly, this technique can be generalized to include many conflicting goals in the minimization.

4. *Annealing schedule.* This requires experimentation. We first generate some random rearrangements, and use them to determine the range of values of ΔE that will be encountered from move to move. Choosing a starting value for the parameter T that is considerably larger than the largest ΔE normally encountered, we proceed downward in multiplicative steps each amounting to a 10% decrease in T. We hold each new value of T constant for, say, $100N$ reconfigurations, or for $10N$ successful reconfigurations, whichever comes first. When efforts to reduce E further become sufficiently discouraging, we stop.

In a Webnote [7], we give a complete implementation of the above ideas for the traveling salesman problem, using the Metropolis algorithm.

10.12.2 Continuous Minimization by Simulated Annealing

The basic ideas of simulated annealing are also applicable to optimization problems with continuous N-dimensional control spaces, e.g., finding the (ideally, global) minimum of some function $f(\mathbf{x})$, in the presence of many local minima, where $\mathbf{x}$ is an N-dimensional vector. The four elements required by the Metropolis procedure are now as follows: The value of f is the objective function. The system state is the point $\mathbf{x}$. The control parameter T is, as before, something like a temperature, with an annealing schedule by which it is gradually reduced. And there must be a generator of random changes in the configuration, that is, a procedure for taking a random step from $\mathbf{x}$ to $\mathbf{x} + \Delta\mathbf{x}$.

The last of these elements is the most problematical. The literature to date [8-12] describes several different schemes for choosing $\Delta\mathbf{x}$, none of which, in our view, inspires complete confidence. The problem is one of efficiency: A generator of random changes is inefficient if, *when local downhill moves exist*, it nevertheless almost always proposes an uphill move. A good generator, we think, should not become inefficient in narrow valleys, nor should it become more and more inefficient as convergence to a minimum is approached. Except possibly for [8], all of the schemes that we have seen are inefficient in one or both of these situations.

Our own way of doing simulated annealing minimization on continuous control spaces is to use a modification of the downhill simplex method (§10.5). Complete code for this is given in a Webnote [9]. The technique amounts to replacing the single point $\mathbf{x}$ as a description of the system state by a simplex of $N + 1$ points. The "moves" are the same as described in §10.5, namely reflections, expansions, and contractions of the simplex. The implementation of the Metropolis procedure is slightly subtle: We *add* a positive, logarithmically distributed random variable, proportional

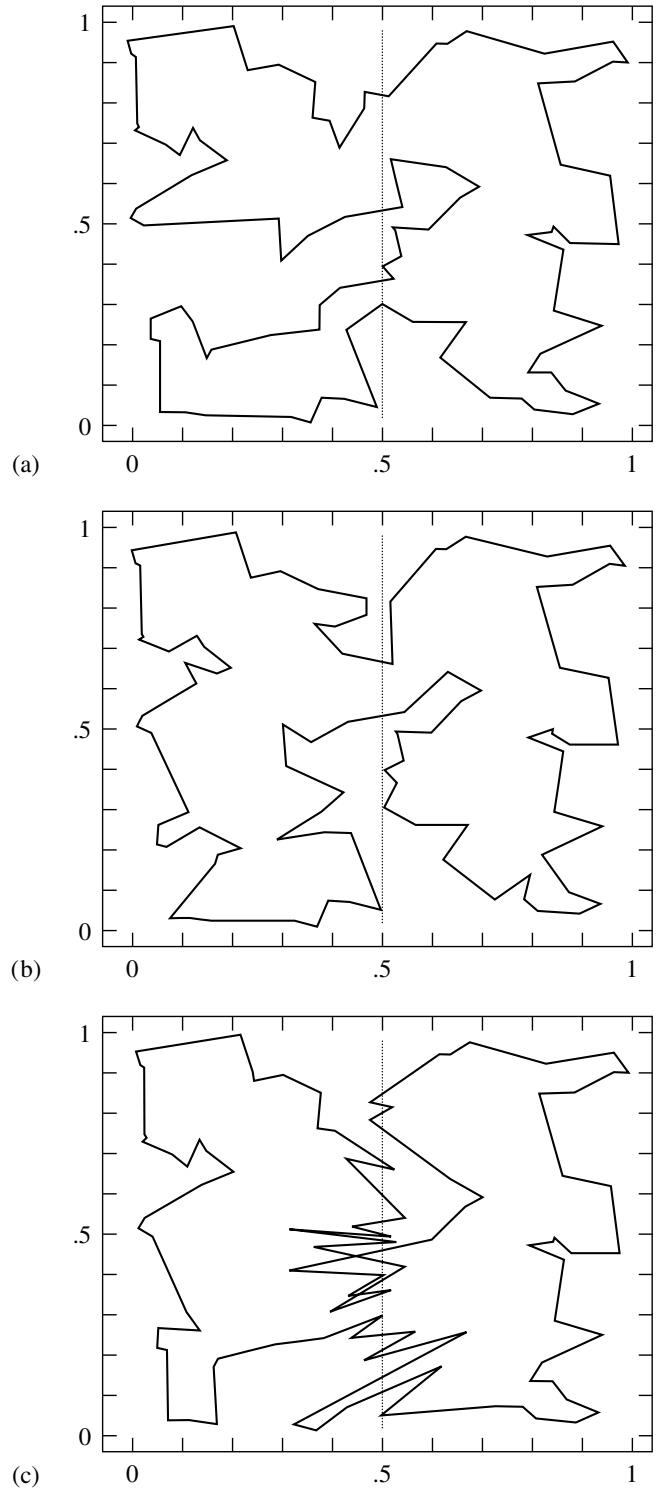


Figure 10.12.1. Traveling salesman problem solved by simulated annealing. The (nearly) shortest path among 100 randomly positioned cities is shown in (a). The dotted line is a river, but there is no penalty in crossing. In (b) the river-crossing penalty is made large, and the solution restricts itself to the minimum number of crossings, two. In (c) the penalty has been made negative: The salesman is actually a smuggler who crosses the river on the flimsiest excuse!

to the temperature T, to the stored function value associated with every vertex of the simplex, and we *subtract* a similar random variable from the function value of every new point that is tried as a replacement point. Like the ordinary Metropolis procedure, this method always accepts a true downhill step, but sometimes accepts an uphill one. In the limit $T \to 0$, this algorithm reduces exactly to the downhill simplex method and converges to a local minimum.

At a finite value of T, the simplex expands to a scale that approximates the size of the region that can be reached at this temperature, and then executes a stochastic, tumbling Brownian motion within that region, sampling new, approximately random, points as it does so. The efficiency with which a region is explored is independent of its narrowness (for an ellipsoidal valley, the ratio of its principal axes) and orientation. If the temperature is reduced sufficiently slowly, it becomes highly likely that the simplex will shrink into that region containing the lowest relative minimum encountered.

As in all applications of simulated annealing, there can be quite a lot of problem-dependent subtlety in the phrase "sufficiently slowly"; success or failure is quite often determined by the choice of annealing schedule. Here are some possibilities worth trying:

- Reduce T to $(1 - \epsilon)T$ after every m moves, where ϵ/m is determined by experiment.
- Budget a total of K moves, and reduce T after every m moves to a value $T = T_0(1 - k/K)^\alpha$, where k is the cumulative number of moves thus far, and α is a constant, say 1, 2, or 4. The optimal value for α depends on the statistical distribution of relative minima of various depths. Larger values of α spend more iterations at lower temperature.
- After every m moves, set T to β times $f_1 - f_b$, where β is an experimentally determined constant of order 1, f_1 is the smallest function value currently represented in the simplex, and f_b is the best function ever encountered. However, never reduce T by more than some fraction γ at a time.

Another strategic question is whether to do an occasional *restart*, where a vertex of the simplex is discarded in favor of the "best-ever" point. (You must be sure that the best-ever point is not one of the other vertices of the simplex when you do this!) We have found problems for which restarts — every time the temperature has decreased by a factor of 3, say — are highly beneficial; we have found other problems for which restarts have no positive, or a somewhat negative, effect.

There is not yet enough practical experience with the method of simulated annealing to say definitively what its place among optimization methods is. The method has several extremely attractive features that are rather unique when compared with other optimization techniques.

First, it is not "greedy," in the sense that it is not easily fooled by the quick payoff achieved by falling into unfavorable local minima. Provided that sufficiently general reconfigurations are given, it wanders freely among local minima of depth less than about T. As T is lowered, the number of such minima qualifying for frequent visits is gradually reduced.

Second, configuration decisions tend to proceed in a logical order. Changes that cause the greatest energy differences are sifted over when the control parameter T is large. These decisions become more permanent as T is lowered, and attention

then shifts more to smaller refinements in the solution. For example, in the traveling salesman problem with the Mississippi River twist, if λ is large, a decision to cross the Mississippi only twice is made at high T, while the specific routes on each side of the river are determined only at later stages.

The analogies to thermodynamics may be pursued to a greater extent than we have done here. Quantities analogous to specific heat and entropy may be defined, and these can be useful in monitoring the progress of the algorithm toward an acceptable solution [1].

CITED REFERENCES AND FURTHER READING:

Salamon, P., Sibani, P., and Frost, R. 2002, *Facts, Conjectures, and Improvements for Simulated Annealing* (New York: SIAM Press).

van Laarhoven, P.J.M., and Aarts, E.H.L. 1987, *Simulated Annealing: Theory and Applications* (Berlin: Springer).

Kirkpatrick, S., Gelatt, C.D., and Vecchi, M.P. 1983, "Optimization by Simulated Annealing," *Science*, vol. 220, pp. 671–680.[1]

Kirkpatrick, S. 1984, "Optimization by Simulated Annealing: Quantitative Studies," *Journal of Statistical Physics*, vol. 34, pp. 975–986.[2]

Vecchi, M.P. and Kirkpatrick, S. 1983, "Global Wiring by Simulated Annealing," *IEEE Transactions on Computer Aided Design*, vol. CAD-2, pp. 215–222.[3]

Otten, R.H.J.M., and van Ginneken, L.P.P.P. 1989, *The Annealing Algorithm* (Boston: Kluwer) [contains many references to the literature].[4]

Metropolis, N., Rosenbluth, A., Rosenbluth, M., Teller A., and Teller, E. 1953, "Equations of State Calculations by Fast Computing Machines," *Journal of Chemical Physics*, vol. 21, pp. 1087–1092.[5]

Lin, S. 1965, "Computer Solutions of the Traveling Salesman Problem," *Bell System Technical Journal*, vol. 44, pp. 2245–2269.[6]

Numerical Recipes Software 2007, "Code Implementation for the Traveling Salesman Problem," *Numerical Recipes Webnote No. 14*, at `http://www.nr.com/webnotes?14` [7]

Vanderbilt, D., and Louie, S.G. 1984, "A Monte Carlo Simulated Annealing Approach to Optimization over Continuous Variables," *Journal of Computational Physics*, vol. 56, pp. 259–271.[8]

Numerical Recipes Software 2007, "Code for Minimization with Simulated Annealing," *Numerical Recipes Webnote No. 15*, at `http://www.nr.com/webnotes?15` [9]

Bohachevsky, I.O., Johnson, M.E., and Stein, M.L. 1986, "Generalized Simulated Annealing for Function Optimization," *Technometrics*, vol. 28, pp. 209–217.[10]

Corana, A., Marchesi, M., Martini, C., and Ridella, S. 1987, "Minimizing Multimodal Functions of Continuous Variables with the Simulated Annealing Algorithm," *ACM Transactions on Mathematical Software*, vol. 13, pp. 262–280.[11]

Bélisle, C.J.P., Romeijn, H.E., and Smith, R.L. 1990, "Hide and Seek: A Simulated Annealing Algorithm for Global Optimization," Technical Report 90–25, Department of Industrial and Operations Engineering, University of Michigan.[12]

Christofides, N., Mingozzi, A., Toth, P., and Sandi, C. (eds.) 1979, *Combinatorial Optimization* (London and New York: Wiley-Interscience) [not simulated annealing, but other topics and algorithms].

10.13 Dynamic Programming

Dynamic programming, or *DP*, is an optimization technique that applies when a known sequence of choices, each with a cost or benefit, is to be made and one wants

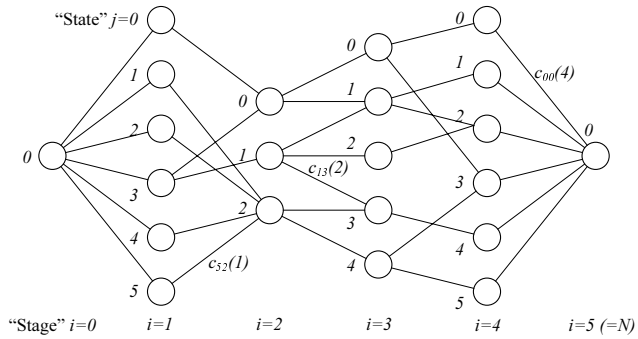


Figure 10.13.1. Canonical dynamical programming problem. It is desired to find the lowest cost path from a starting state to an ending state through $N-1$ intermediate *stages*. Each stage is characterized by a set of *states* (not necessarily the same at each stage). An allowed edge between state j in stage i and state k in stage $i+1$ has a cost denoted $c_{jk}(i)$ (not all labeled in the figure).

to minimize the total cost, or maximize the total benefit, after the sequence has been traversed. More specifically, a problem that is amenable to dynamic programming can be broken up into an ordered series of discrete *stages*, and within each stage a set of discrete *states*. These stages and states form a directed graph (see Figure 10.13.1) that we want to traverse from a given starting state ($i = 0$) to a given ending state ($i = N$). Allowed decisions that take one from state j in stage i to state k in stage $i+1$ are edges in the graph. Their cost is denoted $c_{jk}(i)$. Without any loss of generality, one can connect all the states in stage i to all the states in stage $i+1$, but with $c_{jk}(i) = \infty$ for forbidden paths.

Computer science is rich in graph-theoretic problems and algorithms, but only a few of these are within the scope of this book. Dynamic programming is one of these because its basic idea is very simple and its applications are very broad. It is important that you be able to recognize a problem amenable to DP when you see one. In particular, we will use several of the concepts in this section later, in §16.2, when we discuss the estimation of states from probabilistic data, including probabilistic decoding algorithms.

The key idea of dynamic programming is called the Bellman, Dijkstra, or Viterbi algorithm, depending on the field of training of the caller. As shown in Figure 10.13.2, the idea is that one can do a single sweep of a stage-ordered graph from left to right, labeling each vertex by the single number that is the cost of the *best* way of having reached it. (Henceforth we'll take the canonical DP problem as a cost-minimization problem; if your problem is instead a benefit-maximization problem, just use the negative of your benefits as costs.)

When the end state is reached, the global minimum cost of getting to it becomes known. Now, in a single backward pass, we can read off exactly what set of decisions led to this global minimum, by reconstructing which predecessor state was the one actually in the chain that led to the best result. Arriving back at the starting state, our solution is complete.

The art of DP involves, in many cases, the clever organization of the problem to minimize the number of states at each stage, so as to avoid the "curse of dimensionality" (a phrase first used by Bellman in exactly this context). Sometimes the order of the stages is not chronological at all, but merely reflects the decomposition of a

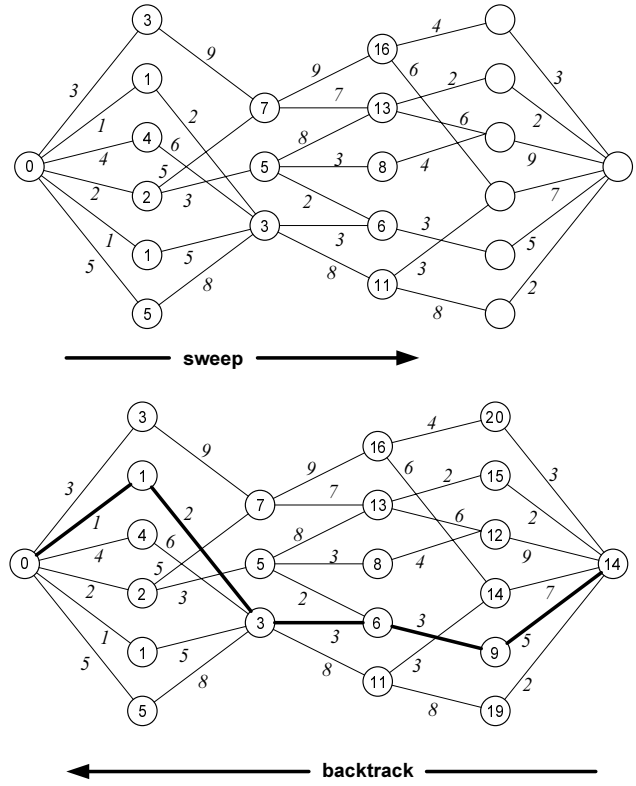


Figure 10.13.2. Two snapshots during the solution of a DP problem by the Bellman-Dijkstra-Viterbi algorithm. Edge costs are given as shown. Top: During the rightward sweep (here not yet complete) each state is labeled by the minimum cost to reach it, as determined solely by the labels of the preceding stage and the connecting edge costs. Bottom: After the rightward sweep is complete, the unique set of edges that produce the global minimum is found by one pass of backtracking.

problem into a convenient form for DP.

Here is a function embodying the Bellman-Dijkstra-Viterbi algorithm. You should consider this function more a precise statement of the algorithm than a production DP code. For example, it simply pushes off to the user function `cost()` the important issue of how to retrieve efficiently the (usually) sparse set of allowed edges that have finite costs. (You might want to consider a hash memory. See §7.6.) Also, this routine loops explicitly over all combinations of states j and k, the origin and destination states in going from stage i to $i + 1$. If you have a problem big enough to need some kind of sparse lookup, then you'll want to change these explicit loops accordingly.

```
VecInt dynpro(const VecInt &nstate,                                    dynpro.h
    Doub cost(Int jj, Int kk, Int ii)) {
```

Given the vector `nstate` whose integer values are the number of states in each stage (1 for the first and last stages), and given a function `cost(j,k,i)` that returns the cost of moving between state `j` of stage `i` and state `k` of stage `i+1`, this routine returns a vector of the same length as `nstate` containing the state numbers of the lowest cost path. States number from 0, and the first and last components of the returned vector will thus always be 0.

```
    const Doub BIG = 1.e99;
```

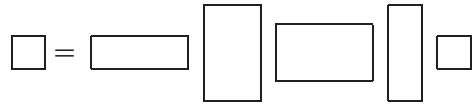
```
    static const Doub EPS=numeric_limits<Doub>::epsilon();
    Int i, j ,k, nstage = nstate.size() - 1;
    Doub a,b;
    VecInt answer(nstage+1);
    if (nstate[0] != 1 || nstate[nstage] != 1)
        throw("One state allowed in first and last stages.");
    Doub **best = new Doub*[nstage+1];          Allocate array-of-arrays for storing scores.
    best[0] = new Doub[nstate[0]];
    best[0][0] = 0.;
    for (i=1; i<=nstage; i++) {                 Forward sweep through stages.
        best[i] = new Doub[nstate[i]];
        for (k=0; k<nstate[i]; k++) {
            b = BIG;
            for (j=0; j<nstate[i-1]; j++) {     Find predecessor giving min cost.
                if ((a = best[i-1][j] + cost(j,k,i-1)) < b) b = a;
            }
            best[i][k] = b;
        }
    }
    answer[nstage] = answer[0] = 0;
    for (i=nstage-1; i>0; i--) {                Backtracking pass.
        k = answer[i+1];
        b = best[i+1][k];
        for (j=0; j<nstate[i]; j++) {           Find a predecessor that gave min.
            Doub temp = best[i][j] + cost(j,k,i);
            if (fabs(b - temp) <= EPS*fabs(temp)) break;
        }
        answer[i] = j;
    }
    for (i=nstage; i>=0; i--) delete [] best[i];    Cleanup storage.
    delete [] best;
    return answer;
}
```

10.13.1 Example: Order of Matrix Multiplication

Suppose we have five matrices to multiply, so as to get a result $\mathbf{T}$,

$$\mathbf{T} = \mathbf{ABCDE} \tag{10.13.1}$$

The matrices may all have different shapes, as long as the number of columns of a matrix is the same as the number of rows of the matrix immediately to its right. Matrix multiplication is associative, and we can do the multiplies in any order we want; but the total number of scalar multiplications can be quite different, depending on which order is chosen. You should be able to see this in the following figure:



What we want to minimize is the total number of scalar multiplications. In this example, a good choice of "stage" is just how many matrix multiplications have been performed. So the stages and states might look something like this:

Stage 0	Stage 1	Stage 2	Stage 3	Stage 4
ABCDE	**(AB)CDE** **A(BC)DE** **AB(CD)E** **ABC(DE)**	**(ABC)DE** **(AB)(CD)E** **(AB)C(DE)** **A(BCD)E** **A(BC)(DE)** **AB(CDE)**	**(ABCD)E** **(ABC)(DE)** **(AB)(CDE)** **A(BCDE)**	**(ABCDE)**

Here parentheses group matrix factors that are already fully multiplied (i.e., they are, by now, a single matrix). We will leave it to you to connect the states by allowed edges, and to calculate the cost of each edge in terms of the dimensions of the various matrices.

So how could we have done this example *wrong*? We might have identified states with all possible ways of parenthesizing **ABCDE**, including, for example, **A(B(CD)E)**. That is unnecessary, because only the outermost parentheses matter: A matrix state doesn't care about the exact path taken to reach it, as long as its factors were multiplied in some associative order. The power of DP is realized when, at every stage, many histories collapse to a (relatively) small number of states, which can then be taken as ignorant of their past history.

10.13.2 Example: DNA Sequence Alignment

DNA sequences of different organisms, at one time identical in a common ancestor, can diverge over time by the deletion, insertion, or substitution of bases in one or the other organism's sequence. It is desired to find the best match between two given sequences. In finding the best match, we are allowed to insert gaps in either sequence; but in the end we will be assessed a penalty for gap positions, a penalty for mismatches, and a reward for matches.

For example [2], before matching, we might have the two sequences

```
G A A T T C A G T T A
G G A T C G A
```

A possible match might be

```
G A A T T C A G T T A
G G A   T C   G     A
```

for which we would earn six rewards, less one mismatch penalty (shown italic) and four gap penalties. (We will consider all rewards and penalties as having positive or zero values, with the accounting done by *adding* rewards and *subtracting* penalties.)

Needleman and Wunsch [1] first pointed out that this problem is amenable to solution by DP, allowing *all possible* matchings to be scored, and the highest-scoring one identified. The clever idea is to form a two-dimensional array with the two sequences defining the columns and rows. In the above example, this looks like

		G	A	A	T	T	C	A	G	T	T	A
	0											
G												
G												
A												
T												
C												
G												
A												

A matching consists of a path through the (initially) empty boxes in the above tableau, starting in the box labeled by a zero and moving, in each step, either one box right, one box down, or diagonally down and right. A right or down move corresponds to using up a letter in the first or second sequence (respectively) without using up one in the second or first (respective) sequence. Therefore it corresponds to inserting a gap and incurs a gap penalty. A diagonal move corresponds to pairing a new character in each sequence. Therefore, it incurs either the matching reward, if the two sequences match, or a mismatch penalty, if they don't.

Also useful is to distinguish right or down moves between two boxes in the bordering rows or columns (respectively) of the table from right or down moves in the interior of the table. The former kind don't open gaps in either sequence, but merely allow the sequences to shift with respect to one another. So we might assess a smaller, or zero, penalty for these overall "skews."

Now, characteristic of dynamic programming, you just fill in the boxes with the total score of the *best* way to reach that box, either from above it, from its left, or from the upper-left diagonal neighbor. A score is computed, naturally, by taking the score of a predecessor box and then adding the reward (or subtracting the penalty) associated with the move. Starting at the upper left, there are always boxes ready to be filled in as you work your way, by rows or columns, to the lower right.

Also characteristic of dynamic programming, when you are done filling in all the boxes, you do a backtrack pass: Start at the lower right box in the table. Now figure out the path back through the table that contributed to that best score. (It may not be unique.) Finally, translate that path into the series of letters and gaps that it implies.

A straightforward implementation of this is the following routine. Note that the matching reward is normalized to one per matched character, while the penalties for mismatches, gaps, and overall skews are input arguments. You can set all three to zero in most cases. If you set a nonzero mismatch penalty, however, you will probably also want to have a gap penalty, since otherwise the program will always avoid a mismatch by creating two gaps, one in each string.

The routine's output includes a summary string showing where the matches, mismatches, and gaps occur. For the example above, the summary string is

```
=!= == = =
```

(You can change the symbols used to your liking.)

You can modify the program in various other ways. For example, you might want to have a bigger penalty for initially opening a gap than for extending it once

opened. This requires a more complicated logic in the initial filling of the cost table. As long as you are able to fill each box with the cost of the *best* way to reach it, then the logic of dynamic programming will still apply.

```
void stringalign(char *ain, char *bin, Doub mispen, Doub gappen,
    Doub skwpen, char *aout, char *bout, char *summary) {
```
stringalign.h

Given null terminated input strings ain and bin, and given penalties mispen, gappen, and skwpen, respectively, for mismatches, interior gaps, and gaps before/after either string, set null terminated output strings aout, bout, and summary as the aligned versions of the input strings, and a summary string. User must supply storage for the output strings of size equal to the sum of the two input strings.

```
    Int i,j,k;
    Doub dn,rt,dg;                          Cost of down, right, and diagonal moves.
    Int ia = strlen(ain), ib = strlen(bin);
    MatDoub cost(ia+1,ib+1);                Cost table, as illustrated in the text.
```

First we fill in the cost table.

```
    cost[0][0] = 0.;
    for (i=1;i<=ia;i++) cost[i][0] = cost[i-1][0] + skwpen;
    for (i=1;i<=ib;i++) cost[0][i] = cost[0][i-1] + skwpen;
    for (i=1;i<=ia;i++) for (j=1;j<=ib;j++) {
        dn = cost[i-1][j] + ((j == ib)? skwpen : gappen);
        rt = cost[i][j-1] + ((i == ia)? skwpen : gappen);
        dg = cost[i-1][j-1] + ((ain[i-1] == bin[j-1])? -1. : mispen);
        cost[i][j] = MIN(MIN(dn,rt),dg);
    }
```

Next, we do the backtrack pass, writing the output (backward, however).

```
    i=ia; j=ib; k=0;
    while (i > 0 || j > 0) {
        dn = rt = dg = 9.99e99;             Any large value will do.
        if (i>0) dn = cost[i-1][j] + ((j==ib)? skwpen : gappen);
        if (j>0) rt = cost[i][j-1] + ((i==ia)? skwpen : gappen);
        if (i>0 && j>0) dg = cost[i-1][j-1] +
            ((ain[i-1] == bin[j-1])? -1. : mispen);
        if (dg <= MIN(dn,rt)) {             Diagonal move produces either match or in-
            aout[k] = ain[i-1];                 equality.
            bout[k] = bin[j-1];
            summary[k++] = ((ain[i-1] == bin[j-1])? '=' : '!');
            i--; j--;
        }
        else if (dn < rt) {                 Down move produces a gap in the B string.
            aout[k] = ain[i-1];
            bout[k] = ' ';
            summary[k++] = ' ';
            i--;
        }
        else {                              Right move produces a gap in the A string.
            aout[k] = ' ';
            bout[k] = bin[j-1];
            summary[k++] = ' ';
            j--;
        }
    }
```

Finally, reverse the output strings.

```
    for (i=0;i<k/2;i++) {
        SWAP(aout[i],aout[k-1-i]);
        SWAP(bout[i],bout[k-1-i]);
        SWAP(summary[i],summary[k-1-i]);
    }
    aout[k] = bout[k] = summary[k] = 0;     Don't forget the terminating nulls!
}
```

Various modifications of the Needleman-Wunsch method are also in use, most

notably the generalization by Smith and Waterman [3]. There are also a number of heuristic methods for identifying sequence similarity, with names like BLAST, FASTA, BLAT, etc. The field is highly developed, so you should use the routine above only pedagogically.

CITED REFERENCES AND FURTHER READING:

Cybenko, G. 1997, "Dynamic Programming: A Discrete Calculus of Variations," *IEEE Computational Science and Engineering*, vol. 4, no. 1, pp. 92-97.

Bertsekas, 2001, *Dynamic Programming and Optimal Control*, 2nd ed., 2 vols. (Belmont, MA: Athena Scientific).

Hillier, F.S. and Lieberman, G.J. 2002, *Introduction to Operations Research*, 7th ed. (New York: McGraw-Hill).

Needleman, S.B. and Wunsch, C.D. 1970, "A General Method Applicable to the Search for Similarities in the Amino Acid Sequence of Two Proteins," *Journal of Molecular Biology*, vol. 48, pp. 443-453.[1]

Rouchka, E.C. 2001, "Dynamic Programming," at multiple Web sites.[2]

Smith, T.F. and Waterman, M.S. 1981, "Identification of Common Molecular Subsequences," *Journal of Molecular Biology*, vol. 147, pp. 195-197.[3]